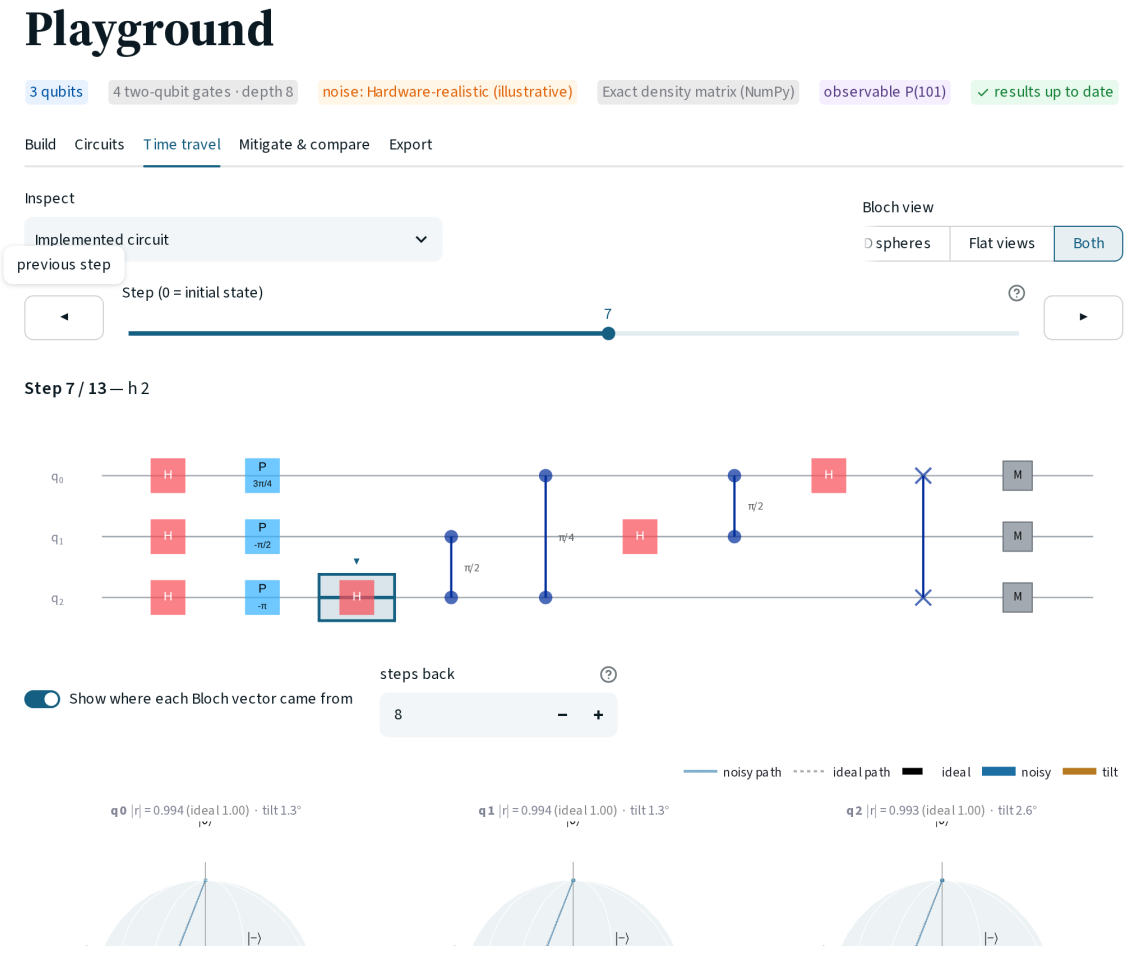
QEM Explorer

User manual, version 1.2

Every feature, the theory behind it, and step-by-step tutorials for an interactive laboratory of quantum error mitigation: zero-noise extrapolation, probabilistic error cancellation, learning-based mitigation, readout mitigation, symmetry verification, virtual distillation, Pauli twirling and dynamical decoupling — on simulators, fake IBM devices and IBM Quantum hardware; with analysis views matched to each result, equal-budget cost accounting, error-correcting codes, Grover search and the hidden subgroup problem, batch studies and a research handbook.



Built with Python, Streamlit, Plotly, Qiskit and Qiskit Aer. All presets and example values are illustrative; none are measured results.

October 3, 2026

Contents

Part I · Getting started **3**
What QEM Explorer is . . . 3
Installation and first run . . . 3
A tour of the interface . . . 4

Part II · Step-by-step tutorials **7**
Tutorial 1 — Your first comparison in five minutes . . . 7
Tutorial 2 — Building circuits . . . 8
Tutorial 3 — What does each method actually run? . . . 11
Tutorial 4 — Time travel through the state . . . 12
Tutorial 5 — Reading a comparison in depth . . . 16
Tutorial 6 — A real device model . . . 18
Tutorial 7 — Learning-based mitigation that cannot collapse . . . 19
Tutorial 8 — Error-correction primitives . . . 20
Tutorial 9 — Cost vs precision and the PEC wall . . . 21
Tutorial 10 — The bias–variance bullseye . . . 23
Tutorial 11 — When the model is wrong . . . 24
Tutorial 12 — Head-to-head sweeps . . . 25
Tutorial 13 — Exporting your work . . . 27
Tutorial 14 — Running on IBM Quantum hardware . . . 28

Part II-B · New in version 1.1 **30**
Tutorial 15 — Logical versus physical: what the compiler did . . . 30
Tutorial 16 — Topology, SWAPs and the lowest-noise qubits . . . 31
Tutorial 17 — Hamiltonian observables . . . 32
Tutorial 18 — Live parameters and energy landscapes . . . 33
Tutorial 19 — Crosstalk and dynamical decoupling . . . 35
Tutorial 20 — The analysis panel: reading each result the right way . . . 36
Tutorial 21 — The five-qubit code and a surface-code plaquette . . . 38
Tutorial 22 — Batch studies: which method wins where? . . . 41
Tutorial 23 — Sweeps: inputs, measurement bases and parameters . . . 43
Tutorial 24 — Learn, then look it up in the handbook . . . 46

Part II-C · New in version 1.2 **48**
Tutorial 25 — Equal budgets: reading the cost table . . . 48
Tutorial 26 — The same experiment on hardware . . . 49
Tutorial 27 — One ZIP with everything . . . 51
Tutorial 28 — Inputs, bases and π in the composer . . . 52
Tutorial 29 — Grover's search and the hidden subgroup problem . . . 53
Tutorial 30 — The redesigned handbook . . . 54

Part III · Reference **57**
Noise injector parameters . . . 57
Text syntax . . . 57
Composer interactions . . . 58
The comparison table . . . 59
Templates and their parameters . . . 59
Methods and their parameters . . . 62

Preset contenders . . . 65
Noise presets . . . 65
Circuits you can inspect . . . 66

Part IV · Theory and sources — a research handbook **67**
1. Notation and conventions . . . 67
2. Noise . . . 68
3. What error mitigation can and cannot do . . . 70
4. Zero-noise extrapolation (ZNE) . . . 71
5. Probabilistic error cancellation (PEC) . . . 72
6. Learning-based mitigation: CDR and NIL . . . 74
7. Readout mitigation . . . 75
8. Pauli twirling and randomized compiling . . . 75
9. Dynamical decoupling . . . 76
10. Symmetry verification . . . 76
11. Virtual distillation . . . 76
12. Test circuits . . . 76
13. Compilation . . . 86
14. The simulators . . . 86
15. Running on hardware . . . 86
16. Advanced mitigation methods . . . 87
17. Toward fault tolerance: a QEC roadmap . . . 90
References . . . 94

Part V · Security, robustness and troubleshooting **100**
Security measures . . . 100
Troubleshooting . . . 101
Testing . . . 101

Part I · Getting started

What QEM Explorer is

QEM Explorer is an interactive laboratory for **quantum error mitigation** (QEM). A noisy quantum computer returns $\langle O\rangle_{\text{noisy}} = \mathrm{Tr}(\mathcal{E}(\rho)O)$ instead of the ideal $\langle O\rangle = \mathrm{Tr}(\rho O)$. Error mitigation does not correct the device; it runs more circuits, or differently prepared ones, and combines their results classically so that the *estimate* of $\langle O\rangle$ is less biased. The price is paid in variance: more shots are needed for the same statistical precision. The whole app revolves around one equation,

$$\mathrm{MSE}(N) = \underbrace{\left(\mathbb{E}[\hat{O}] - O\right)^2}_{\text{bias}^2} + \underbrace{\frac{\gamma^2\sigma^2}{N}}_{\text{variance}},$$

and lets you see, for every method and every circuit, which of the two terms you are paying.

What you can do:

- **Build** a circuit from 23 templates in five families (basics, algorithms, NISQ variational circuits, error-correcting codes, deep fault-tolerant algorithms) or by drag and drop, as a table or as text; measure a bitstring probability, a parity or any **Hamiltonian** $\sum_k c_k P_k$ you type.
- **Inject** exactly the noise you want (15 noise parameters including crosstalk, 10 presets, noise channels placed in the circuit), or choose a **fake IBM device** whose calibration supplies the noise.
- **Look** at the circuits each method really executes, see how the compiler turns every logical gate into native pulses and SWAPs on a real **coupling map**, find the lowest-error physical qubits, and **scrub** through the quantum state gate by gate — or move variational angles with **live sliders**.
- **Compare** any number of mitigation recipes at the **same total shot budget** — target, training and calibration shots counted together — on bias, variance, RMSE, sampling overhead, circuits sent and QPU time, and read the result through the **view that matches its mathematics**: an output histogram for probabilities, estimates with error bars for expectation values, logical error and syndromes for codes, and each method's diagnostic curve.
- **Analyze** cost against precision, the bias–variance trade-off, robustness to a wrong noise model and head-to-head performance over a noise parameter.
- **Study** whole grids of algorithms × methods × noise models × noise scales with one *Run all*.
- **Learn** with hands-on guides that set up the Playground for you, and a theory handbook that runs from the noise models to the road to fault tolerance.
- **Run** the same experiment on IBM Quantum hardware, and **export** a standalone Qiskit script.

> All presets and example numbers in the app and in this manual are **illustrative**. They are not measured results of any device or of any thesis.

Installation and first run

Requirements. Python 3.10–3.13. The packages are listed in `requirements.txt` (Streamlit ≥ 1.64, Plotly, NumPy, SciPy, pandas, scikit-learn, Matplotlib, Qiskit ≥ 1.0, Qiskit Aer, qiskit-ibm-runtime, Kaleido). `requirements-lock.txt` pins the exact versions that were tested.

Step by step (terminal).

1. Unzip `qem-explorer.zip` and open a terminal in the folder.
2. Create and activate a virtual environment:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS / Linux
```

3. Install the dependencies: `python -m pip install -r requirements-dev.txt`
4. Start the app: `python -m streamlit run app.py`
5. Your browser opens `http://localhost:8501`. If it does not, open that address yourself.

Step by step (VS Code).

1. *File* → *Open Folder*… and choose the unzipped folder.
2. Install the recommended *Python* extension when VS Code offers it.
3. Command palette (Ctrl/Cmd + Shift + P) → *Python: Create Environment* → *Venv* → your Python → tick `requirements-dev.txt`.
4. Open *Run and Debug* (Ctrl/Cmd + Shift + D), choose **QEM Explorer (Streamlit app)** and press **F5**. The launch configuration shows full error details, which the deployed configuration hides.
5. The same menu has **Tests (fast)** and **Tests (all, incl. app and export)**.

High-resolution figures. PNG and SVG export uses Kaleido, which needs Chrome or Chromium on the machine (`packages.txt` installs Chromium on Streamlit Community Cloud). Without it, every download still contains the Plotly JSON of each figure (no HTML files are written).

Deployment. Any host with Python works; on Streamlit Community Cloud point the app at `app.py`. The shipped `.streamlit/config.toml` hides stack traces from visitors, enables XSRF protection and limits uploads to 20 MB.

A tour of the interface

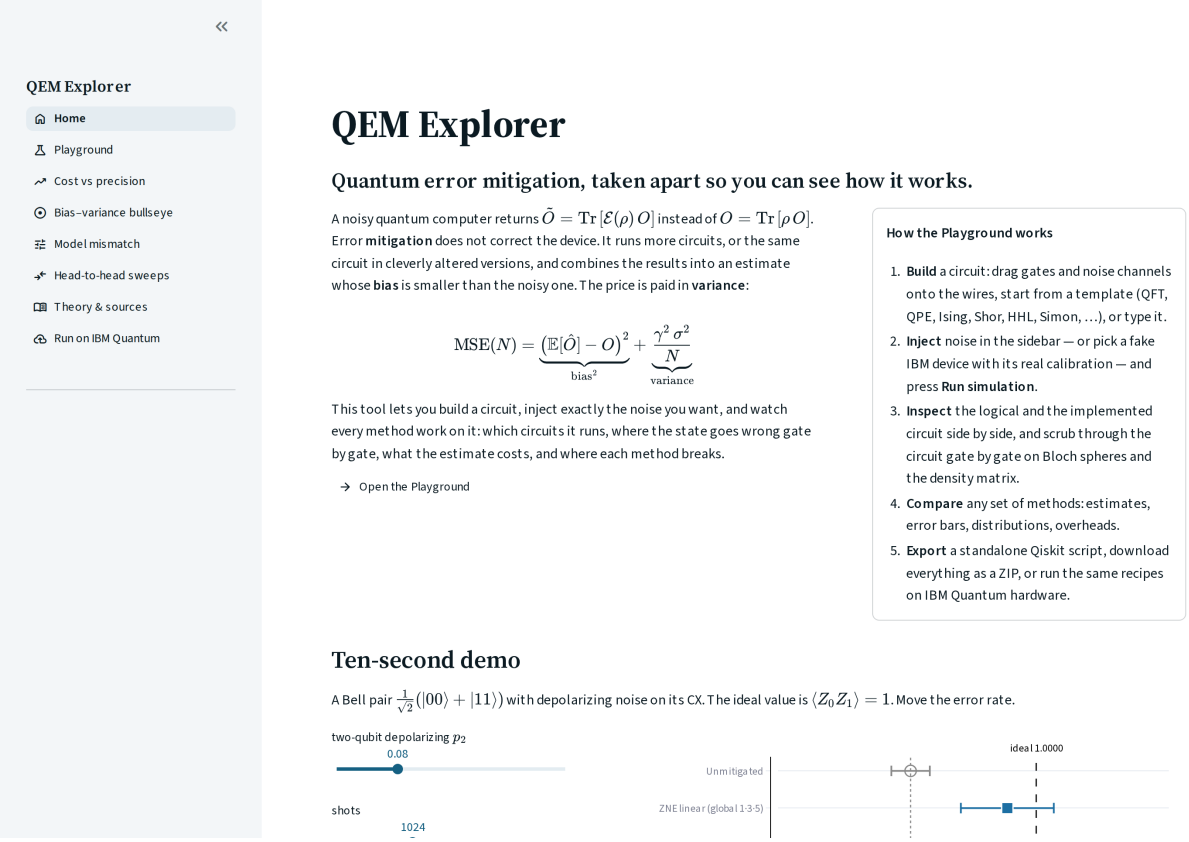


Figure 1: The landing page: the trade-off in one equation, a ten-second live demo and the method map.

Navigation. The left sidebar lists the ten pages: *Home, Learn: guides & algorithms, Playground, Batch run, Cost vs precision, Bias–variance bullseye, Model mismatch, Head-to-head sweeps, Theory & sources, Run on IBM Quantum*. Every page keeps its settings when you leave it and come back; so do the tabs inside a page. The Playground has six tabs: *Build, Circuits, Topology & layout, Time travel, Mitigate & compare, Export*.

Simulator and noise (sidebar).

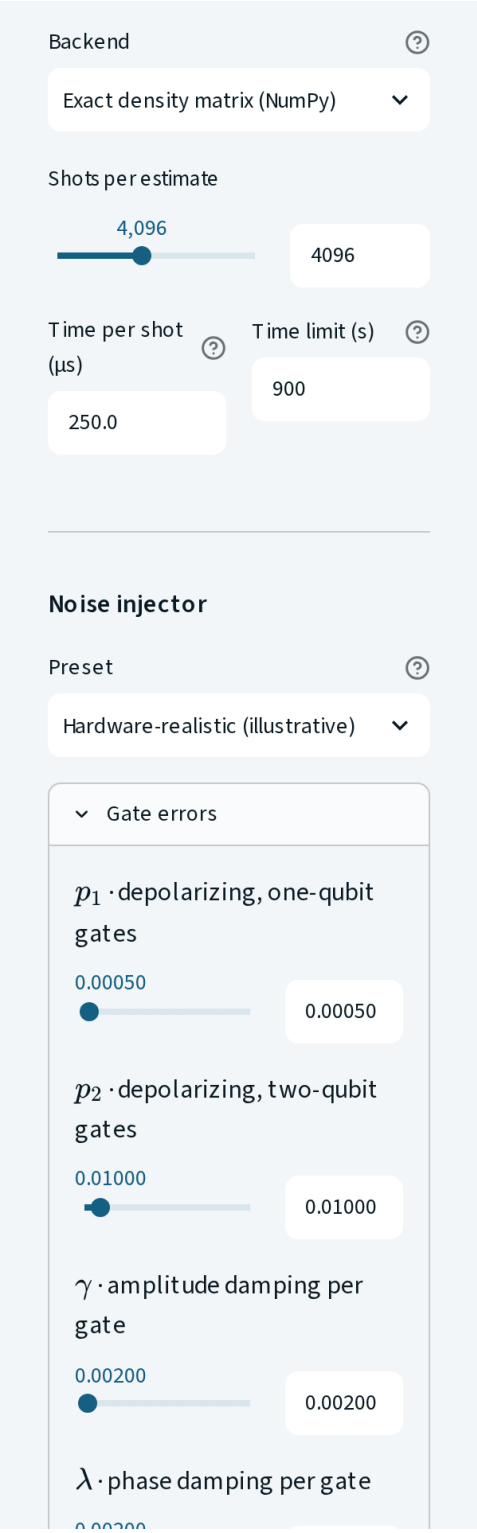


Figure 2: The sidebar: Run button, simulator, shots, time per shot, time limit and the noise injector.

- **Run simulation** starts all heavy work — the one button for the Playground's comparison

(time travel, every contender, the analysis), also used by *Cost vs precision*. Changing a setting never starts a simulation by itself; results that no longer match the inputs are marked *out of date* (a red badge on the Playground, a warning elsewhere). Pages with an experiment of their own (a sweep, a mismatch curve) have one button that names it.

- **Backend**: *Exact density matrix* (*NumPy*), *Qiskit Aer* (*density_matrix*), or one of twelve *fake devices* (FakeFez, FakeTorino, FakeMarrakesh, FakeKingston, FakeAachen, FakeBrisbane, FakeSherbrooke, FakeKyiv, FakeOsaka, FakePrague, FakeAlgiers, FakePerth).
- **Shot budget per method** — the total number of shots B every contender may spend on one estimate: target, training and calibration circuits together (Part IV, §3.1). A slider for quick exploration next to a number field for exact values (every numeric control in the app works this way).
- **Time per shot (µs)** converts shots into QPU seconds. **Time limit (s)** stops a run cleanly when it takes too long (0 = no limit).
- **Noise injector** — a preset and 15 parameters in five groups (gate errors, coherent errors, crosstalk, idle qubits, readout); see the reference in Part III. A caption shows the datasheet view, the average gate infidelity $r = (d-1)p/d$.

Badges. The chips under the Playground title summarize the state: number of qubits, two-qubit gates and depth after compilation, noise preset or device, simulator, observable, and whether results are up to date.

"Why this happens?" Every major figure has an expander with that title. It explains, with equations, why the figure looks the way it does. Metric cards and table headers carry a **?** tooltip with their formula, written in Unicode math ($\sqrt{(b^2 + v/N)}$, $\gamma = \sum_i |q_i|$) so it reads correctly in the small popup.

Downloads. Each figure has an *Export* menu (interactive HTML, CSV of its data, PNG at 3× resolution, SVG). Each page has a **Download all (ZIP)** button that bundles its tables, samples, figures and the state; the Playground's ZIP holds every cross-page result of the comparison (Tutorial 27).

Part II · Step-by-step tutorials

Tutorial 1 — Your first comparison in five minutes

1. Open **Playground**. The *Build* tab shows the QFT template: $F\,F^{\dagger}|5\rangle = |5\rangle$ on three qubits, with the observable $P(101)$, whose ideal value is 1.
2. In the sidebar keep the preset *Hardware-realistic* (*illustrative*) and press **Run simulation**. A status box reports the progress (time travel first, then every contender).
3. Open the tab **Mitigate & compare**. The four cards show the ideal value, the unmitigated estimate with its bias, the contender with the lowest RMSE at your shot budget and its RMSE.
4. Read the **Estimates** chart: each marker is a method's mean, the bar its $\pm 1\sigma$ at the chosen number of shots, the dashed line the ideal value. A method whose bar does not reach the dashed line is biased; a long bar means high variance.
5. Scroll to the **Cost and accuracy** table. Every method spends the same total budget, so the table is an equal-cost comparison: bias, sd, RMSE, sampling overhead γ, circuits sent to the backend, extra qubits, post-selection acceptance, target / training / calibration shots, QPU seconds and — if something needs your attention — a *status* flag next to the method's name (Tutorial 25).

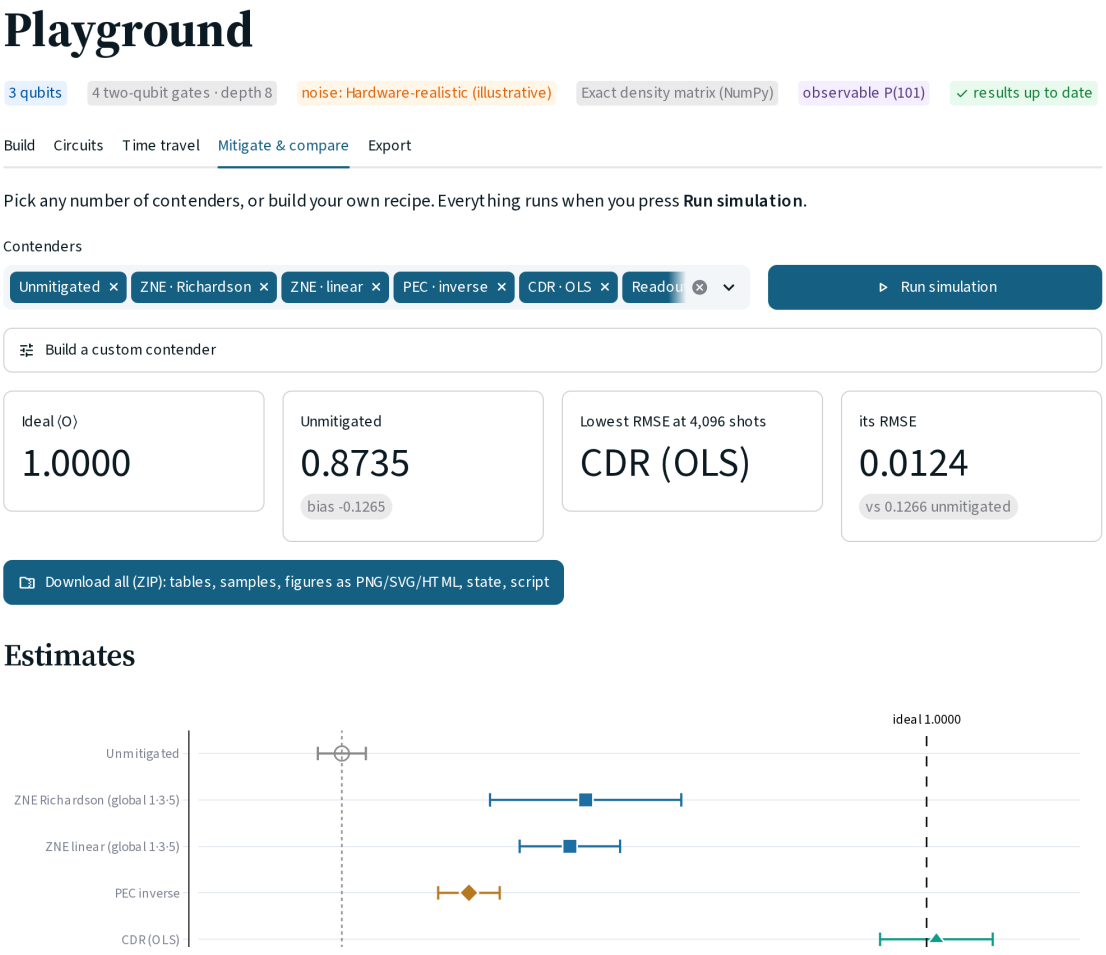


Figure 3: Mitigate & compare: headline cards and the estimates chart.

Tutorial 2 — Building circuits

From a template. In *Start from a template* choose a **Family** and a **Template**, set its parameters (all typed numbers; bounds follow the other parameters, e.g. $s < 2^n$) and press **Load template**. Loading replaces the circuit; *Undo* in the composer brings the previous one back. The card shows the template's equation and notes.

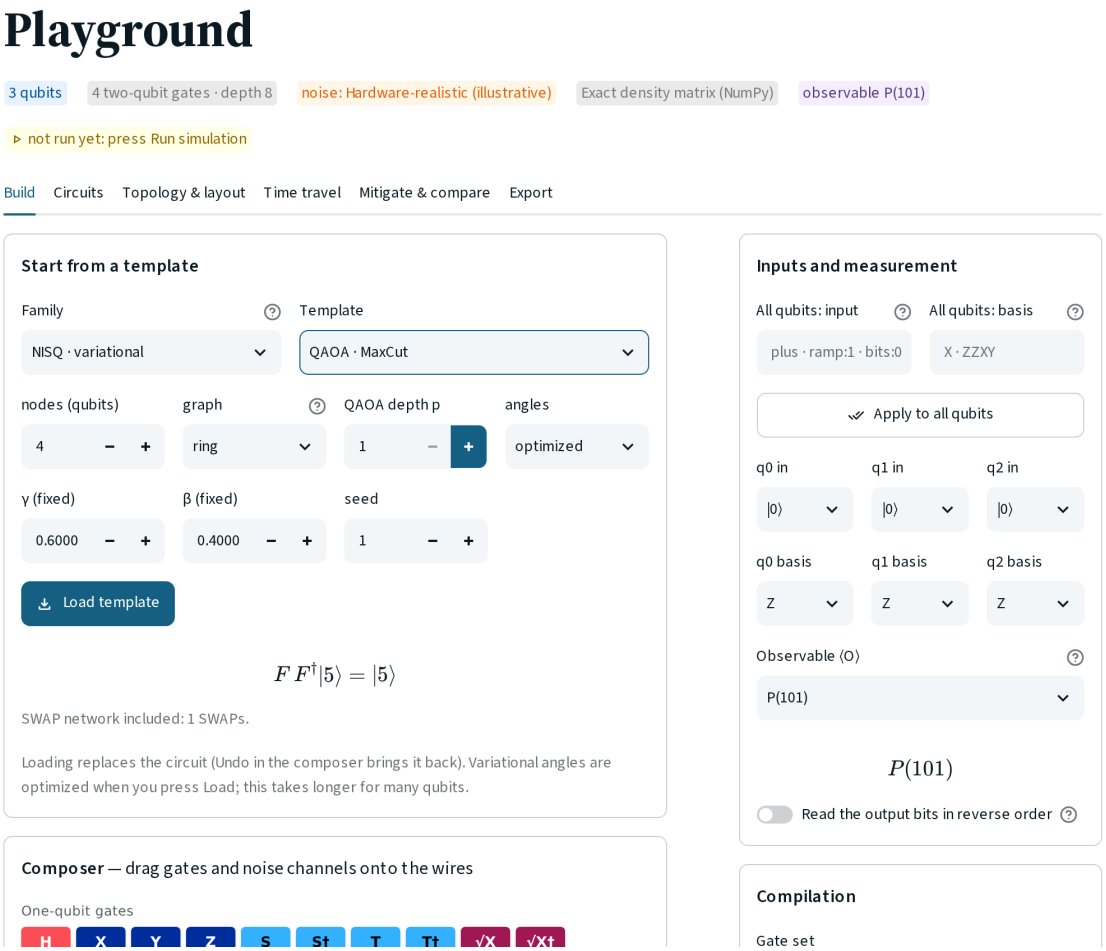


Figure 4: The template card with a NISQ template (QAOA MaxCut) selected.

The five families are:

Family	Templates
Basics	Bell pair, GHZ state, Grover (2 qubits)
Algorithms	QFT, approximate QFT, phase estimation, QPE of an Ising Hamiltonian, Trotterized Ising, Bernstein–Vazirani, Simon
NISQ · variational	VQE with a hardware-efficient ansatz, QAOA for MaxCut, QML ZZ-feature-map classifier, data re-uploading classifier
FTQC · error correction	3-qubit bit-/phase-flip code, repetition-code memory, surface code d = 2 (one plaquette), five-qubit code [[5,1,3]], Steane [[7,1,3]] code
FTQC · deep algorithms	Shor order finding for N = 15 (four depth variants), Grover's search (any n, any marked item), the hidden subgroup problem (Z_2^n and Z_N), HHL for a 2×2 system

Variational templates (VQE, QAOA) can optimize their angles when you press *Load*: the app minimizes the energy (or maximizes the cut) on a noiseless statevector, so the ideal value is the optimum

and every deviation in the comparison is caused by noise. The optimization respects the sidebar's time limit.

Drag and drop. The composer works like IBM Quantum Composer:

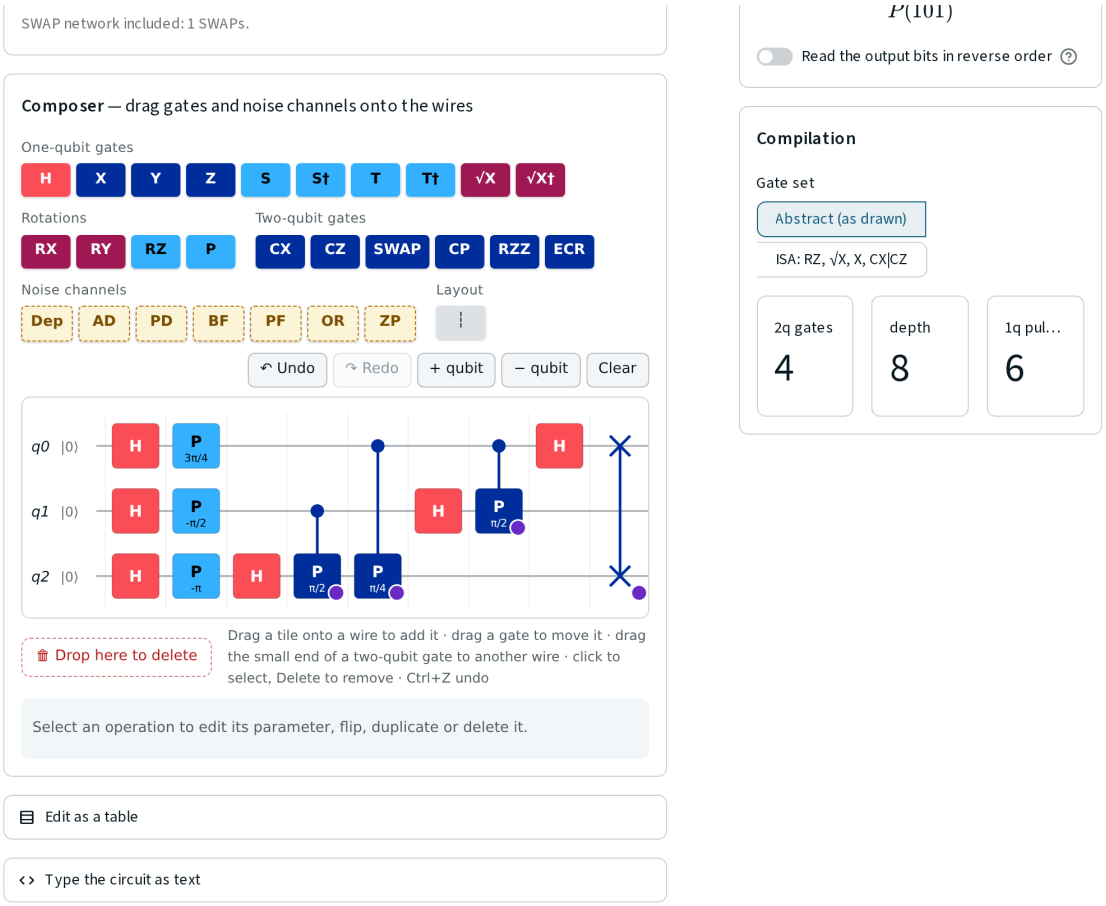


Figure 5: The drag-and-drop composer with its palette of gates, rotations, two-qubit gates, noise channels and barriers.

- drag a tile from the palette onto a wire to add it at that position (a CNOT is dropped on its control wire);
- drag a gate to move it; drag the small violet handle of a two-qubit gate to another wire to retarget it;
- drop a gate on **Drop here to delete** to remove it; click a gate to select it (Delete removes it);
- double-click a rotation to type its angle (numbers or expressions such as `pi/4`);
- **Undo / Redo**, **+ qubit / − qubit**, **Clear**. Removing a qubit that carries operations tells you how many were removed (Undo restores them).

Every edit is validated by the Python side before it changes the circuit; an invalid edit is refused with a message and never corrupts the state.

As a table or as text. *Edit as a table* lists one operation per row (operation, qubits, parameter). *Type the circuit as text* accepts one operation per line or separated by `;`, e.g. `h 0; cx 0 1; rz(pi/8) 1; depol(0.05) 0 1`. Parameters are evaluated by a safe expression reader — nothing you type is ever executed as code.

Inputs and measurement. Each qubit can start in $|0\rangle, |1\rangle, |+\rangle, |-\rangle, |+i\rangle, |-i\rangle$ and be measured in the Z, X or Y basis. The **Observable** menu offers every $\langle Z_i \rangle$, neighboring $\langle Z_i Z_{i+1} \rangle$, the full parity, the magnetization, the template's own observable, and $P(bitstring)$... for any bitstring you type, and *Hamiltonian: a sum of Pauli strings*... (Tutorial 17). *Read the output bits in reverse order* relabels the classical bits (the QFT without SWAPs).

Compilation. *Abstract (as drawn)* runs the gates you placed. *ISA* transpiles to RZ, $\sqrt{\text{X}}$, X and CX or CZ on a coupling map (all-to-all, line, ring, 2D grid or IBM heavy-hex) with SABRE qubit selection and routing and an optimization level 0–3. The switch *Show the compiled (physical) circuit* draws the result under the logical circuit (Tutorial 15). With a fake device selected, the device's own target is used (next tutorial). The metrics show two-qubit gates, depth and one-qubit pulses after compilation.

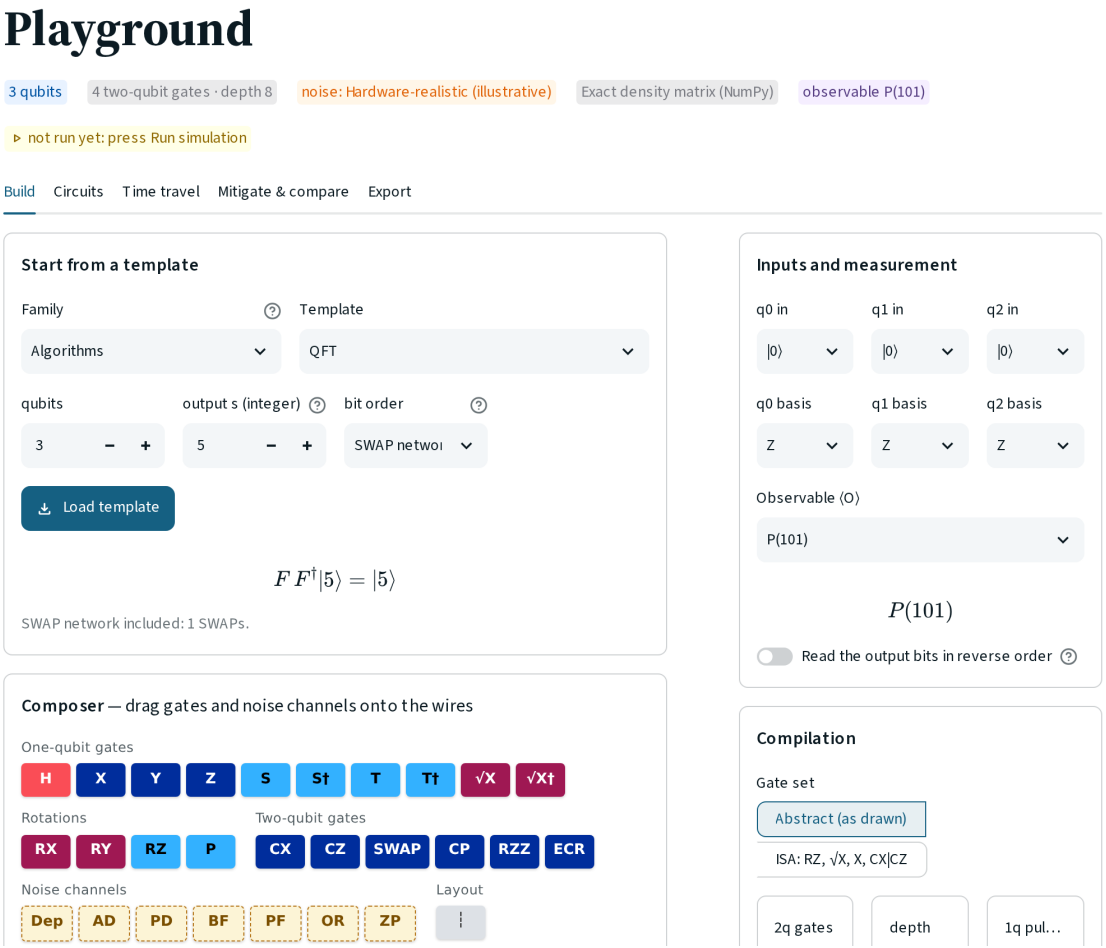


Figure 6: The Build tab: template card, composer, inputs and measurement, compilation.

Tutorial 3 — What does each method actually run?

Open **Circuits**. On the left is your logical circuit; on the right the circuit a method executes. Choose *Implemented circuit of*: compiled only, ZNE folding (the $U^\dagger U$ blocks are shaded; choose λ and global or local folding), a sampled PEC instance (amber insertions; *show an instance with insertions* redraws until one appears), a Pauli-twirl instance, DD pulses in idle windows, a CDR/NIL training circuit (changed angles in teal) or the virtual-distillation copies. Metrics count operations, two-qubit gates,

depth and the operations the method added. The drawing can be the interactive composer view or Qiskit's IBM-style drawing (for very long circuits the composer view is used automatically).

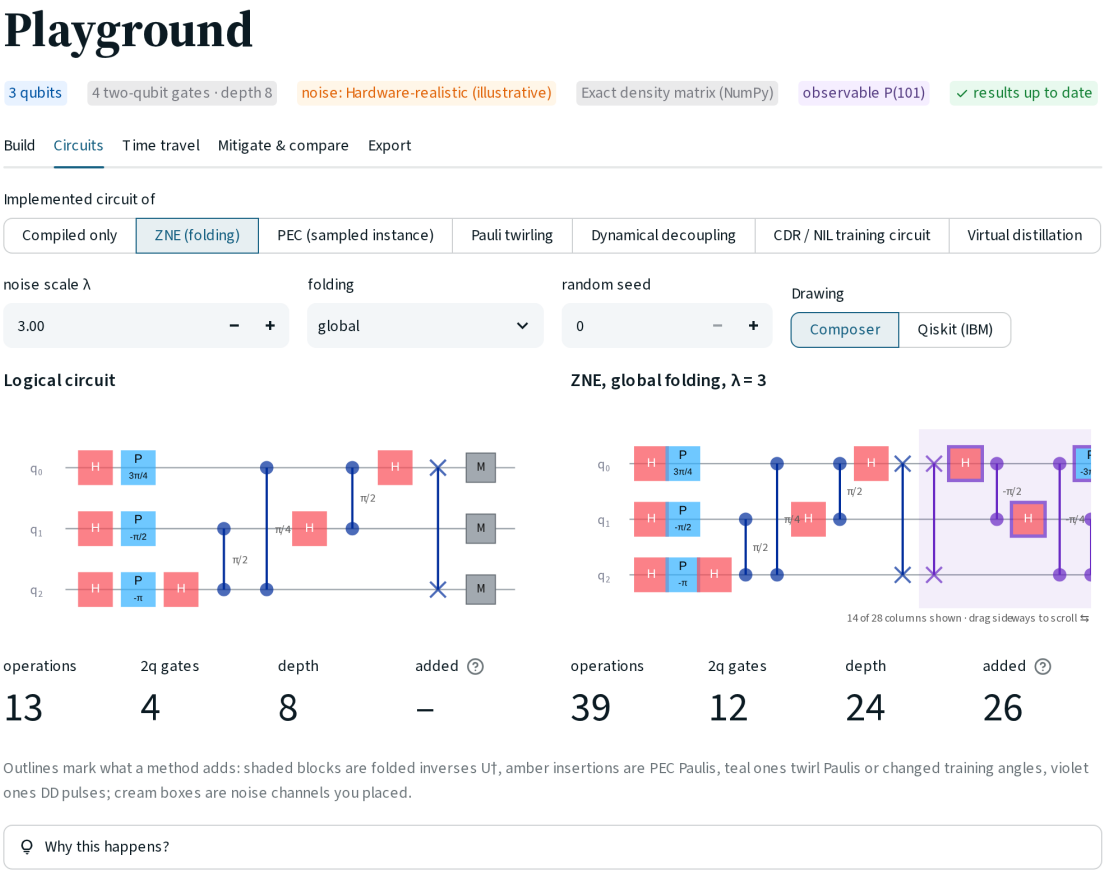


Figure 7: Circuits: the logical circuit next to the ZNE-folded circuit at λ = 3.

Tutorial 4 — Time travel through the state

1. Press **Run simulation**, then open **Time travel**.
2. Choose what to **Inspect**: the implemented circuit, a ZNE-folded circuit, the circuit with DD, or twirled noise. Each choice remembers its own position and starts at the final state.
3. Move with ◀ / ▶ or drag the **Step** slider. The operation executed in this step is boxed in blue — **only that gate and the wires it acts on**. A two-qubit gate on distant qubits gets one box per qubit and a band along its connector; idle wires are never highlighted.

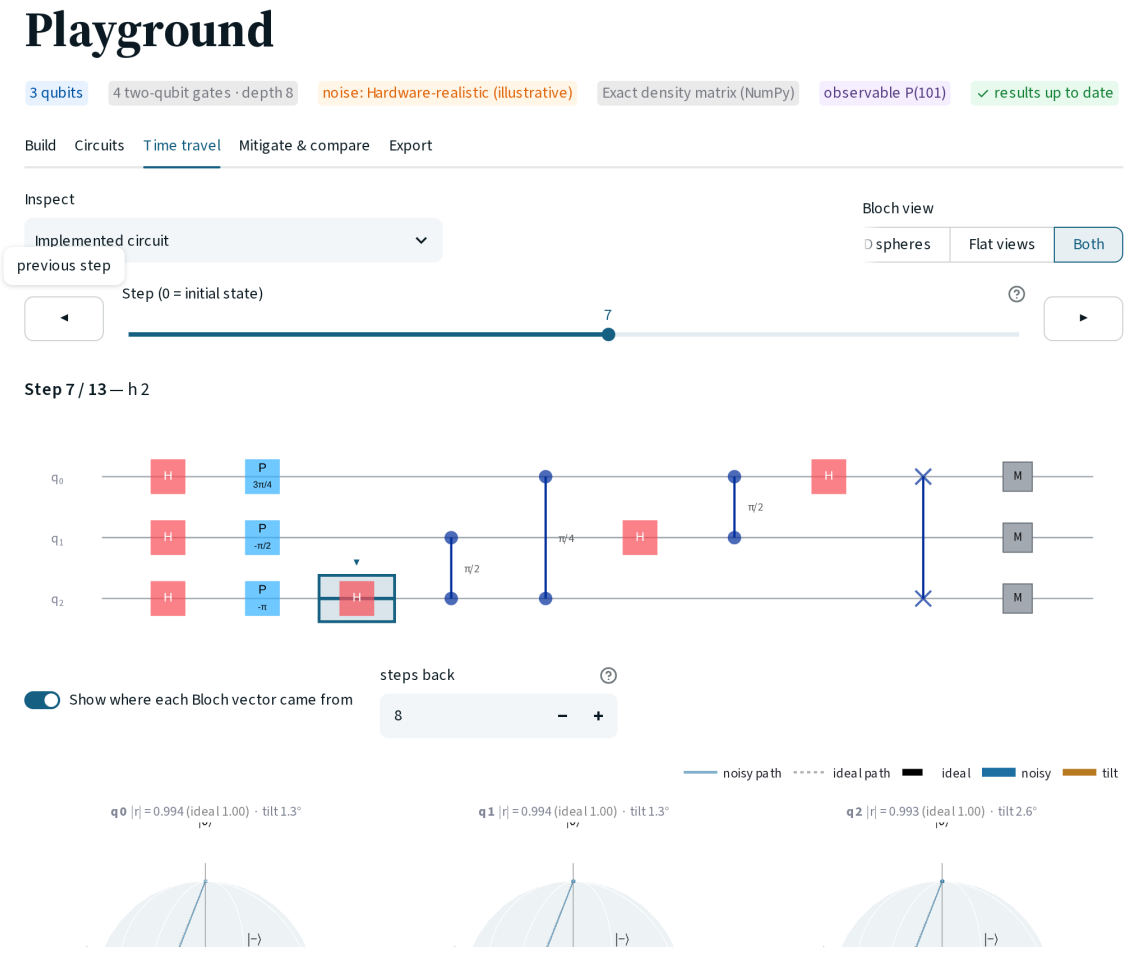


Figure 8: Time travel: the executed gate (here H on q2) is boxed; wires it does not touch stay unmarked.

4. Below the circuit, **Bloch spheres** show each qubit's reduced state: the noisy vector (blue) against the ideal one (black), the tilt between them (amber arc, in degrees) and a trail of where the vector came from. A coherent error *tilts* the vector; an incoherent error *shortens* it; entanglement shortens the ideal vector too, so only the comparison separates the two. *Flat views* project the same vectors on the XZ, YZ and XY planes and stay readable for many qubits.

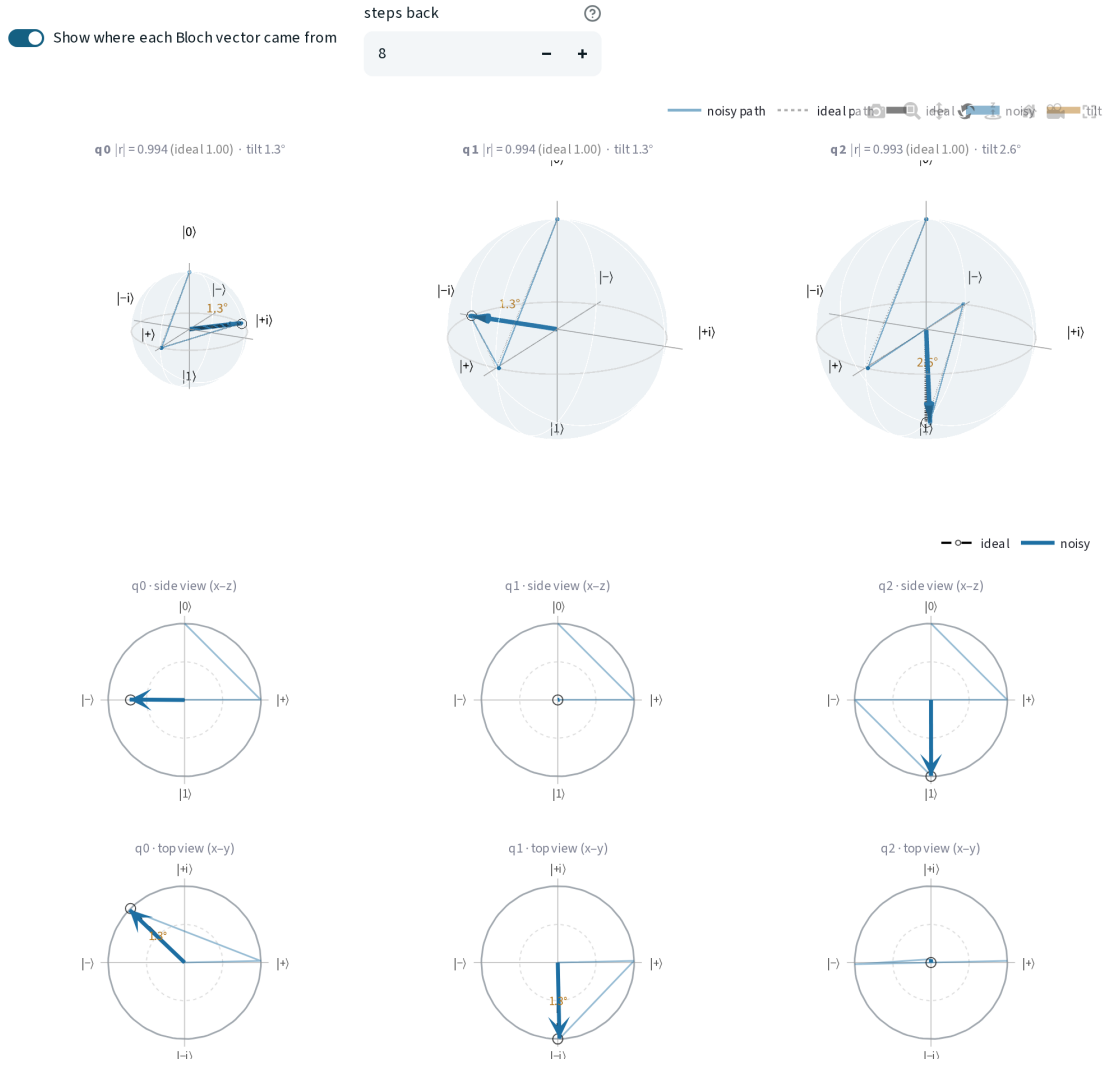


Figure 9: Bloch spheres with trails and flat projections.

5. The table lists $|\vec{r}|$, polar and azimuthal angles and the tilt for every qubit. Further down, the **density matrix** (magnitude as area, phase as color; or the error $\rho - \rho_{\text{ideal}}$) and the **fidelity and purity** along the circuit. On a folded circuit the fidelity keeps falling through the $U^\dagger U$ blocks although the ideal state returns — that is exactly the amplified noise ZNE measures.

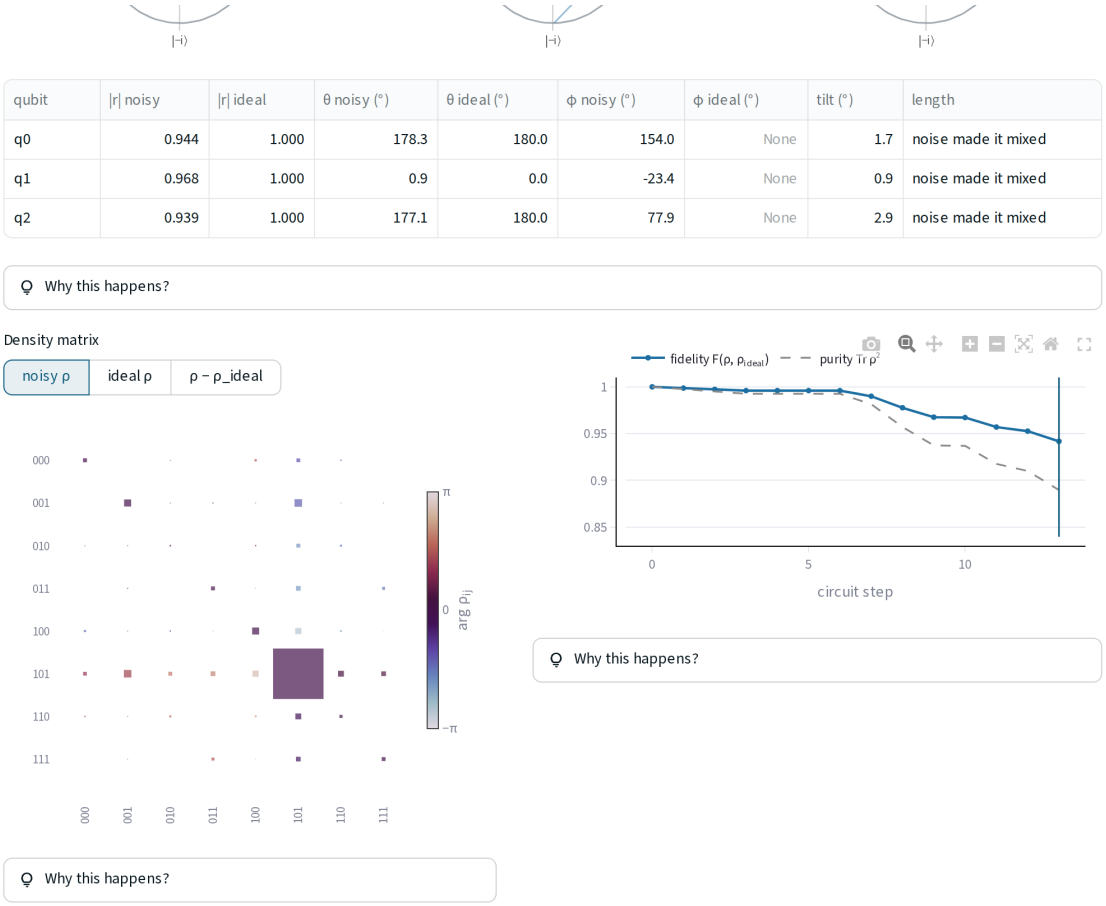
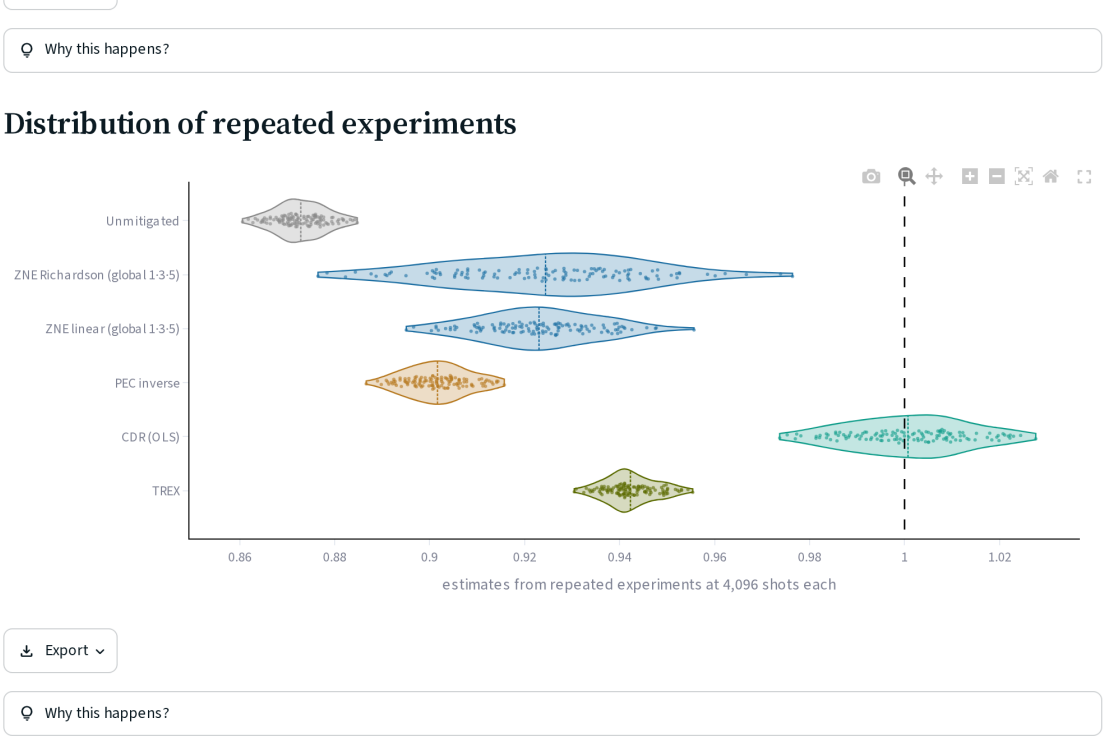


Figure 10: Density matrix and fidelity/purity along the circuit.

For large registers the app keeps per-step summaries only (a density matrix of n qubits has 4^n entries).

Tutorial 5 — Reading a comparison in depth



Cost and accuracy

method	estimate	bias	sd	sd from	sd (repeats)	RMSE	γ	circuits	extra qubits	accepted	training/cal. shots
Unmitigated	0.8735	-0.1265	0.0052	analytic	0.0052	0.1266	1.000	1	0	1.000	0
ZNE Richardson (global 1·3·5)	0.9262	-0.0738	0.0207	analytic	0.0203	0.0766	3.500	3	0	1.000	0
ZNE linear (global 1·3·5)	0.9228	-0.0772	0.0109	analytic	0.0120	0.0780	1.833	3	0	1.000	0
PEC inverse	0.9010	-0.0990	0.0067	analytic	0.0063	0.0993	1.083	1	0	1.000	0
CDR (OLS)	1.0021	+0.0021	0.0122	repeats	0.0122	0.0124	1.198	17	0	1.000	65536

Figure 11: The distribution of 120 repeated experiments per method.

- **Distribution of repeated experiments.** Each violin is the spread of 120 simulated experiments with your shot budget. It shows what one run could give you, not only the mean. Failed repeats (e.g. an exponential fit when $E(\lambda)$ changes sign) are counted and reported.
- **Cost and accuracy.** *sd* is analytic for estimators that are linear in the data and empirical for the learning family (the fit is refitted in every repeat). *QPU s* follows the sidebar's current time per shot. CSV and LaTeX downloads are next to the table.

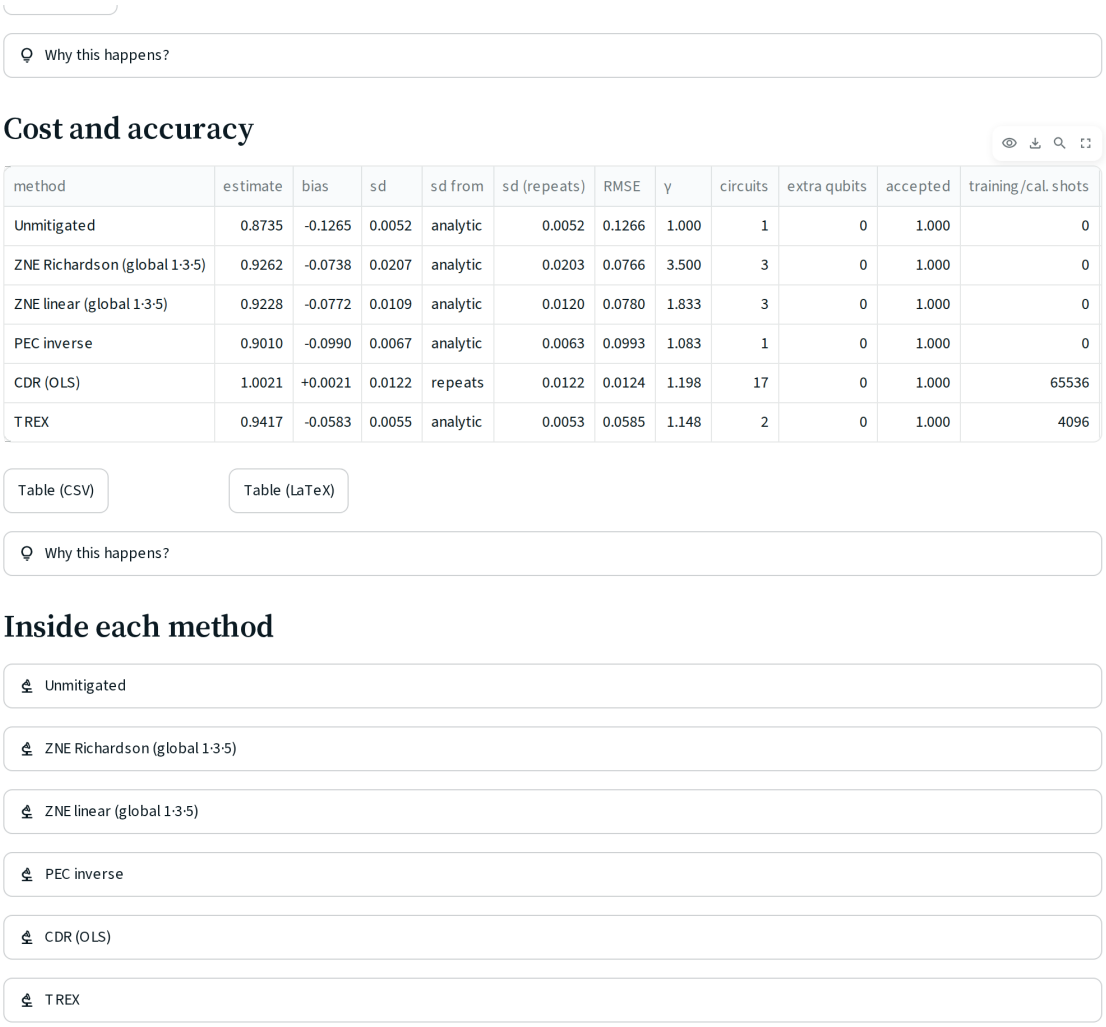


Figure 12: The cost table.

- **Inside each method.** One expander per contender: the formula, notes, and the method's internals — the ZNE fit with its weights η_j and optimal shot split, the PEC γ growth and quasi-probabilities, the learning map, the VD spectrum, the symmetry acceptance — and the circuit it ran.

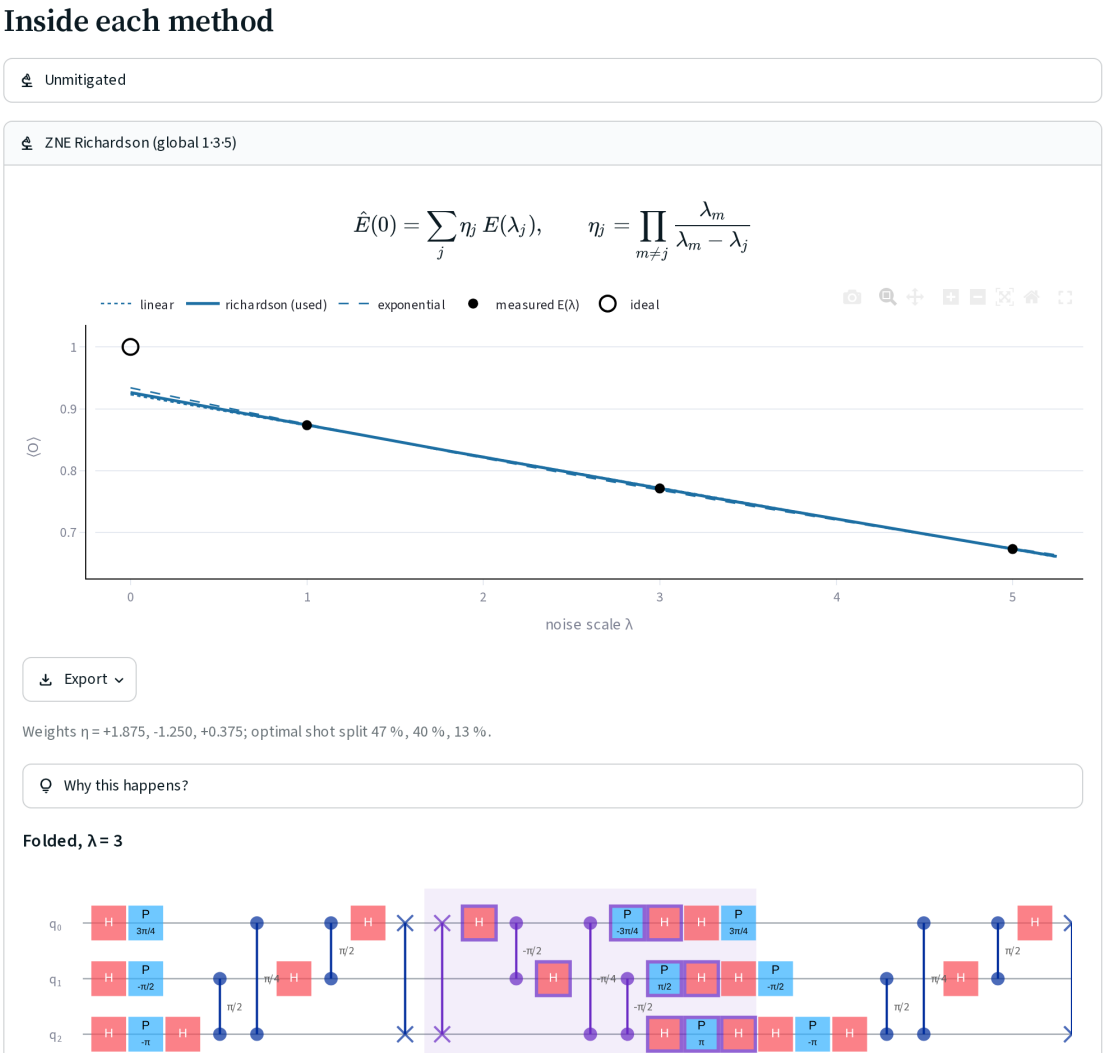


Figure 13: Inside ZNE: the extrapolation fit and the folded circuit.

Your own contenders. *Build a custom contender* combines an estimator (with all its parameters) with twirling, dynamical decoupling and readout mitigation, e.g. *ZNE* (*Richardson*) + *Pauli twirling* + *TREX*. Two identical contenders are kept apart as "(#2)".

Tutorial 6 — A real device model

1. In the sidebar choose **Backend → Fake device: FakeFez (156 qubits · Heron r2 · CZ)**.
2. The *Compilation* card switches to the device: the circuit is compiled for its coupling map and native gates, broken couplers are avoided, and the chosen **physical qubits** are shown with their calibration (T_1, T_2, gate and readout errors). You may pin physical qubits and set the transpiler seed and optimization level.
3. Press **Run simulation**. Gate errors (depolarizing + thermal relaxation over each gate's duration), idle thermal relaxation in the as-late-as-possible schedule and per-qubit readout errors come from the calibration snapshot; the density matrix covers only the physical qubits used.

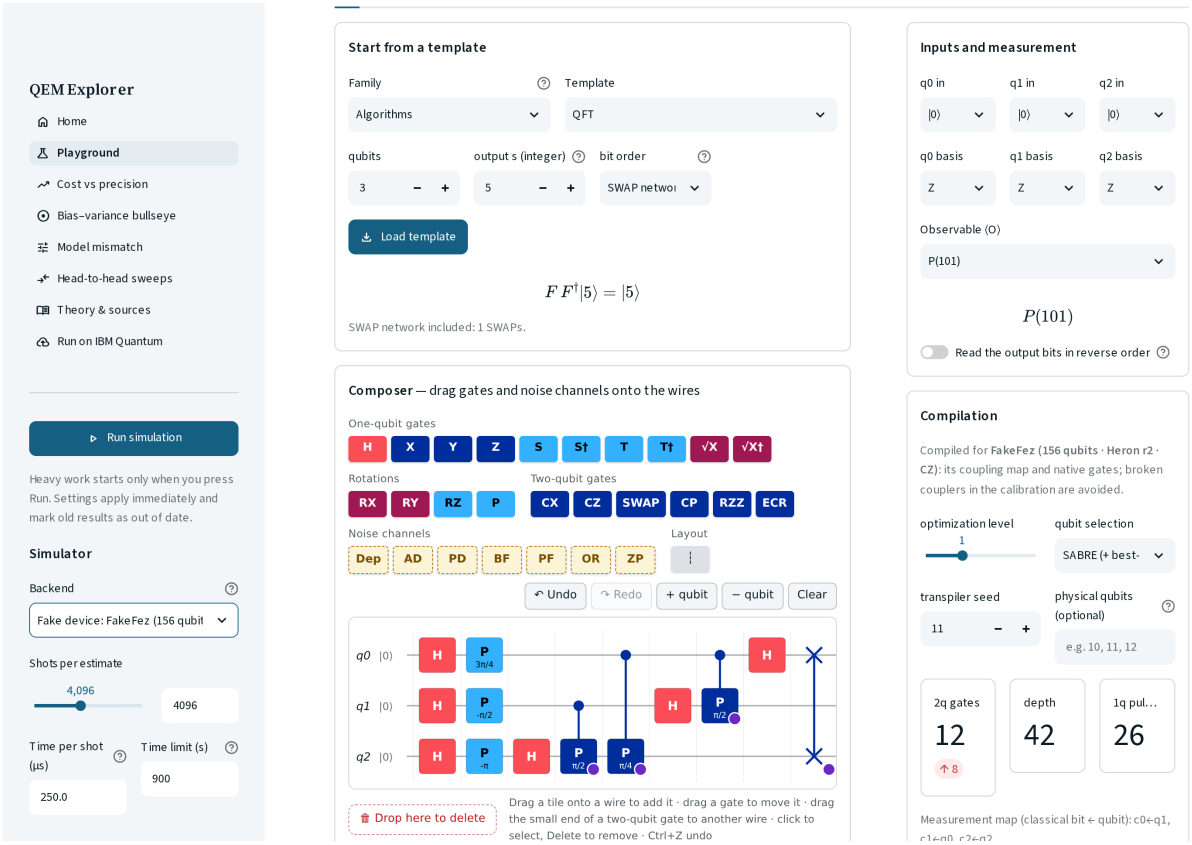


Figure 14: A fake device selected: device-aware compilation and calibration.

The device model is Markovian, so dynamical decoupling shows only its cost on a fake device; the injector's quasi-static field shows the case where DD helps. PEC and TREX are told the device's median error rates.

Tutorial 7 — Learning-based mitigation that cannot collapse

Clifford data regression (CDR) learns $y_{\text{ideal}} \approx b + c\, x_{\text{noisy}}$ from training circuits whose non-Clifford rotations were replaced by Clifford angles. If every training circuit has the **same** ideal value, the slope is undetermined, $\hat{c} = 0$, and the "mitigated" value collapses to that constant (often exactly 0.0000). QEM Explorer prevents this:

- **Adaptive training set** (default). Before any noisy circuit runs, the app checks the spread of the training values classically and escalates until it is large enough: near-Clifford angles → uniformly random Clifford angles → one more kept non-Clifford gate → … → every rotation redrawn → random basis-state inputs. The strategy used and the stages tried are reported under the method.
- **Non-Clifford gates kept** is tunable: training circuits closer to the target, at a higher classical cost.
- **Min. spread** sets the required standard deviation of the training values (a fraction of the observable's range). *Training-set strategy = fixed* reproduces plain CDR for comparison.
- If no strategy yields varying values, the fit is **not applied**: a warning appears, the regression plot is replaced by an explanation, the table's *status* column says *not applied*, and the unmitigated value is reported with its own uncertainty and cost. No training circuits are charged, because the problem is detected before they would run.

Try it: load **NISQ · variational → QAOA · MaxCut**, choose the observable $\langle Z_0 \rangle$ (zero for every QAOA circuit by the $X^{\otimes n}$ symmetry), keep *CDR · OLS* among the contenders and run.

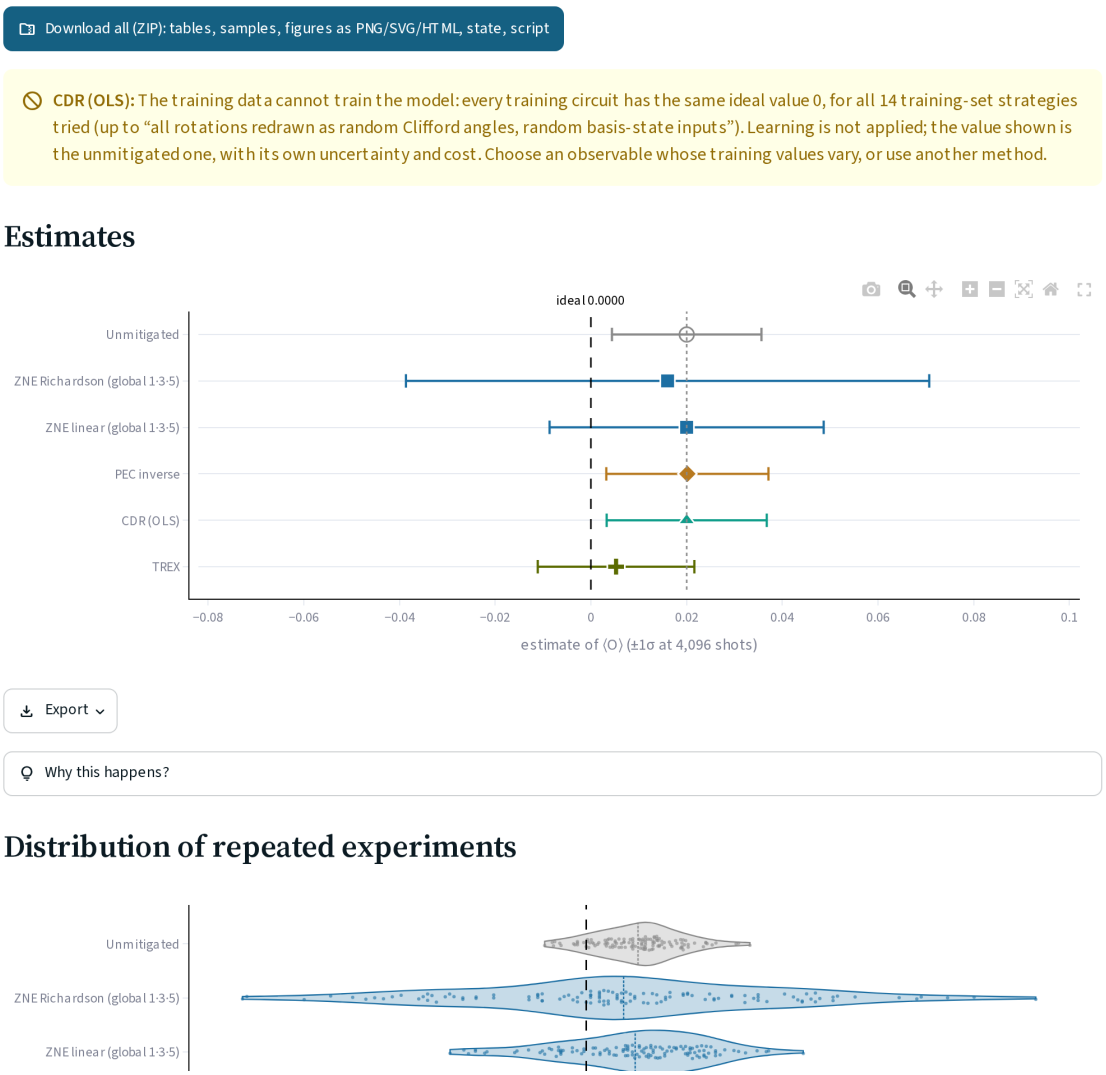


Figure 15: A degenerate training set is reported instead of plotted as a constant.

Tutorial 8 — Error-correction primitives

Load **FTQC · error correction → Steane [[7,1,3]] code**. The encoder prepares $\cos\frac{\theta}{2}|0_L\rangle + \sin\frac{\theta}{2}|1_L\rangle$; the logical Z is read transversally. With *logical readout = decoded* the Hamming syndrome of the measured bits corrects one flipped bit before the parity is taken, so an X *on qubit k* error leaves $\langle Z_L\rangle = \cos\theta$ unchanged; with *raw parity* it flips the sign. The template sets the stabilizer $Z_3Z_4Z_5Z_6 = +1$ as the symmetry, so the *Symmetry verification* contender post-selects on it (error detection).

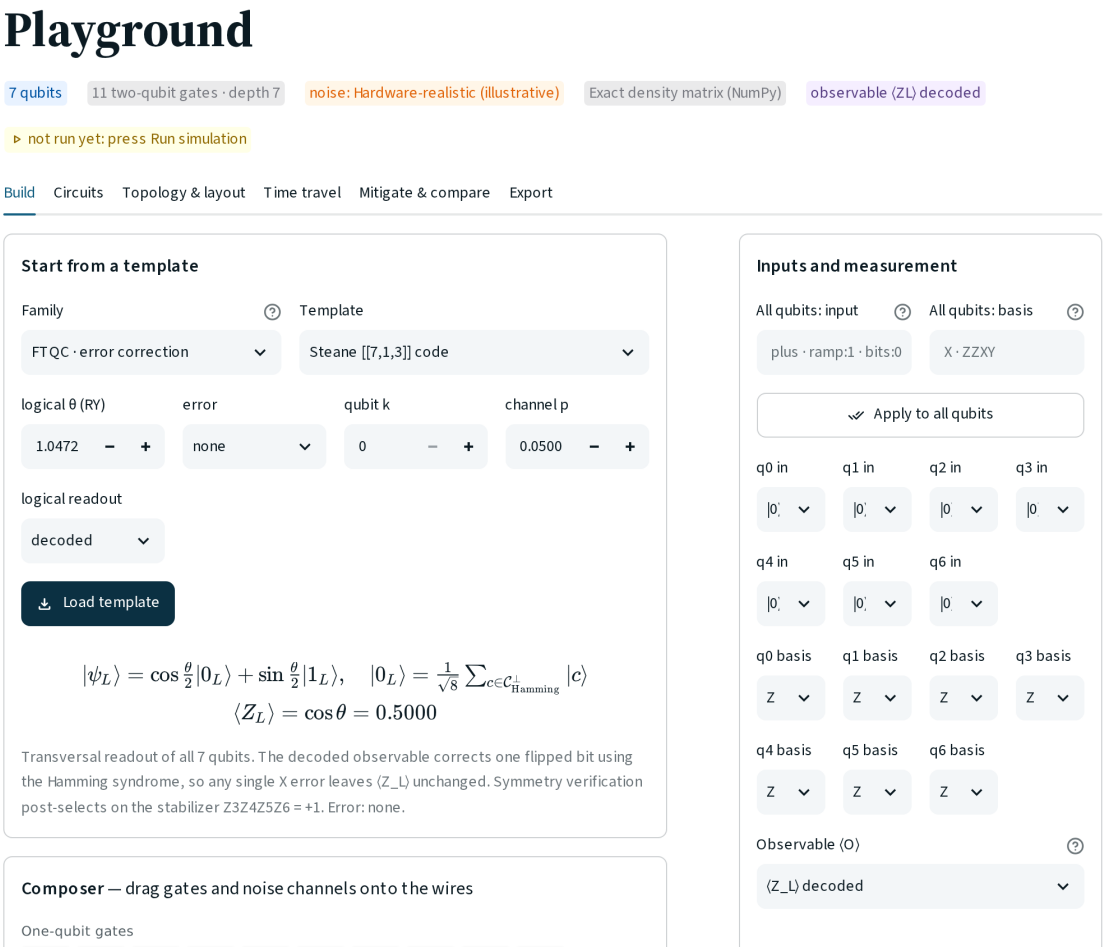


Figure 16: The Steane template: encoder, decoded logical readout and parameters.

The **3-qubit code** shows the full cycle — encode, error, syndrome ancillas, coherent Toffoli correction, decode — for bit or phase flips, and compares it with syndrome-only and majority-vote readout. The **repetition code** scales the distance d and the number of syndrome rounds; its notes give the exact logical error rate of the in-circuit bit-flip channels, which the simulation reproduces. Error channels placed in a template are part of the circuit (the ideal reference excludes them), so the gate noise of the checks competes with the code's suppression.

Two more codes complete the family: the **five-qubit code** (the smallest code that corrects any single-qubit error) and **one plaquette of the surface code** (d = 2, error detection); Tutorial 21 walks through both with the analysis panel.

The **deep algorithms** family turns Shor's order finding into a circuit with up to thousands of two-qubit gates (*repeated* powers, *synthesized* multipliers): the regime where mitigation runs out of budget and correction becomes necessary.

Tutorial 9 — Cost vs precision and the PEC wall

Open **Cost vs precision**.

- **Your circuit**: $\mathrm{RMSE}(B) = \sqrt{b^2 + v/N_t + w/s + u}$ for every contender of the Playground at a total budget B (press **Run simulation** if there is no comparison yet). Set a target precision ε,

a QPU budget and the budget range; the table gives the budget and QPU time each method needs for ε (∞ when its bias, or its training-noise floor, exceeds ε) and the crossover budget B^* above which it beats the unmitigated estimate. Training and calibration shots are included, so a method with a large fixed cost starts late.

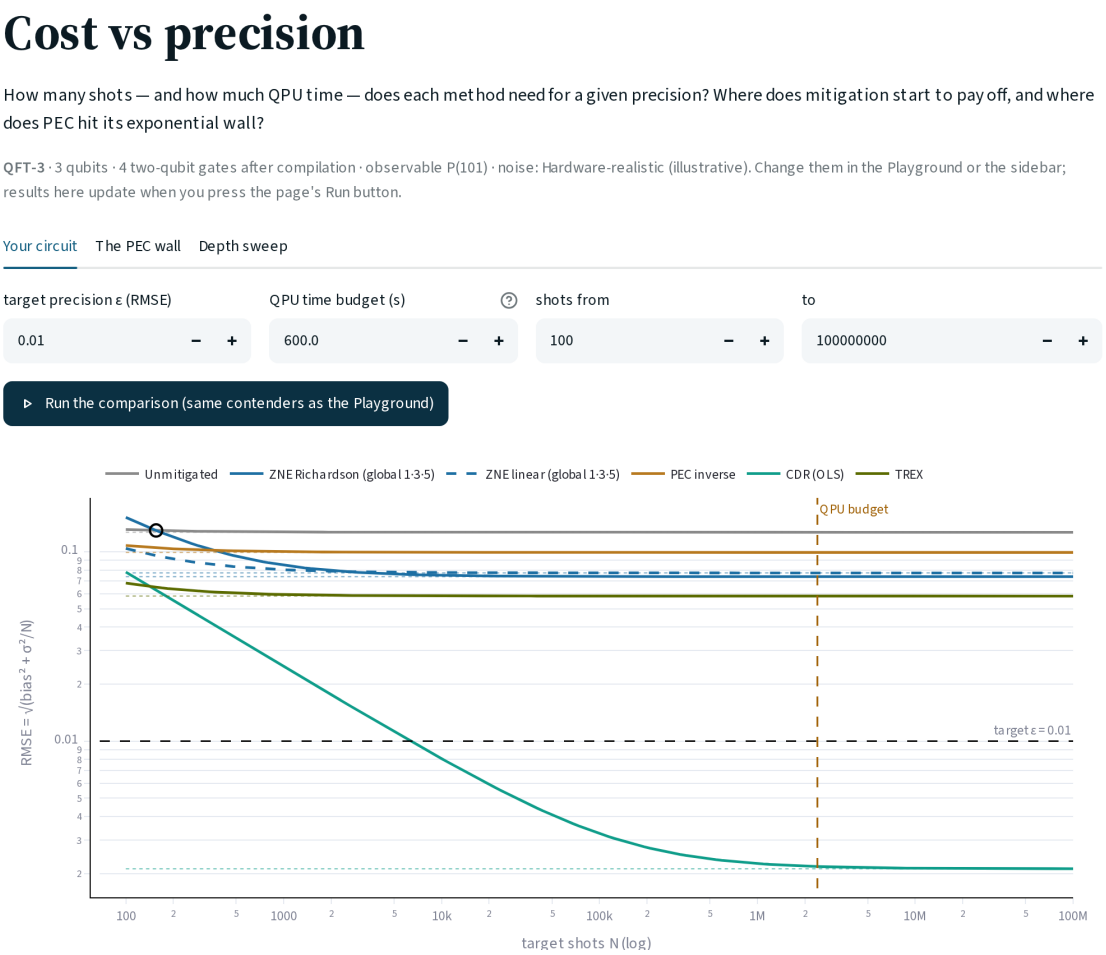


Figure 17: RMSE against shots with the target ε and the budget line.

- **The PEC wall**: the total sampling overhead $\Gamma = \prod_g \gamma_g$ of a brickwork circuit grows exponentially with depth; the chart and the heat map show where the budget runs out over depth and two-qubit error rate.

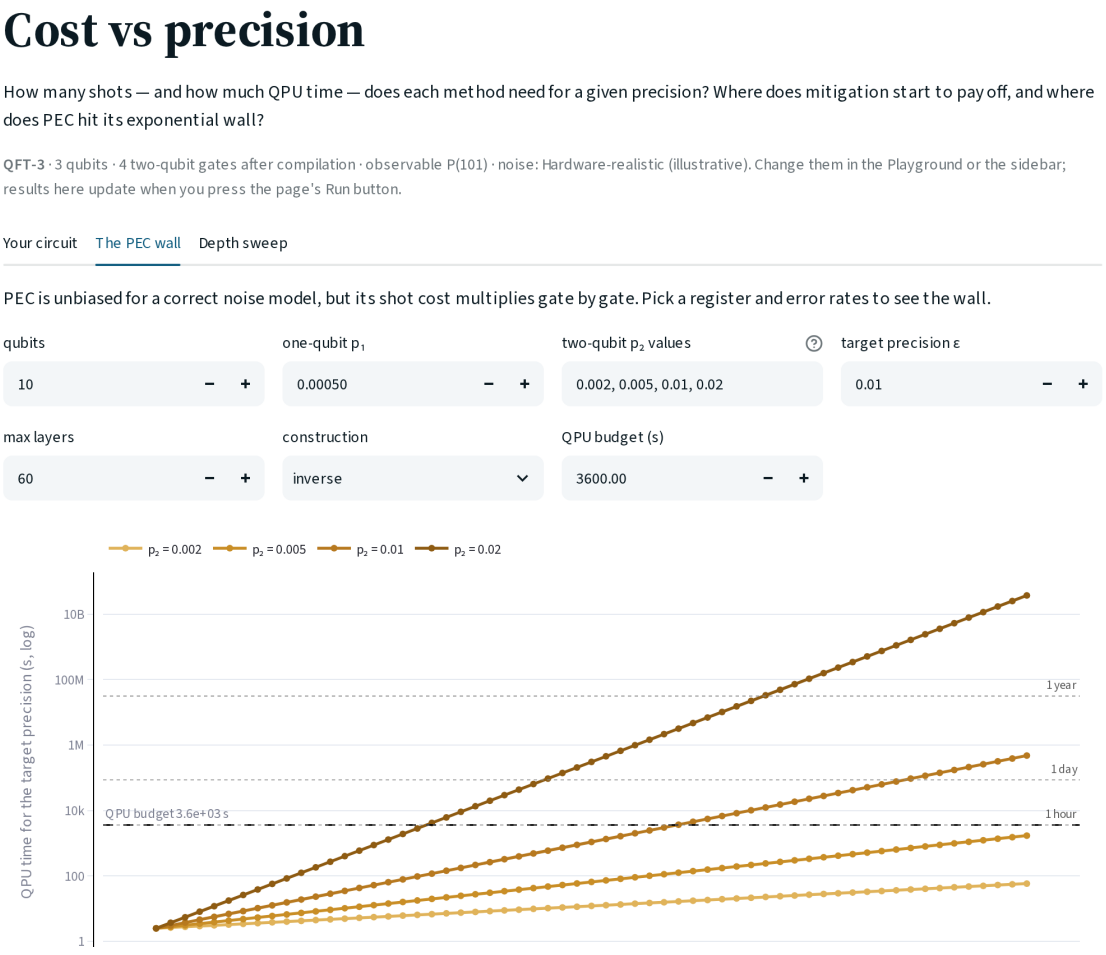


Figure 18: The PEC wall.

- **Depth sweep**: real simulations of circuits of growing depth that share one ideal answer (your circuit echoed as $U(U^\dagger U)^k$, or k Trotter steps of an Ising model).

Tutorial 10 — The bias–variance bullseye

Each dart is one simulated experiment, placed by its error in the Playground's observable (x) and a second observable (y) you choose. Mitigation moves the cloud's center onto the bullseye (less bias) but spreads it (more variance). Choose the contenders, the shots per experiment and the number of experiments.

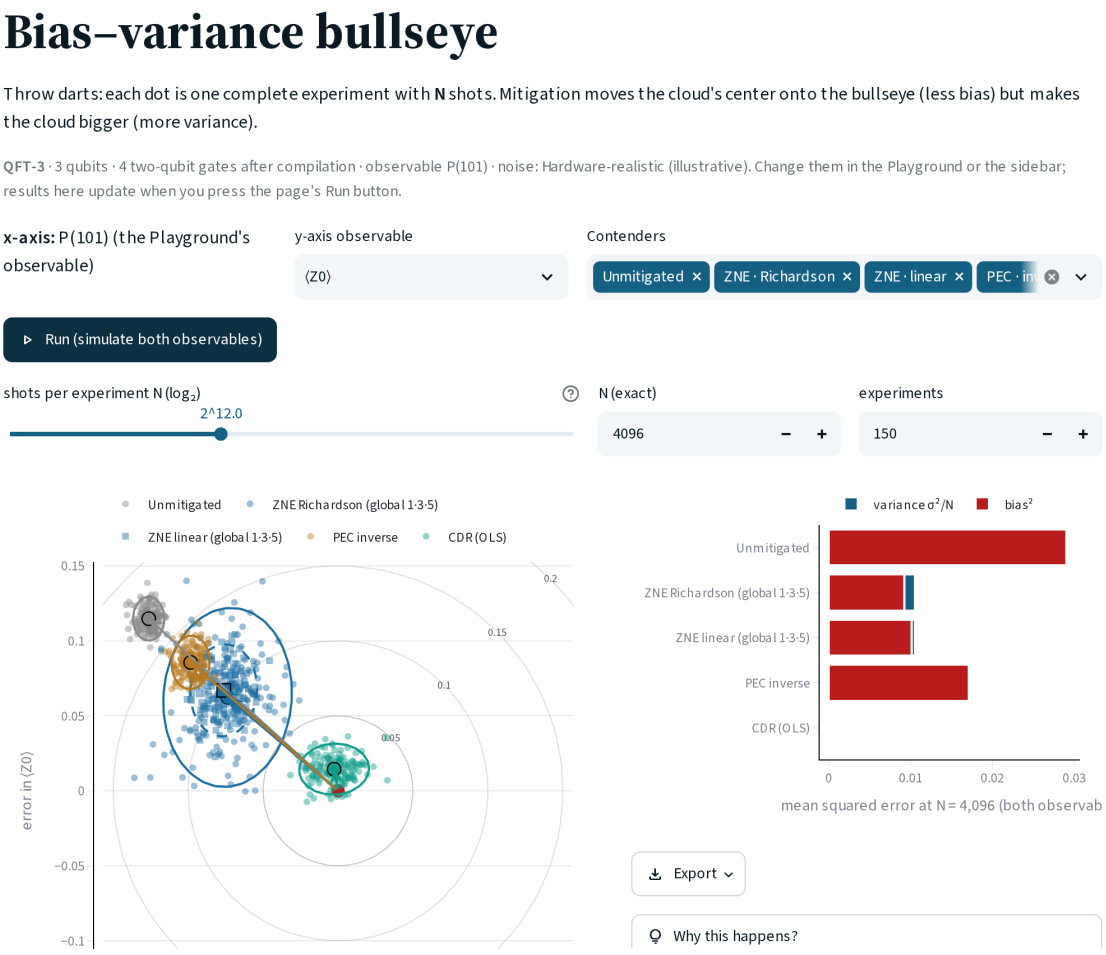


Figure 19: The bullseye.

Tutorial 11 — When the model is wrong

Model mismatch separates the noise a method *assumes* from the *true* noise: PEC with an assumed rate that is a ratio × the true one (curve and a heat map over assumed × true p_2), readout mitigation calibrated with drifted flip rates, and PEC facing an unmodeled coherent over-rotation. Only a correct model makes PEC unbiased.

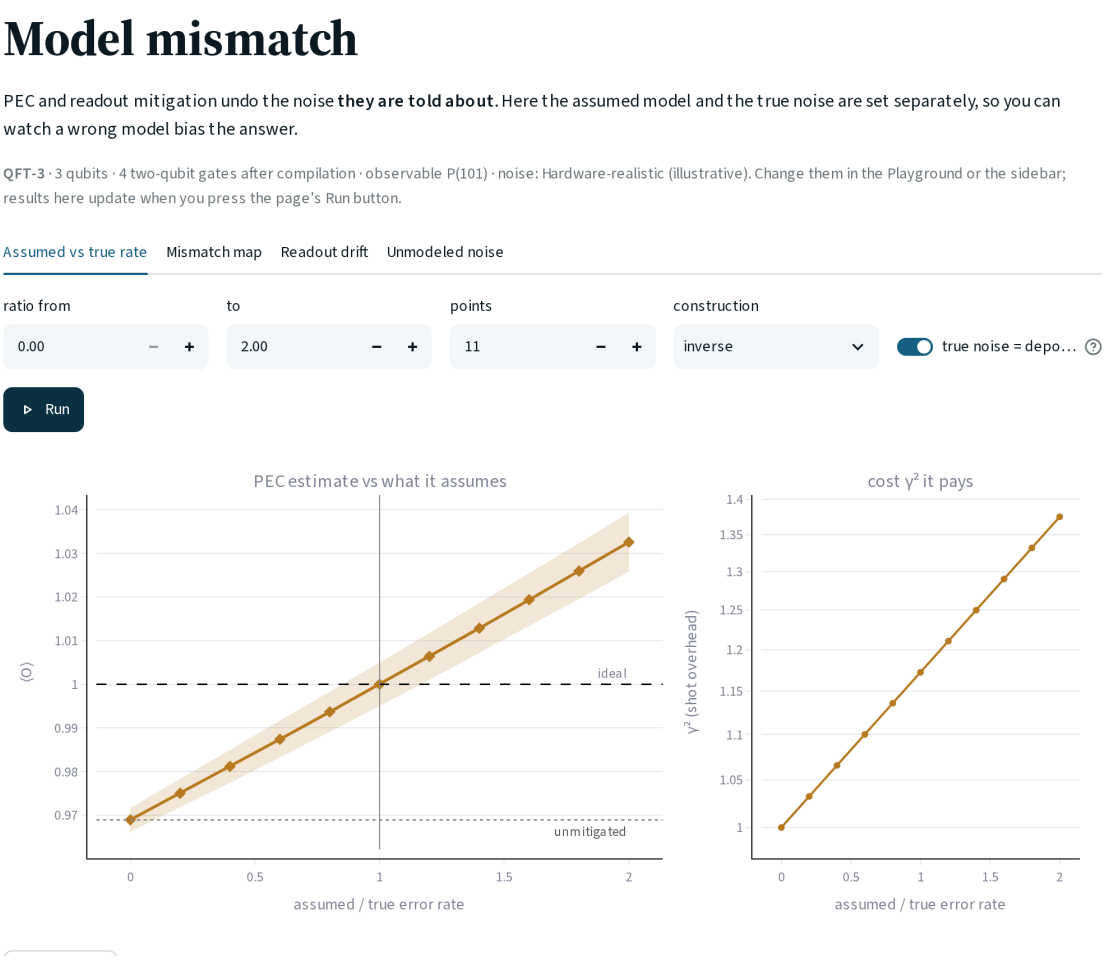


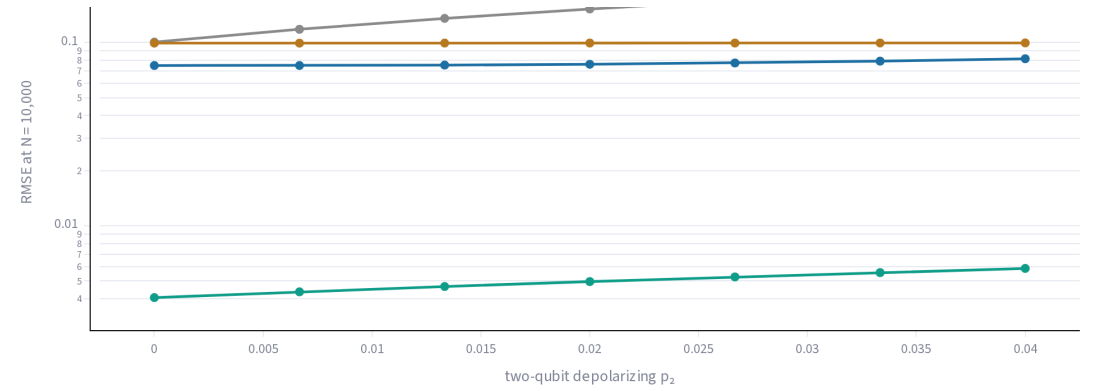
Figure 20: PEC bias against the ratio of assumed to true error rate.

Tutorial 12 — Head-to-head sweeps

Choose two or more contenders (presets and your custom recipes), a noise parameter to sweep, its range and the number of points, then **Run sweep**. The line chart shows RMSE, |bias|, the estimate, sd or γ^2 against the parameter; the **Who wins where** map colors each (parameter, shots) cell by the method with the lowest RMSE.



Figure 21: A head-to-head sweep over the two-qubit error rate.



Export

Why this happens?

Who wins where

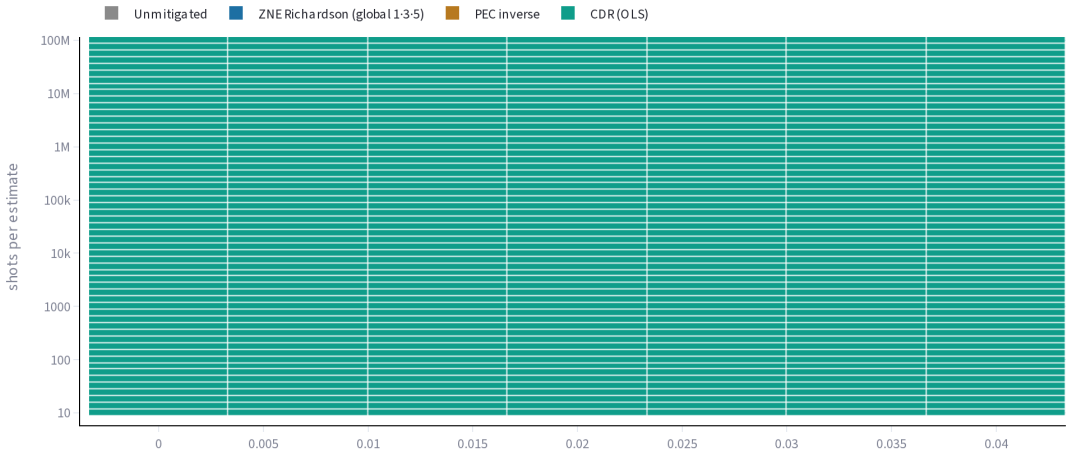


Figure 22: Who wins where, over the swept parameter and the number of shots.

Tutorial 13 — Exporting your work

Script. On the Playground's *Export* tab press **Generate Qiskit script** and download it. The script rebuilds the circuit, the compilation, the noise model (or fake device) and every contender with a verbatim copy of the engine, and prints each estimate next to the value the app showed (✓ when they agree).

- In a terminal: `python qem_explorer_export.py` (the executor you used), `--aer`, `--numpy`, or `--ibm ibm_xxx` for hardware.
- In **Jupyter**: paste the whole file into one cell and run it. The last lines detect IPython and call `main([])` without `sys.exit`, so the kernel keeps running; afterwards call `main(["--aer"])` etc. in another cell, or use `build_state()` and `user_circuit()` for your own analysis.

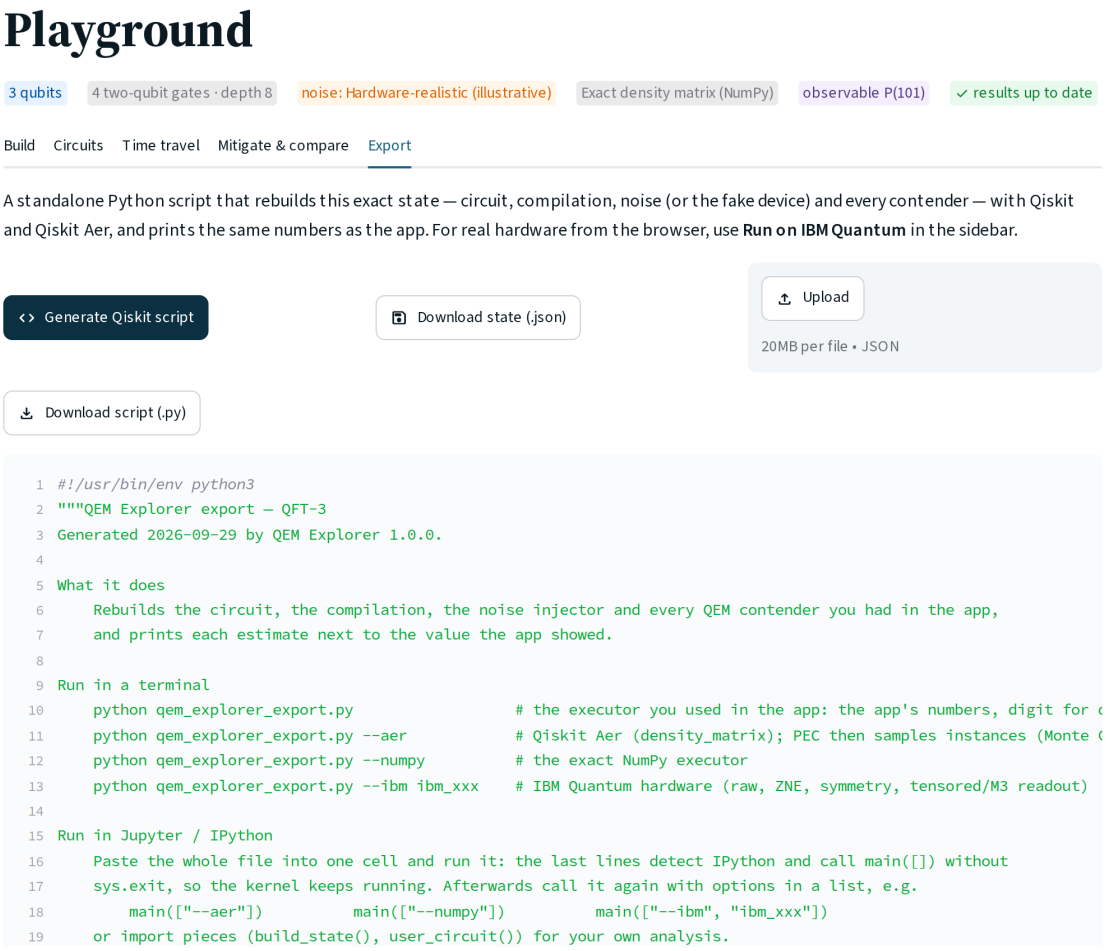


Figure 23: The Export tab with the generated script.

State files. *Download state* (*.json*) saves the complete state (circuit, compilation, noise, observable, contenders, simulator, shots); *Load a state* restores it after validating every field. Neither file ever contains a token.

ZIP bundles. *Download all* (*ZIP*) on each page packs its tables (CSV and LaTeX), repeated-experiment samples, figures (PNG at 3× and SVG; Plotly JSON when no Chrome/Chromium is available — never HTML) and the state. The Playground's bundle also includes the script and every cross-page result (Tutorial 27).

Tutorial 14 — Running on IBM Quantum hardware

Open **Run on IBM Quantum**. The contenders are those of the Playground; the expander lists which of them run on hardware.

1. **Rehearse locally.** *Where* → *local*: choose a fake backend, the number K of PEC sampled circuits and the transpiler optimization level, and press **Run locally**. The same transpiled circuits and Sampler calls as on the real service run on Qiskit Aer with the device's gate and readout errors. Every contender gets the sidebar's budget, allocated exactly as in simulation, and the results appear in the Playground's full results view (Tutorial 26).

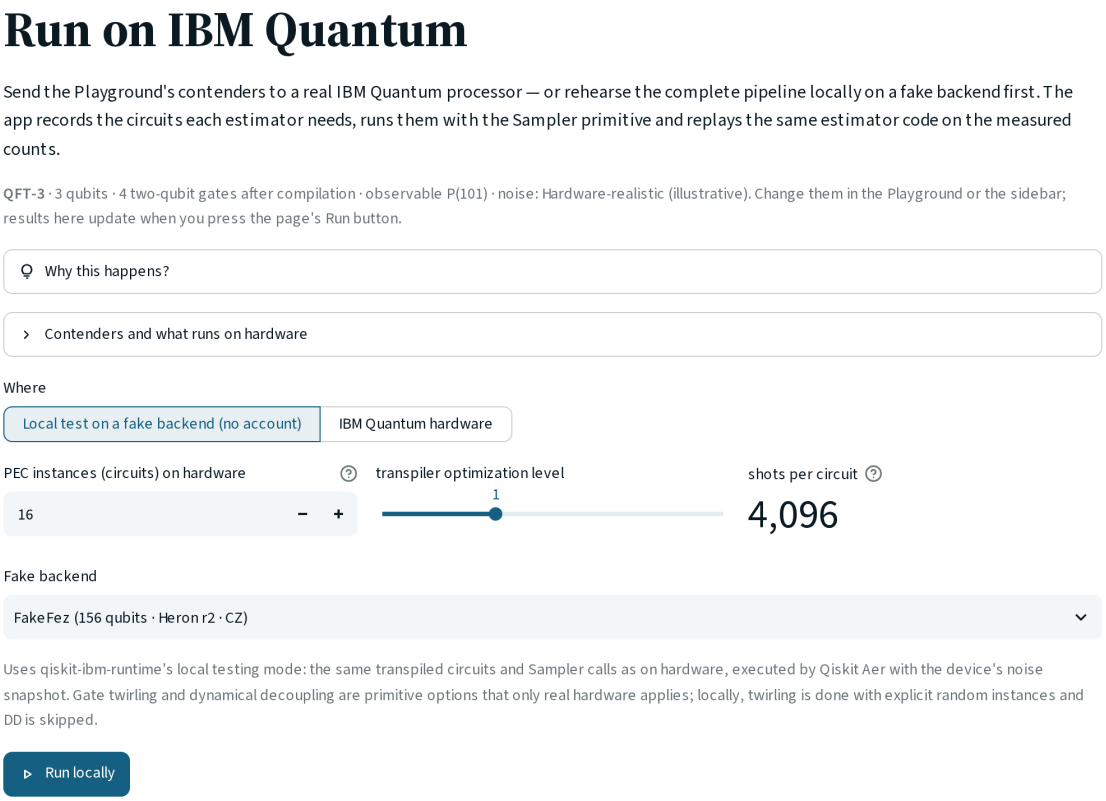


Figure 24: Run on IBM Quantum: the local rehearsal.

2. **Submit.** *Where* → *IBM Quantum hardware*: paste your API token into the password field, choose the channel (`ibm_quantum_platform`), optionally an instance and a backend (blank = least busy), and submit. The token is read inside the submit callback, the field is cleared at once, and the token is never stored, logged or included in any download.
3. **Collect.** Later, enter the token again (it is not kept between steps) and fetch the results. The measured counts are analyzed with exactly the estimator code used in simulation, and the page shows the same figures, tables and ZIP as the simulator, plus the plan and the job IDs. The circuits go out as one job per option group and shot count, inside a Batch when the account allows it.

> A token is a password. Before this notebook goes into a repository or an appendix, clear it and use one of the other two. The exported script reads `IBM_QUANTUM_TOKEN` (or `QISKIT_IBM_TOKEN`) from the environment or uses a saved account instead.

Part II-B · New in version 1.1

The tutorials in this part cover what version 1.1 adds: the compiler made visible, noise-aware qubit selection, Hamiltonian observables, live variational angles, crosstalk, the analysis panel, two new error-correcting codes, batch studies with sweeps of inputs, bases and parameters, the Learn page and the research handbook. The fastest way to try each one is the matching guide on the **Learn** page: its *Set up the Playground* button loads the circuit, noise and contenders for you.

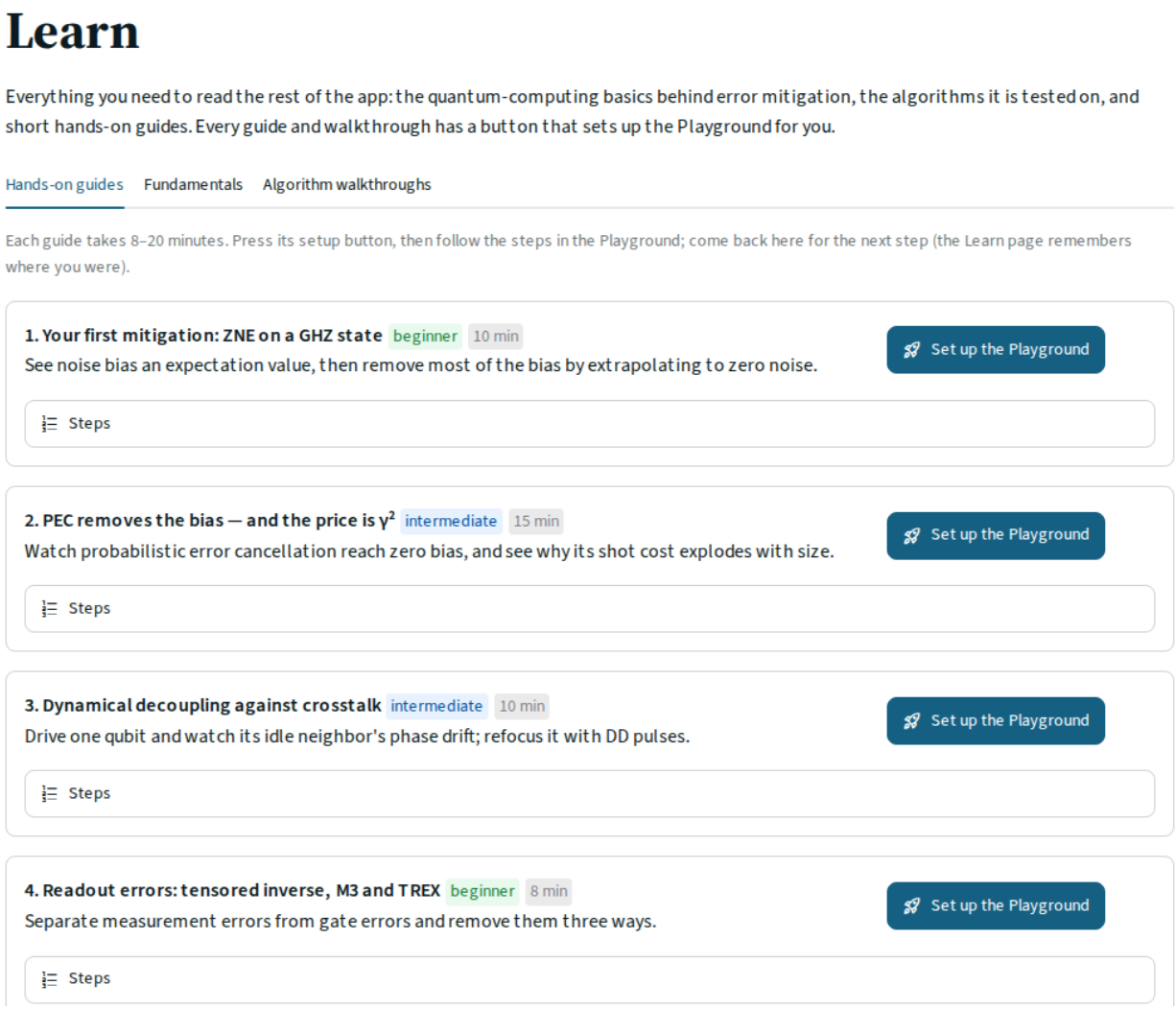


Figure 25: Learn → Hands-on guides: every guide sets up the Playground with one button.

Tutorial 15 — Logical versus physical: what the compiler did

1. On the Playground's **Build** tab choose *Compilation → ISA* (or press *Compile on a heavy-hex (IBM)*), then turn on **Show the compiled (physical) circuit** under the composer.
2. The view *Logical over physical (aligned)* stacks your circuit (top rows, `q0`, `q1`, …) above the executed one (bottom rows, labeled with the physical qubit, e.g. `p4 (q0)`). Each logical gate sits directly above the native gates that implement it, framed together; an **amber frame marked SWAP** is a routing SWAP that has no logical counterpart. Hover any native gate to see which logical gate it belongs to.

3. The cards count routing SWAPs, two-qubit gates, qubits used and depth before → after compilation. The other views show the routed circuit (layout + SWAPs), the native gates only, and the executed circuit after optimization.
4. *How each logical gate was rewritten* lists every rewrite, e.g. $H \rightarrow R_Z(\pi/2)\sqrt{X}R_Z(\pi/2)$ and $\mathrm{CP}(\varphi) \rightarrow R_Z \cdot \mathrm{CX} \cdot R_Z \cdot \mathrm{CX} \cdot R_Z$, with the gate counts.

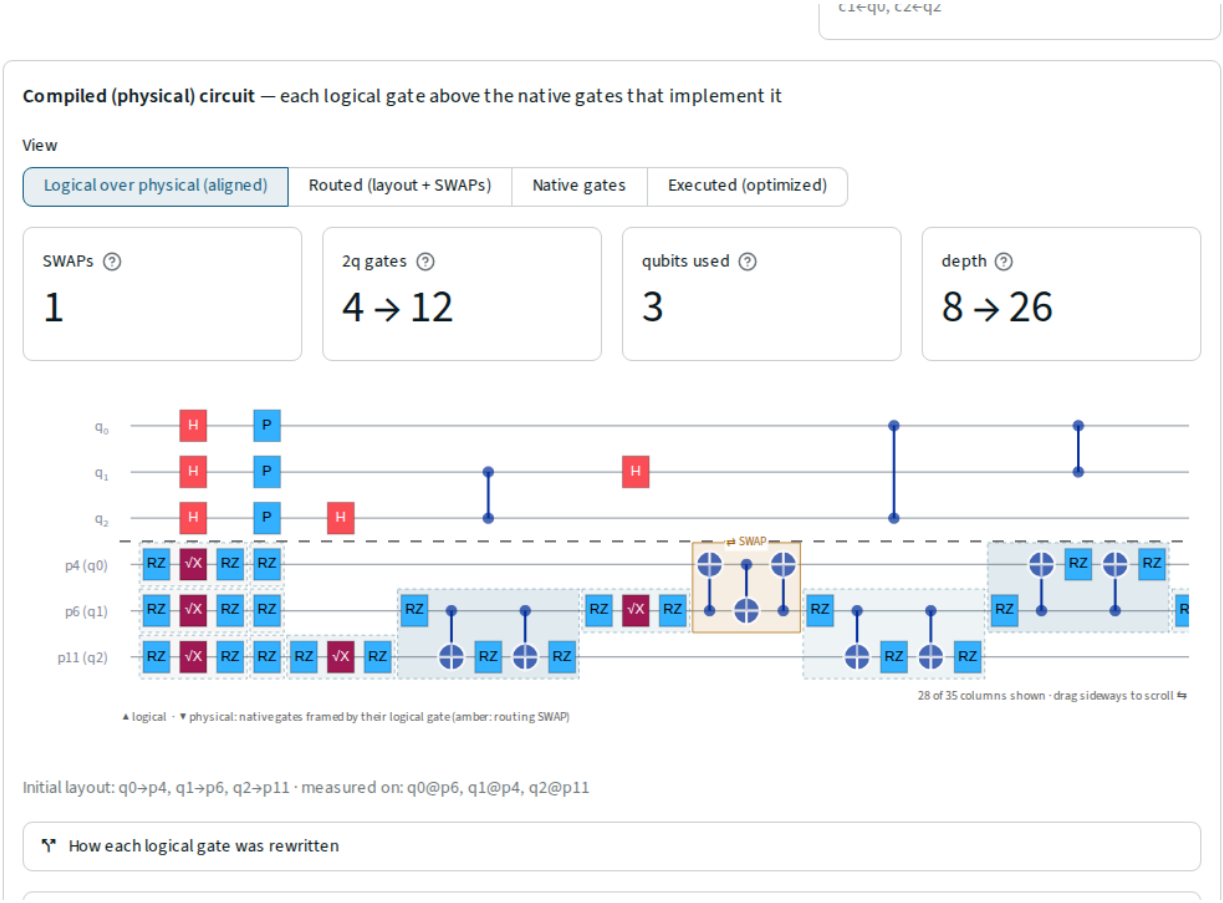


Figure 26: The aligned diff: logical gates above the native gates that implement them; the amber frame is a routing SWAP.

Why it matters for mitigation: ZNE folds, PEC inverts and twirling randomizes the **native** gates. Every SWAP adds three two-qubit gates whose noise the mitigation has to remove too.

Tutorial 16 — Topology, SWAPs and the lowest-noise qubits

1. Choose a fake device (e.g. *FakeFez*) or a lattice (*2D grid*, *heavy-hex*) and open **Topology & layout**.
2. The coupling graph colors every qubit by its **readout error** or **√X error**; broken couplers are dashed. The circuit's qubits are ringed and each coupler that carried a routing SWAP is drawn in amber.
3. *Why each SWAP was inserted* lists every SWAP: the coupler, the logical qubits it exchanged and the gate it made possible.
4. **Find the lowest-error physical qubits** places the compiled circuit on every equivalent set of qubits (same connections, a VF2 subgraph search, as *mapomatic* does) and ranks the placements by the expected success $F = \prod_g (1 - e_g) \prod_q (1 - e_q^{\mathrm{RO}})$ over two-qubit gates, pulses and readouts. **Use layout #1** recompiles on the best placement; the dotted rings on the graph mark it.

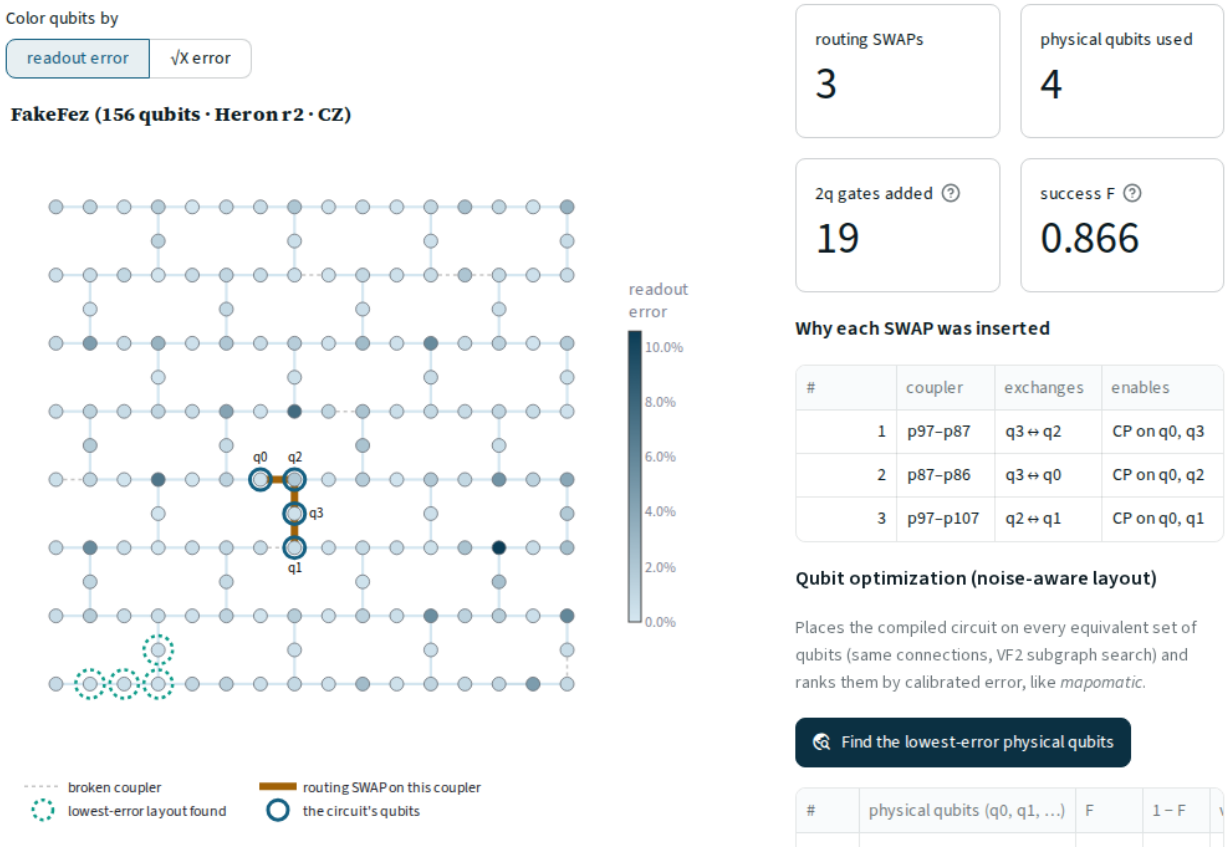


Figure 27: Topology & layout on FakeFez: the circuit's qubits, the SWAPs, and the ranked placements.

A layout can also be typed (Part III). Changing the device, the coupling map or the compilation mode clears a typed layout, so a stale layout never breaks compilation.

Tutorial 17 — Hamiltonian observables

1. In **Observable** choose *Hamiltonian: a sum of Pauli strings…*.
2. Type $H = \sum_k c_k P_k$ in the box, e.g. `- Z0*Z1 - Z1*Z2 - X0 - X1 - X2`, and press Ctrl+Enter (or leave the box) to apply it — or click one of the examples under the box (transverse-field Ising, Heisenberg XXX chain, MaxCut on a ring, a mix to edit), which applies it at once. A caption lists the terms and the measurement settings; a typing error is shown instead, and the previous observable stays in use.
3. The app groups the terms into qubit-wise commuting settings (terms that agree qubit by qubit share one measurement), adds the basis change (H for X, $S^\dagger H$ for Y) before measurement and runs **every** method on every setting. The combined estimate and its variance are exact: $\hat{H} = c_I + \sum_g \hat{O}_g$ and $\mathrm{Var}(\hat{H}) = \sum_g v_g / N_g$.

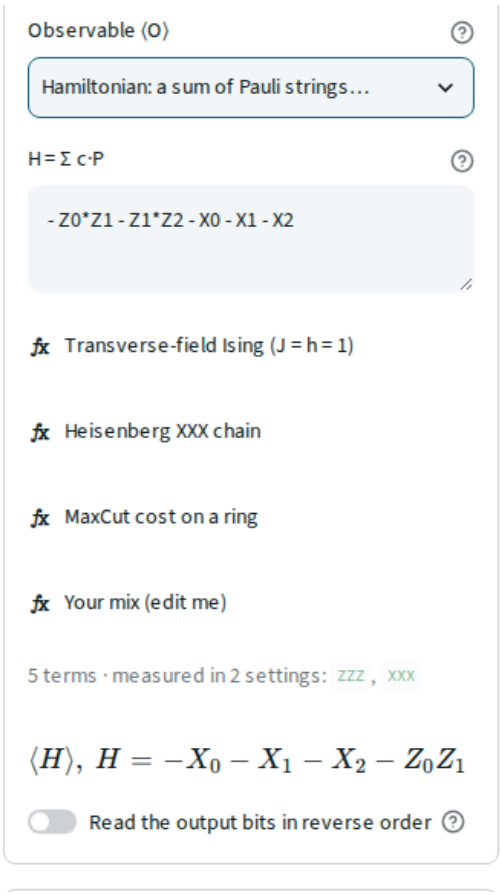


Figure 28: A Hamiltonian observable with ready-made examples.

The exported script reproduces the same grouping, so a Hamiltonian run can be repeated in Qiskit or on hardware.

Tutorial 18 — Live parameters and energy landscapes

1. Load **VQE · hardware-efficient ansatz** or **QAOA · MaxCut** and press **Run simulation**; open **Time travel**.
2. **Live parameters θ** shows one slider per variational angle (for QAOA: γ_k with period 2π and β_k with period $\pi/2$). Moving a slider redraws the circuit, the Bloch spheres and the ideal and noisy $\langle O \rangle$ at once (up to eight qubits; for larger circuits use the angles in the circuit and run the simulation).
3. **Use these angles in the circuit** writes the slider values into the circuit; **Reset to the template's angles** undoes the changes.
4. **Energy landscape $\langle \mathbf{O} \rangle(\boldsymbol{\theta})$** scans one angle (up to 401 points) or, for QAOA with $p = 1$, the (γ, β) plane (up to 41×41 points) with the current noise — behind its **Scan** button. The qubit layout is fixed during a scan, so a 15×15 scan takes seconds.

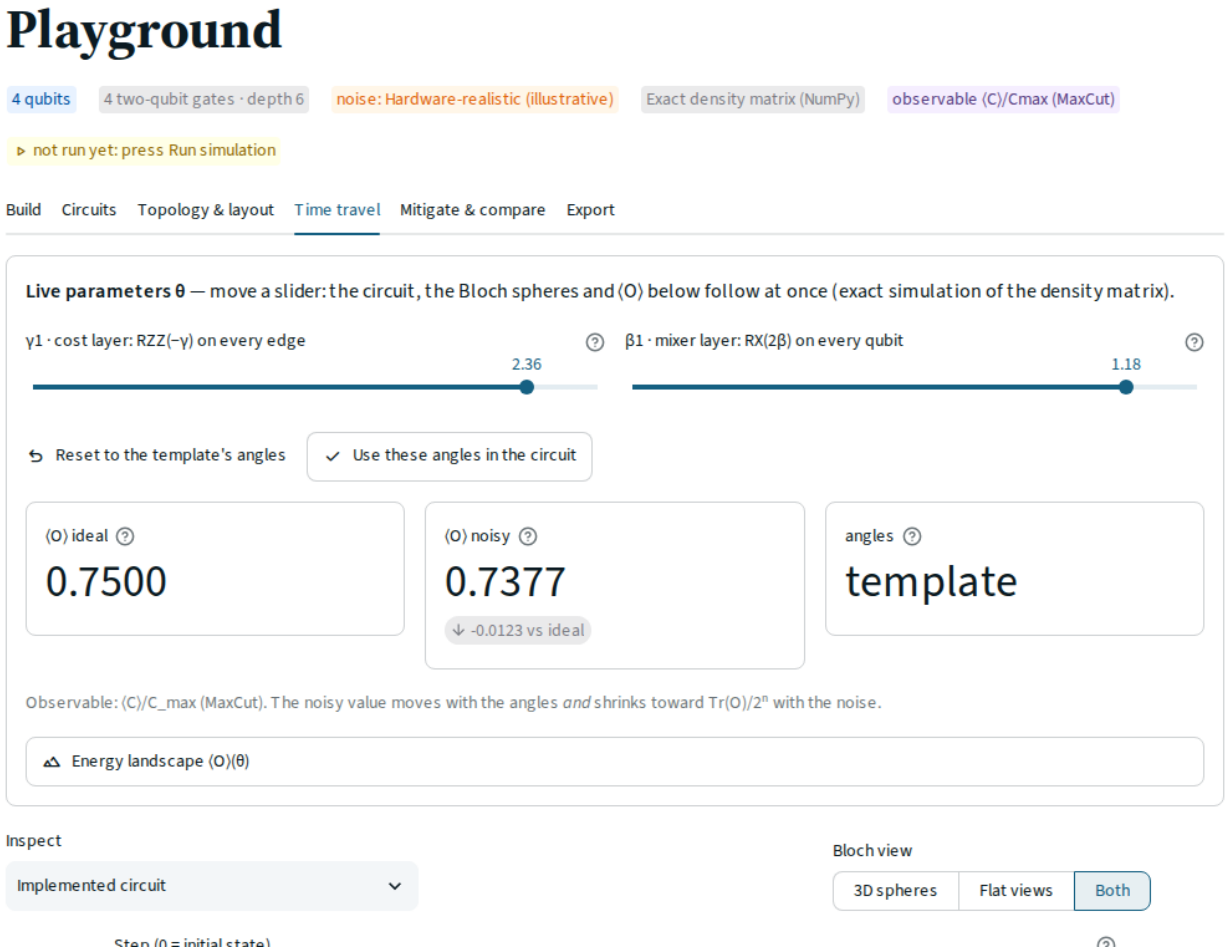


Figure 29: Live sliders move the circuit and the Bloch spheres together.

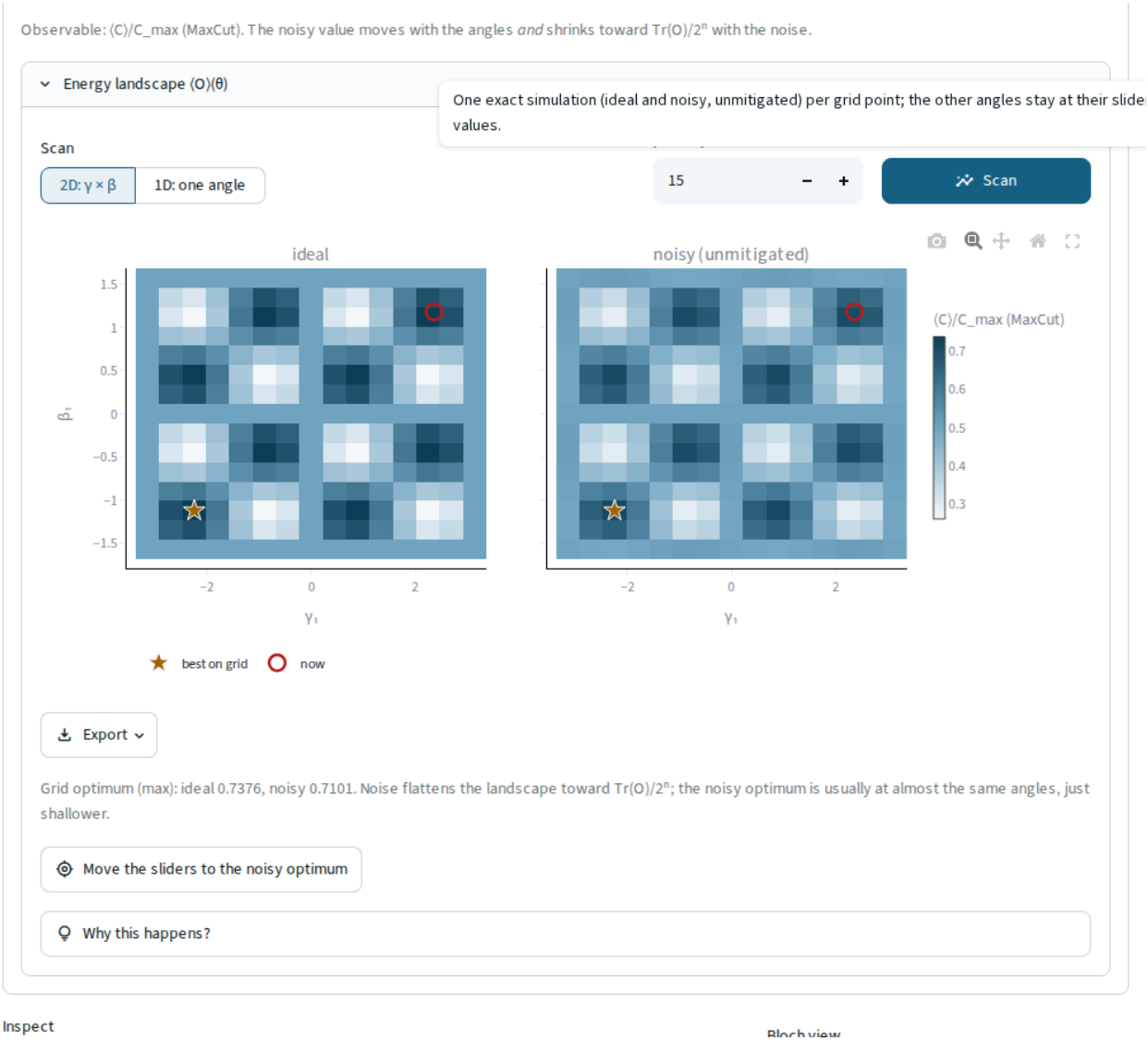


Figure 30: The noisy landscape is a flattened copy of the ideal one: noise shrinks the contrast, not the location of the optimum.

Under global depolarizing noise the landscape shrinks toward the maximally mixed value, $\langle O\rangle_{\text{noisy}}(\theta) \approx f\,\langle O\rangle(\theta) + (1-f)\,\mathrm{Tr}(O)/2^n$: the optimum stays where it was, but the gradient that an optimizer has to find shrinks by the factor f.

Tutorial 19 — Crosstalk and dynamical decoupling

1. On the Learn page press the setup button of **Dynamical decoupling against crosstalk**. It loads a *spectator Ramsey* circuit — q0 and q2 start in $|+\rangle$ while their neighbor q1 is driven with eight $\sqrt{\text{X}}$ pulses, then q0 is read in the X basis, ideally $\langle Z_0\rangle = 1$ — and the *Crosstalk* (*spectator*) noise preset.
2. Crosstalk adds $R_Z(\delta_x)$ on every idle neighbor of a driven qubit and $R_{ZZ}(\zeta)$ between the driven qubit and each neighbor (sidebar group *Crosstalk* (*spectators*)). Neighbors follow the coupling map (a line when the circuit is abstract).
3. In **Time travel** q0's Bloch vector turns about z a little with every pulse on q1. In **Mitigate & compare**, *Dynamical decoupling* (*XY4*) removes most of the bias at almost no sampling cost: each echo pulse in q0's idle window flips the sign of the kicks that follow, so they cancel in pairs. Pauli twirling does nothing here (it randomizes two-qubit gates, not these one-qubit

kicks), and ZNE overshoots, because a coherent phase gives $E(\lambda) = \cos(\lambda\phi)$, which a polynomial extrapolation does not follow.

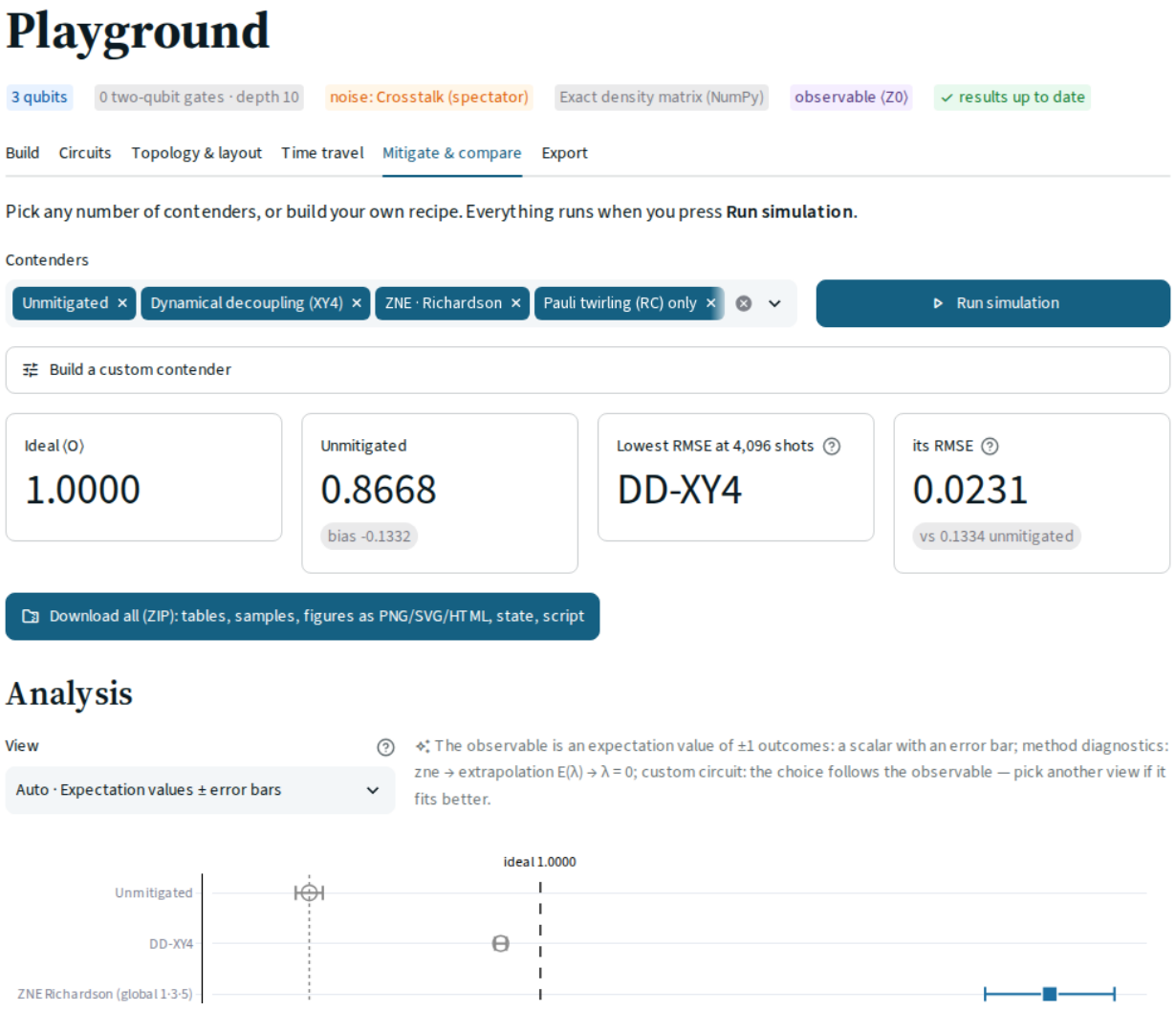


Figure 31: DD refocuses the spectator phase that crosstalk accumulates.

Tutorial 20 — The analysis panel: reading each result the right way

After **Run simulation**, the *Analysis* section of **Mitigate & compare** chooses the view that matches the mathematics of what you measured, and says why next to the **View** selector:

What you measure	Automatic view	What it shows
a probability $P(s)$ or a projector (values 0 and 1)	**Output distribution (histogram)**	ideal, noisy, readout-corrected and ZNE-extrapolated distributions; the outcomes counted by O are shaded; TVD and Hellinger fidelity
a parity, a magnetization or a Hamiltonian	**Expectation values ± error bars**	each method's estimate with $\pm\sigma$ and the spread of repeated experiments

What you measure	Automatic view	What it shows
a code's logical observable	**Logical error & syndromes**	the effective logical error p_L of every method and the syndrome histogram
(second choice for any circuit)	**Method diagnostics**	one card per method: ZNE fit, PEC quasi-probabilities and γ, training fit, VD spectrum, acceptance, readout assignment

The distributions are exact. Readout mitigation and linear ZNE act outcome by outcome, so they can be drawn as distributions; summing the observable over the readout-corrected or ZNE-extrapolated histogram reproduces that method's estimate exactly. A corrected distribution is a *quasi*-probability: small negative bars are allowed and meaningful. PEC, learning-based mitigation, VD and symmetry verification correct the expectation value, not the distribution, so they appear in the other views. Distributions are computed for circuits with up to 16 measured bits and drawn over the bits the observable reads; the other bits are summed over.



Figure 32: Shor's order finding: the output distribution. The shaded outcomes are those counted by the observable.

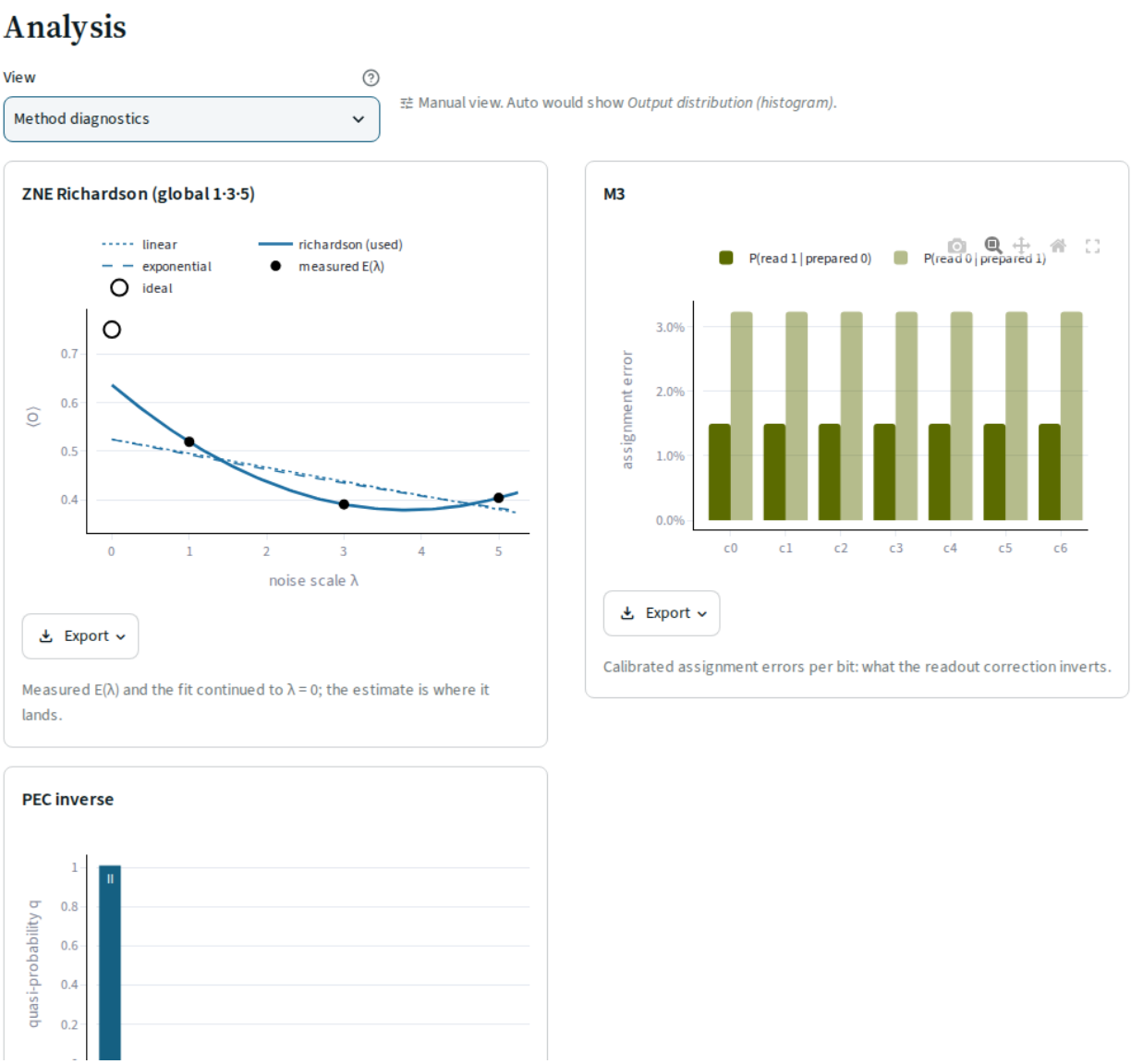


Figure 33: The same run through *Method diagnostics*.

Choosing the view yourself. The **View** menu lists every view that applies, recommended first (the logical error view needs a code template read with its own observable). For a circuit you built yourself the reason next to the selector says that the choice follows the observable, so pick the view that fits your question. The choice is kept with the result: changing the circuit afterwards marks the results *out of date* but never reinterprets them.

Tutorial 21 — The five-qubit code and a surface-code plaquette

Five-qubit code [[5,1,3]]. The smallest code that corrects an arbitrary error on any one qubit. Its four stabilizers are the cyclic shifts of $XZZXI$. The template encodes $\cos\frac{\theta}{2}|0_L\rangle + \sin\frac{\theta}{2}|1_L\rangle$ with a Clifford encoder, applies the chosen error, and runs the encoder backwards: the error becomes a logical Pauli on q0 and a four-bit syndrome on q1–q4, which a 16-entry lookup table decodes.

1. Load **FTQC · error correction → Five-qubit code**, set *error* = *Y on qubit k* with any k and press **Load**. With *logical readout* = *decoded* the ideal value stays $\langle Z_L\rangle = \cos\theta$; with *raw* it does not.
2. Choose *depolarizing channel on every qubit*, run, and read **Logical error & syndromes**: the syndromes show how often each check fired, and $p_L = \frac{1}{2}(1 - \langle Z_L\rangle/\langle Z_L\rangle_{\text{intended}})$ compares every method with the value the code was asked to protect, $\cos\theta$.

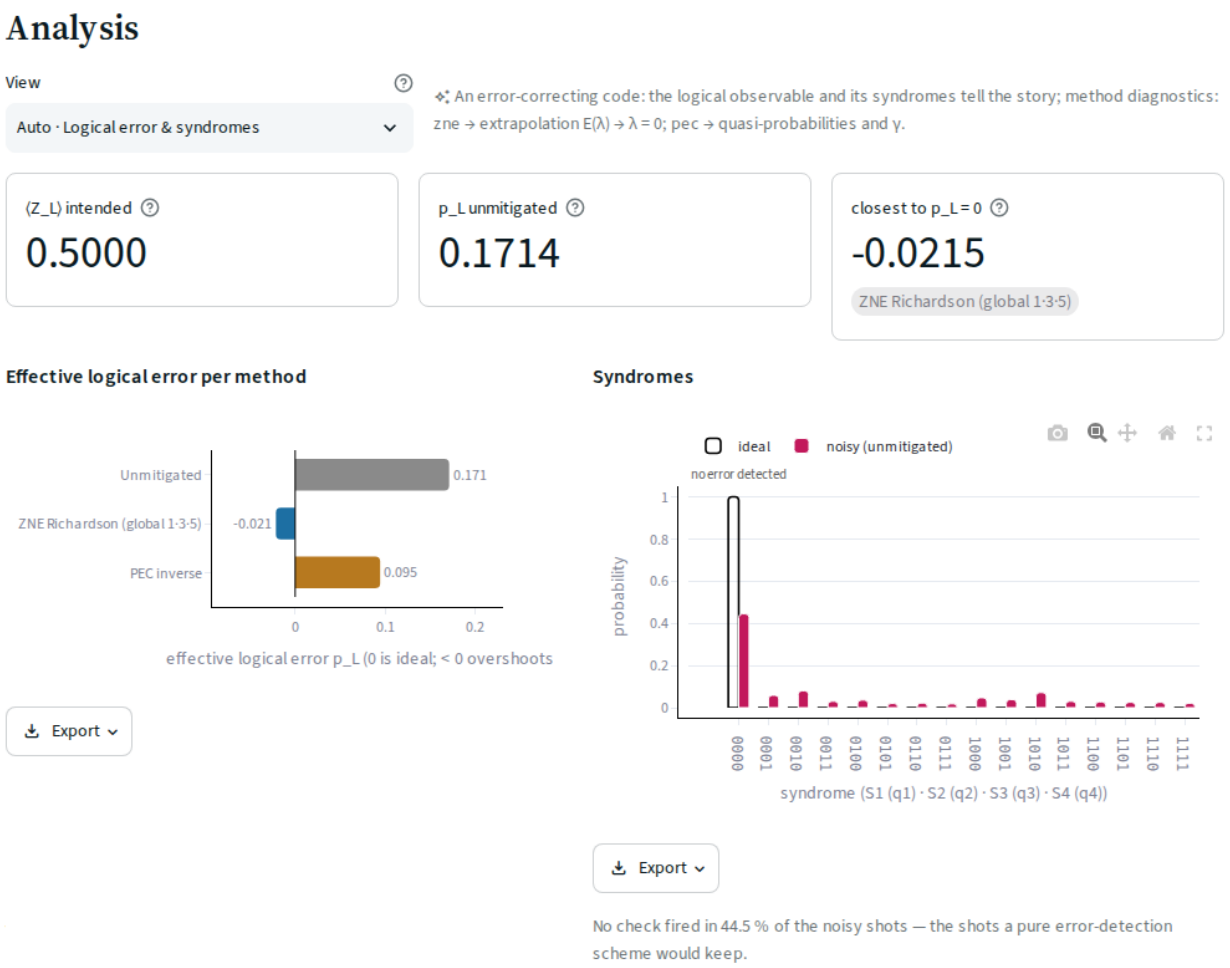


Figure 34: The five-qubit code under depolarizing noise: effective logical error per method and the syndromes.

Surface code, d = 2 [[4,1,2]]. One plaquette: data qubits 0–3 with the stabilizers $X_0X_1X_2X_3$, Z_0Z_1 and Z_2Z_3, logical operators $Z_L = Z_0Z_2$ and $X_L = X_0X_1$. With distance 2 it **detects** any single error but cannot tell which qubit had it. *Syndrome = ancilla extraction* measures the three checks on ancillas q4–q6 like a real surface-code cycle; *transversal readout* computes the Z checks from the data bits.

3. Load it with *bit-flip channel on every data qubit* and add **Symmetry verification**. The preset checks both Z stabilizers at once (e.g. $Z_5 = +1$ and $Z_6 = +1$ on the ancillas), which is exactly error detection by post-selection: the logical error drops, and the price is the rejected shots ($1/P_{\text{accept}}$ more shots).

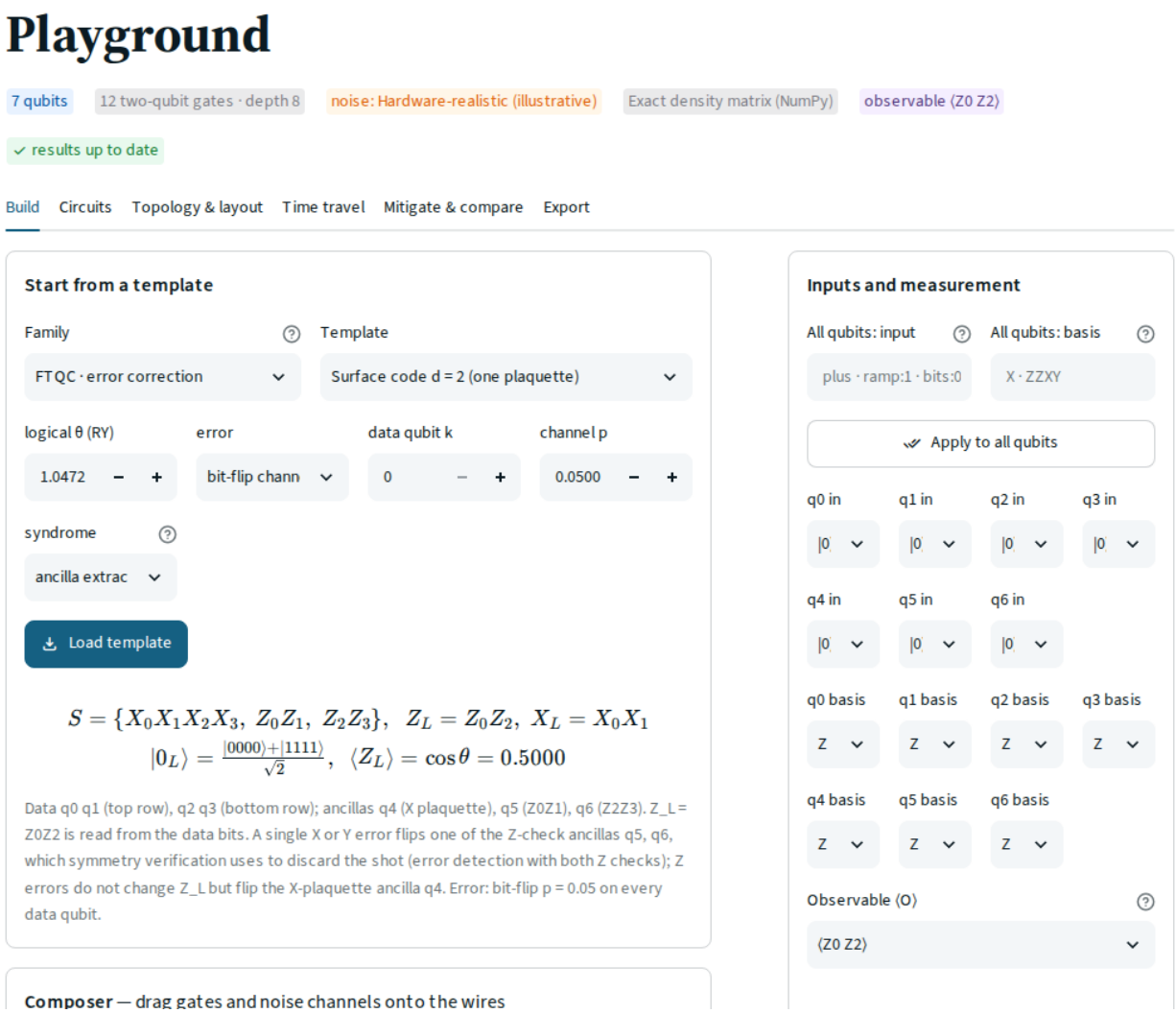


Figure 35: One plaquette of the surface code, with ancilla syndrome extraction.

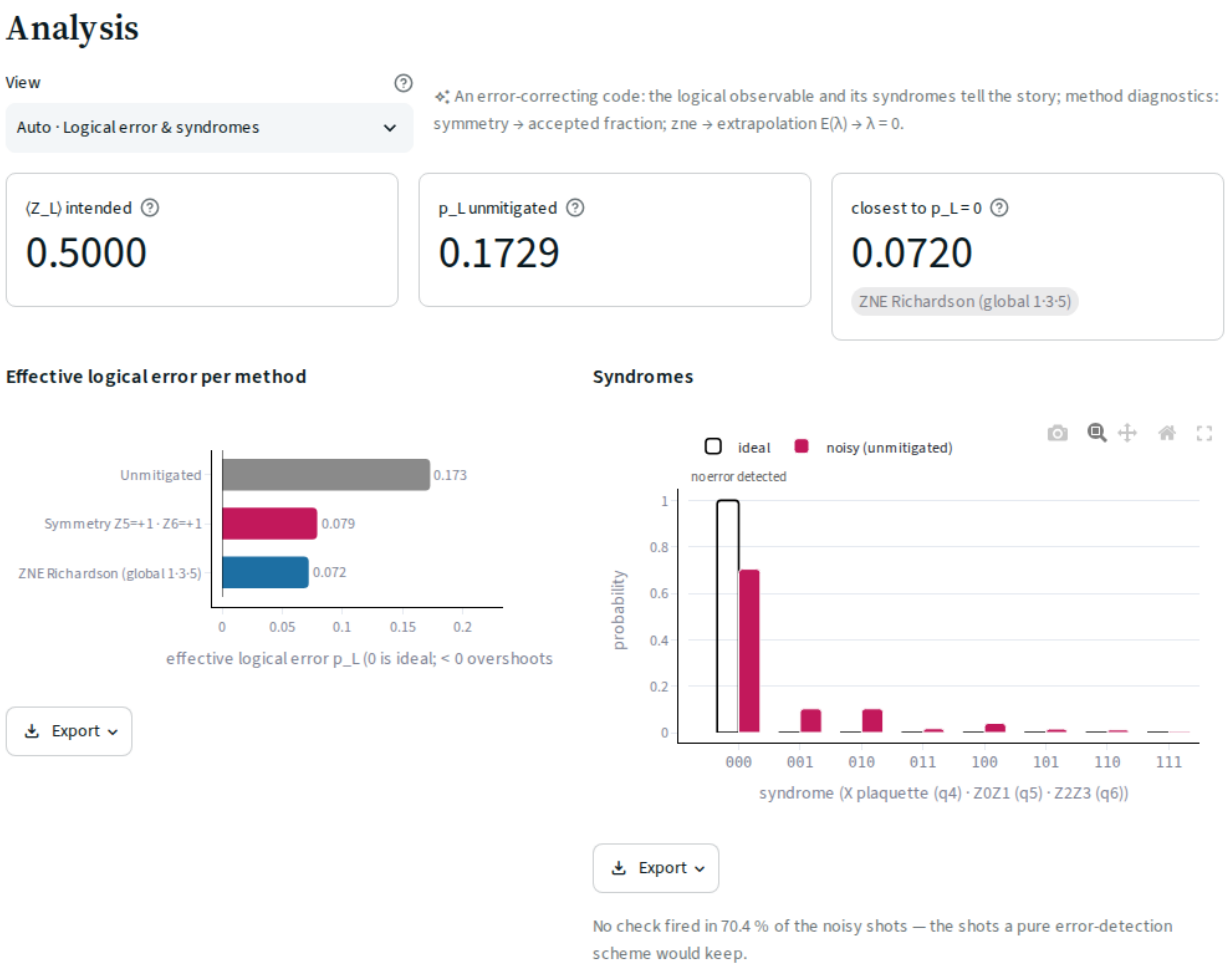


Figure 36: Error detection as post-selection: symmetry verification with both Z checks.

Tutorial 22 — Batch studies: which method wins where?

1. Open **Batch run** (or press the setup button of the guide *Which method wins where?*).
2. Tick the **Algorithms** (the templates' default settings, plus your current Playground circuit), the **Mitigation methods** (presets and your custom contenders), the **Noise models**, the **Noise scales λ** (each multiplies every error rate and coherent angle of the model) and the **Simulators**. The line above the button counts the scenarios and estimates; symmetry verification uses each algorithm's own symmetry.
3. Press **Run all**. Each scenario runs in its own `try/except`, so one failure (e.g. a method that does not apply to that circuit) is reported in the table and the rest continue.
4. Read the results: a heat map of every scenario × method colored by RMSE, |bias|, the estimate or γ, with RMSE at the shot budget you choose (★ marks the best method of each scenario; failed cells stay empty), **Best method per scenario** with a tally of wins, and **Error against the noise scale** for each algorithm.

Batch run

Tick the algorithms, methods, noise models, noise scales and simulators to compare, then press **Run all**. Every combination is simulated exactly (bias and variance, no shot noise), one after the other; a combination that fails is reported and the batch carries on.

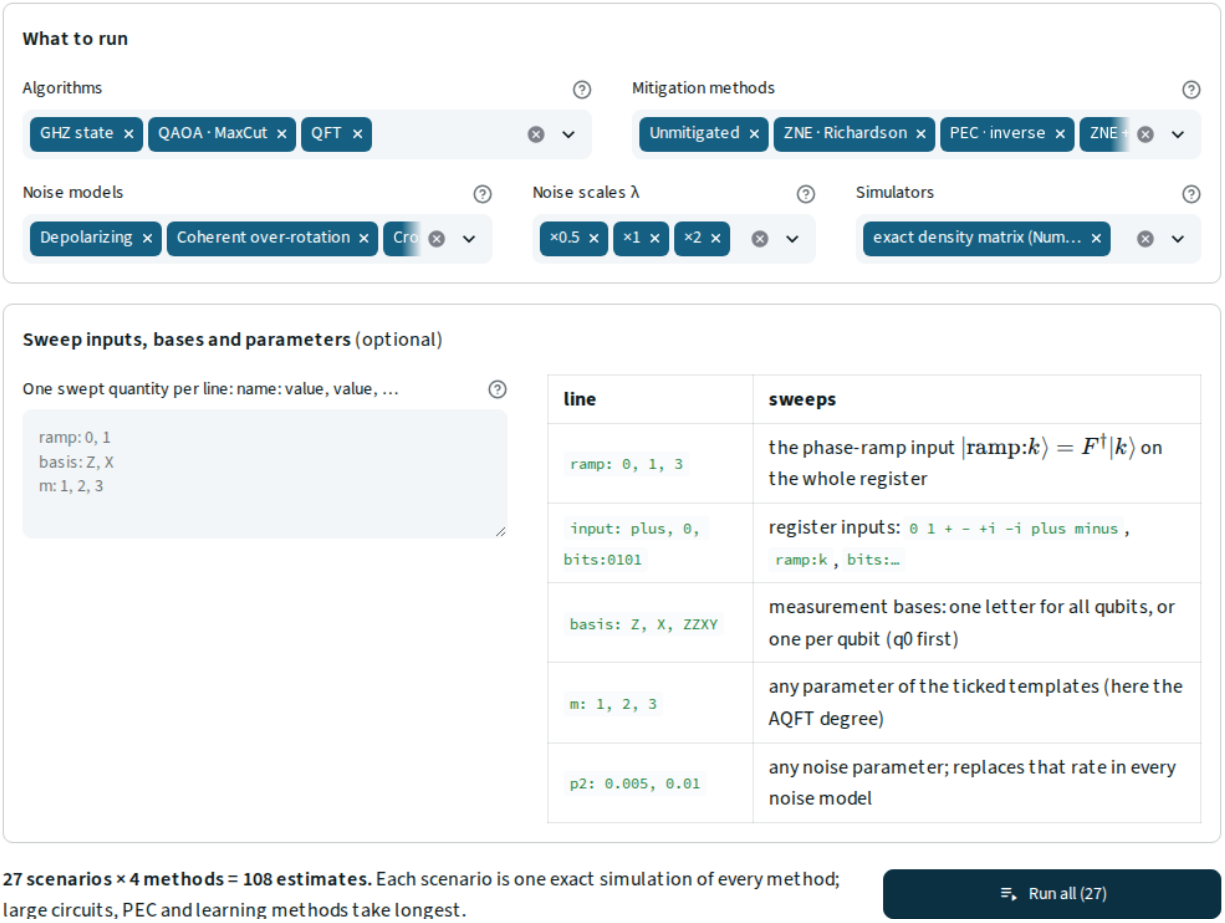


Figure 37: Batch run: every scenario × every method.

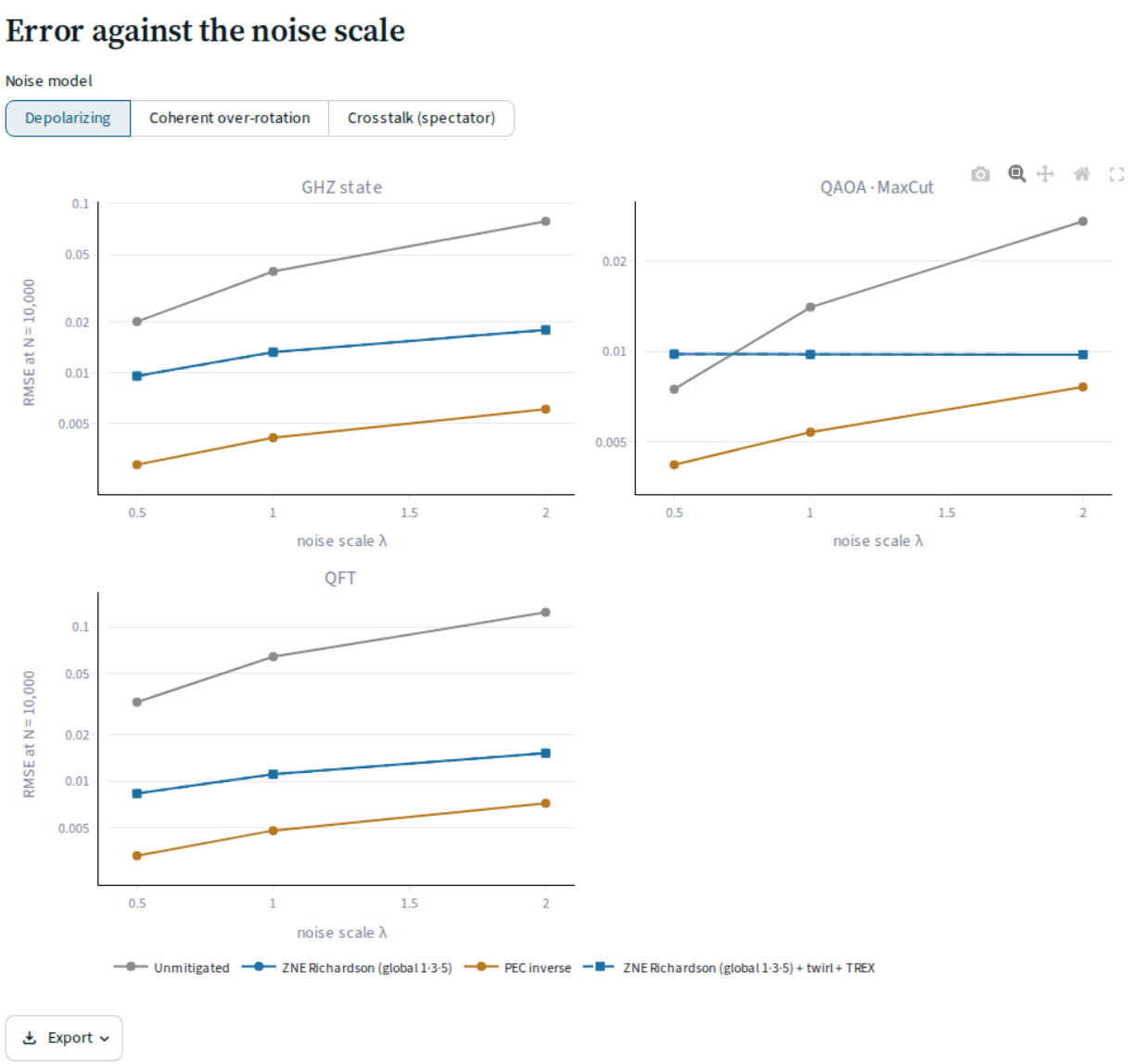


Figure 38: The winners and the error against the noise scale.

Download all (*ZIP*) bundles the full results table, the winners and every figure (PNG, SVG); the Playground's ZIP also includes the last batch.

Tutorial 23 — Sweeps: inputs, measurement bases and parameters

A sweep turns one batch into a controlled experiment: one quantity changes, everything else stays fixed.

1. **Set every qubit at once.** In the Playground's *Inputs and measurement* card, type a register input in **All qubits: input** — `plus`, `ramp:1`, `bits:0101` — and/or a basis in **All qubits: basis** — `X`, or one letter per qubit such as `ZZXY` (q0 first) — and press **Apply to all qubits**. The phase ramp $|\text{ramp}{:}k\rangle = F^\dagger|k\rangle = \frac{1}{\sqrt{N}}\sum_j e^{-2\pi ijk/N}|j\rangle$ is a product state: qubit q gets $(|0\rangle + e^{-2\pi ik2^q/N}|1\rangle)/\sqrt{2}$, shown as $|\varphi t\rangle$ (the phase t in turns, e.g. `φ.875`) in its input menu.

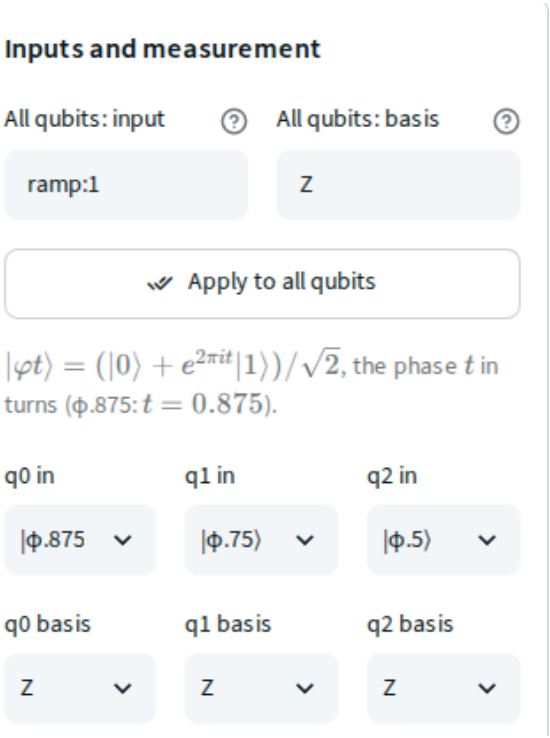


Figure 39: Register-wide input and basis: ramp:1 on every qubit.

2. **Sweep in Batch run.** Under *Sweep inputs, bases and parameters* write one swept quantity per line, `name: value, value, ….` Every combination becomes its own scenario for every algorithm it applies to:

line	sweeps
`ramp: 0, 1, 3`	the phase-ramp input on the whole register
`input: plus, 0, bits:0101`	register inputs
`basis: Z, X, ZZXY`	measurement bases
`m: 1, 2, 3`	any parameter of the ticked templates (here the AQFT degree)
`p2: 0.005, 0.01`	any noise parameter, replacing that rate in every noise model

Template parameters apply only to templates that have them, and noise parameters not to fake devices (their noise is the calibration). Inputs are prepared before the circuit, so a template that prepares its own input — the QFT's *output* s — is swept with its own parameter (`s: 0, 1`), while a QFT drawn in the Playground takes `ramp:` inputs directly.

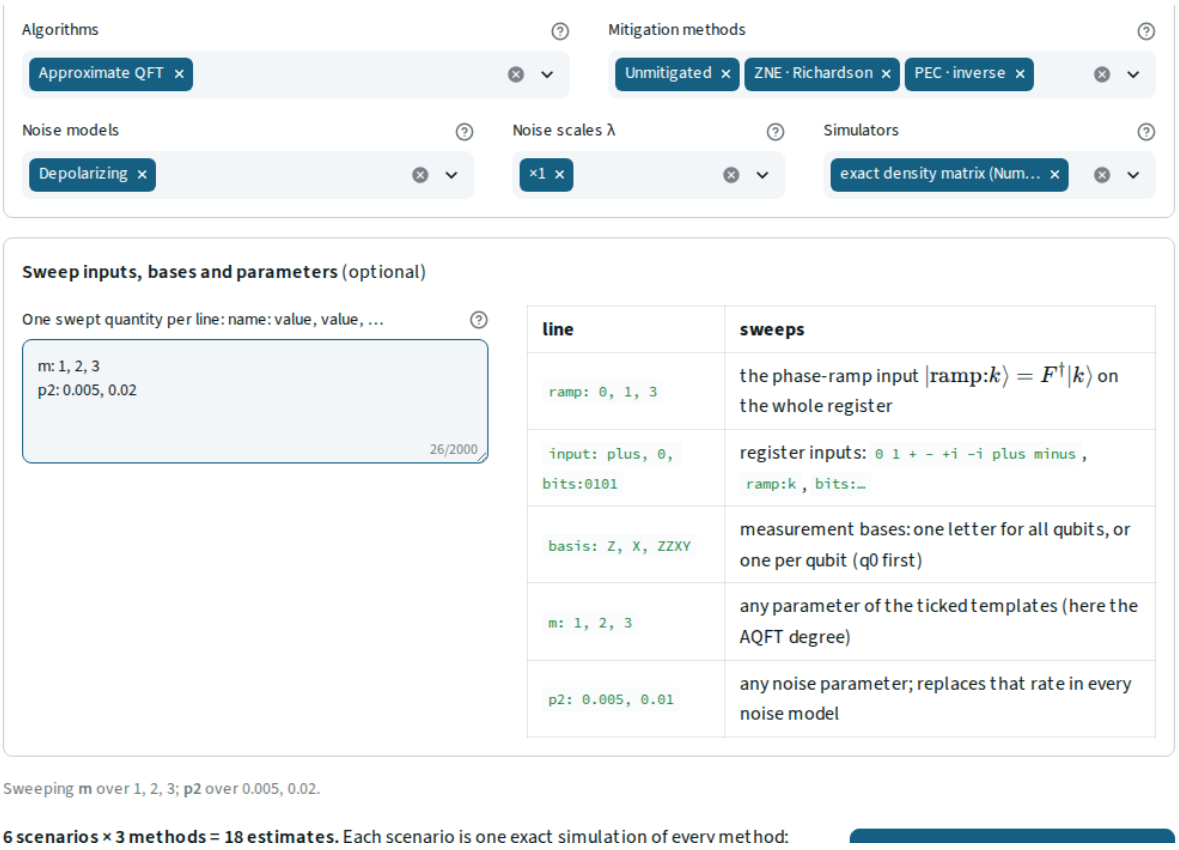


Figure 40: The sweep panel: every AQFT degree m at two two-qubit error rates.

3. Press **Run all**. The heat map gets one row per variant, and **Error against the swept values** plots the metric against the quantity you choose, one panel per combination of everything else.

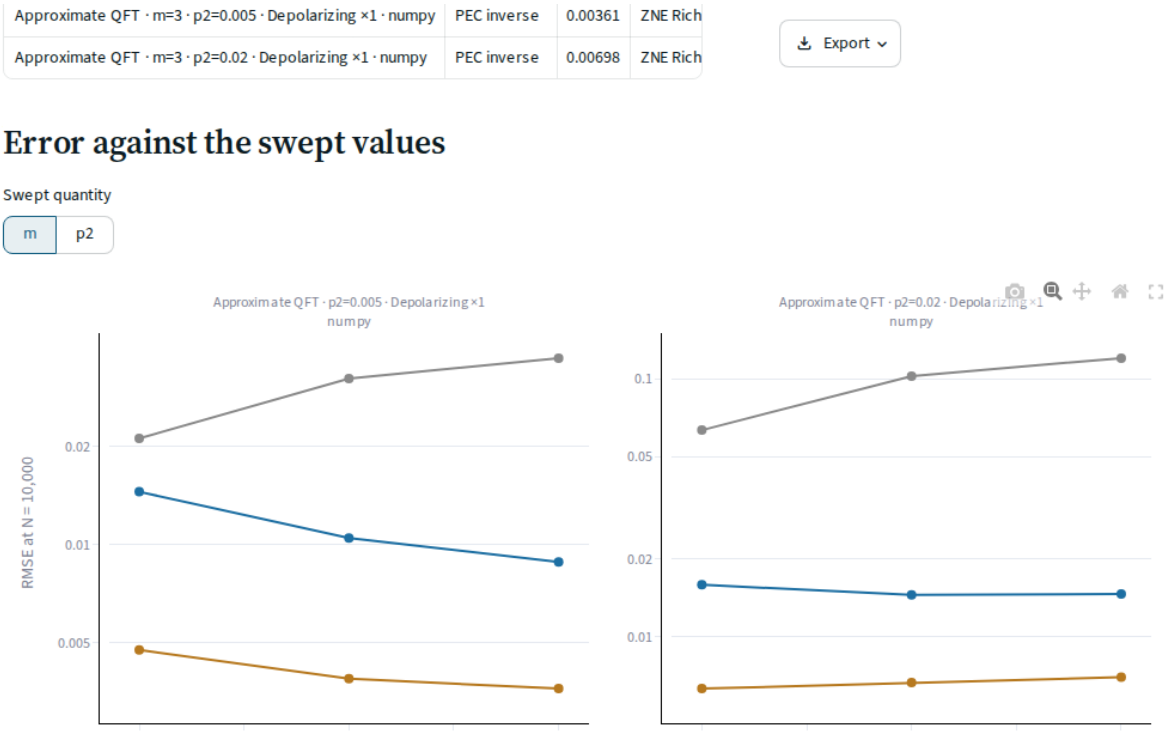


Figure 41: Error against the AQFT degree m, one panel per two-qubit error rate.

A worked example: why `plus` hides truncation. Draw a 4-qubit QFT in the Playground (no input preparation), choose the observable $P(0001)$ and sweep `input: plus, ramp:1` on an exact and an approximate QFT. On $|+\cdots+\rangle$ every controlled phase meets a qubit already in $|0\rangle$, so dropping phases changes nothing: the benchmark is blind to the truncation. `ramp:1` exposes it — its ideal value falls from 1 to $|\langle 0001|\,\mathrm{AQFT}_{m=2}|\mathrm{ramp{:}1}\rangle|^2 = 0.961940$ — which is the ceiling a mitigated AQFT estimate can reach. The tests check exactly these numbers.

Tutorial 24 — Learn, then look it up in the handbook

Learn has three tabs. *Hands-on guides* are seven exercises of 8–20 minutes, each with a *Set up the Playground* button and numbered steps. *Fundamentals* covers the quantum computing that error mitigation relies on (states and measurement, gates, density matrices, noise as a channel, expectation values and shot noise). *Algorithm walkthroughs* explain VQE, QAOA, Shor's algorithm, HHL and the QEC primitives in the app — what each circuit does, where noise hurts it, and which mitigation fits — each with a setup button.

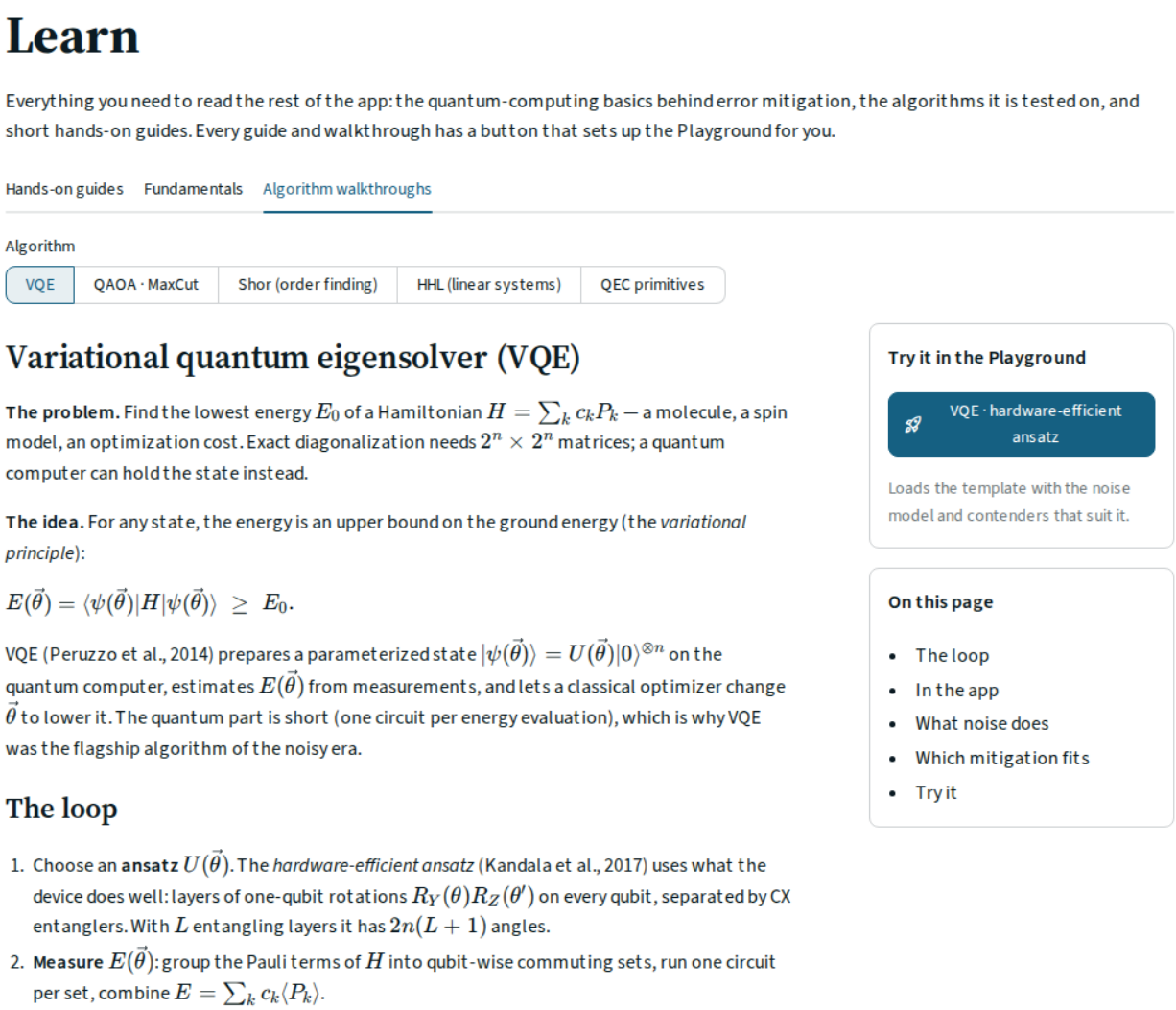


Figure 42: An algorithm walkthrough (VQE) with its one-click setup.

Theory & sources is now a research handbook in five parts: noise and the limits of mitigation; the methods in the app; the test circuits, compiler and simulators; advanced mitigation methods (symmetry verification and symmetry expansion, pulse-level ZNE and probabilistic error amplification,

quantum subspace expansion, classical shadows, M3 in depth); and the road to fault tolerance (stabilizer formalism, code taxonomy, decoding, the threshold theorem, logical error mitigation and a roadmap). Its full text is Part IV of this manual.

16.6 How the methods relate, and their limits

method	knowledge spent	bias removed	overhead	scaling with circuit size
readout (tensored, M3, TREX)	calibration of the measurement	readout only	$\prod_i (1 - 2\bar{\epsilon}_i)^{-2}$	per measured qubit
symmetry verification / expansion	a symmetry of the ideal state	errors that break it	$1/P_{\text{acc}}$	P_{acc} decays exponentially
ZNE (folding, pulse, PEA)	a noise knob λ	up to order λ^{m-1}	$(\sum_j \lvert\eta_j\rvert)^2$	modest; bias grows with depth
PEC	a full noise model	all modeled noise	$\gamma^2 = \prod_g \gamma_g^2$	exponential in total error
learning (CDR, NIL)	near-Clifford training data	the learned map	training circuits, $\lVert c\rVert_1^2$	depends on training fidelity
virtual distillation / QSE	copies / a subspace	incoherent errors	$\mathrm{Tr}(\rho^M)^{-2}$ / $O(m^2)$ terms	exponential in total error

Two general results frame all of them. First, lower bounds show that for broad classes of noise every unbiased mitigation protocol needs a number of samples exponential in circuit depth or size (§3.2) [Takagi 2022; Quek 2024]. Second, mitigation and correction compose: logical qubits have much smaller error rates, so the same estimators at the logical level cost exponentially less (§17.5) [Suzuki 2022]. A thesis that proposes a method should therefore report bias *and* sampling overhead as functions of the circuit's total error $\sum_g \epsilon_g$, as the Cost vs precision and Head-to-head pages do.

Figure 43: The handbook: advanced mitigation methods, with their relations and limits.

Part II-C · New in version 1.2

Version 1.2 makes every comparison an exact control experiment. All contenders spend the same total shot budget, every cost is counted the way it is paid, and a hardware run produces the same results as a simulation. The tutorials below cover these changes, Grover's search and the hidden subgroup problem, and the redesigned handbook.

Tutorial 25 — Equal budgets: reading the cost table

The sidebar's **Shot budget per method** is a *total*. Each contender splits it between three kinds of circuits:

$$B = N_t + F + c\,s.$$

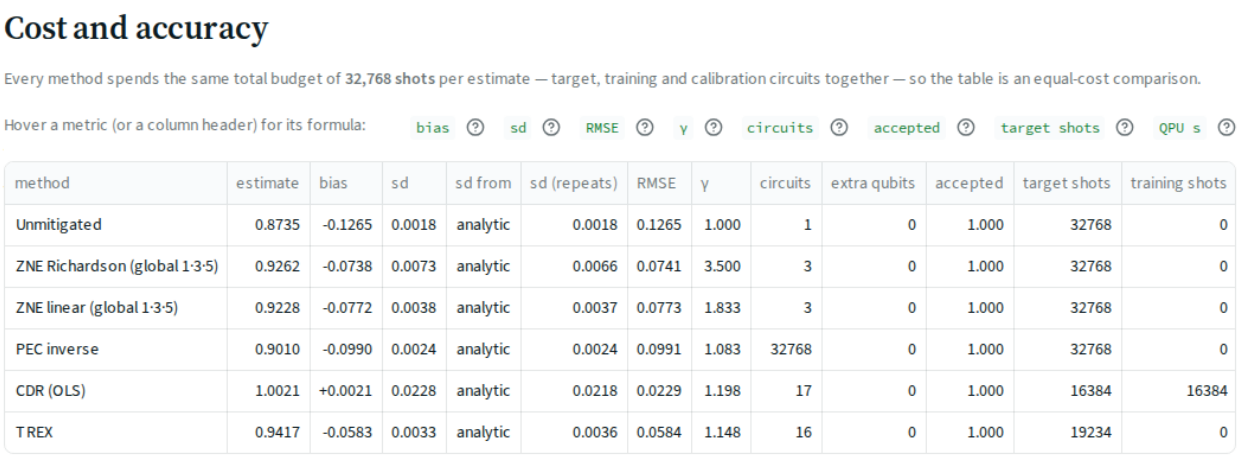
- N_t — **target shots**: the circuits that carry the estimate. Examples are ZNE's folded copies (split $N_j \propto |\eta_j|\sigma_j$), PEC's sampled circuits and the post-selected circuit of symmetry verification.
- F — **training shots** of CDR/NIL: training circuits × *shots per training circuit*.
- $c\,s$ — **calibration shots**: c readout-calibration circuits with s shots each. s is chosen to minimize the variance, $s^* = R\kappa/(1 + c\kappa)$ with $R = B - F$ and $\kappa = \sqrt{w/(cv)}$.

The standard error is then

$$\sigma = \sqrt{v/N_t + w/s + u}.$$

The three terms are the shot noise of the target circuits, the error of a finitely sampled calibration (delta method) and the shot noise of the fixed training data. The training noise does not shrink with B.

1. Run the default comparison and open **Mitigate & compare → Cost and accuracy**.
2. Compare *target shots*: Unmitigated, ZNE and PEC spend the whole budget on target circuits. CDR splits it between target shots and training shots (16×1024). TREX gives part of it to calibration.
3. Read *circuits*. ZNE sends one circuit per noise scale. PEC sends one freshly sampled circuit per shot (32,768 circuits, the textbook protocol). CDR sends its training circuits plus the target. TREX sends every circuit with 8 random measurement masks.
4. Compare *sd* with *sd (repeats)*, the spread of 120 simulated experiments at the same budget. They agree, because the repeats draw every part of the budget (target, training and calibration) as the analytic formula assumes.



Cost and accuracy

Every method spends the same total budget of **32,768 shots** per estimate — target, training and calibration circuits together — so the table is an equal-cost comparison.

Hover a metric (or a column header) for its formula: bias sd RMSE γ circuits accepted target shots QPU s

method	estimate	bias	sd	sd from	sd (repeats)	RMSE	γ	circuits	extra qubits	accepted	target shots	training shots
Unmitigated	0.8735	-0.1265	0.0018	analytic	0.0018	0.1265	1.000	1	0	1.000	32768	0
ZNE Richardson (global 1·3·5)	0.9262	-0.0738	0.0073	analytic	0.0066	0.0741	3.500	3	0	1.000	32768	0
ZNE linear (global 1·3·5)	0.9228	-0.0772	0.0038	analytic	0.0037	0.0773	1.833	3	0	1.000	32768	0
PEC inverse	0.9010	-0.0990	0.0024	analytic	0.0024	0.0991	1.083	32768	0	1.000	32768	0
CDR (OLS)	1.0021	+0.0021	0.0228	analytic	0.0218	0.0229	1.198	17	0	1.000	16384	16384
TREX	0.9417	-0.0583	0.0033	analytic	0.0036	0.0584	1.148	16	0	1.000	19234	0

Figure 44: The equal-budget cost table: every row spends 32,768 shots.

PEC's shots per circuit. In the PEC contender's parameters, *shots per sampled circuit s* trades circuits for precision. With $s > 1$ the shots of one circuit share its sign and Pauli insertions. The per-shot variance becomes $v_1 + (s-1)\,\mathrm{Var}_{\mathrm{inst}}$, and the table shows $\lceil N_t/s \rceil$ circuits.

Cost vs precision. The same accounting gives RMSE(B) on the *Cost vs precision* page. The table there reports the budget each method needs for a target ε. It shows ∞ when the bias (or, for CDR/NIL, the training-noise floor) exceeds ε. It also gives the crossover budget above which a method beats the unmitigated estimate.

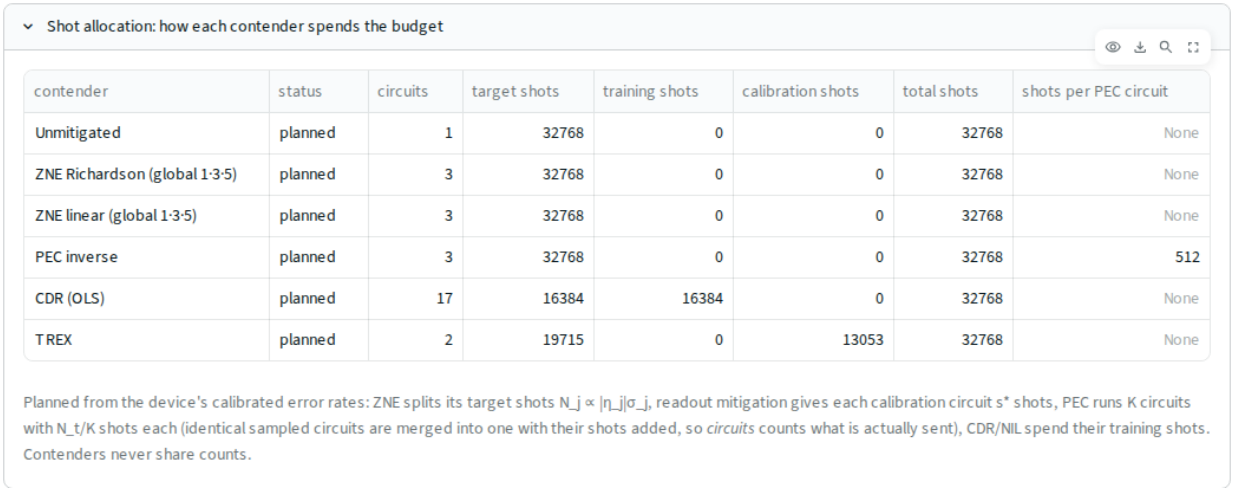
method	bias	per-shot σ	budget for ε	QPU time for ε (s)	within budget	beats unmitigated above B
Unmitigated	-0.1265	0.332	∞	∞	no	–
ZNE Richardson (global 1·3·5)	-0.0738	1.324	∞	∞	no	156
ZNE linear (global 1·3·5)	-0.0772	0.696	∞	∞	no	37.1
PEC inverse	-0.0990	0.426	∞	∞	no	11.4
CDR (OLS)	+0.0021	0.398	∞	∞	no	1.64e+04
TREX	-0.0583	0.355	∞	∞	no	20.2

Figure 45: Cost vs precision: the budget each method needs for ε = 0.01. Here none reaches it, because the bias or, for CDR, the training-noise floor is larger than ε.

Tutorial 26 — The same experiment on hardware

1. Open **Run on IBM Quantum**, keep *Local test on a fake backend*, and press **Run locally**.
2. Open **Shot allocation: how each contender spends the budget**. The planner allocates each contender's budget exactly as the simulator's accounting does:
   - ZNE's optimal split over its scales.
   - The optimal calibration shots s^*.
   - PEC's K sampled circuits with N_t/K shots each.
   - CDR's training shots.

   The planner predicts each circuit's outcome from the device's calibrated error rates. Contenders never share counts.



Shot allocation: how each contender spends the budget

contender	status	circuits	target shots	training shots	calibration shots	total shots	shots per PEC circuit
Unmitigated	planned	1	32768	0	0	32768	None
ZNE Richardson (global 1·3·5)	planned	3	32768	0	0	32768	None
ZNE linear (global 1·3·5)	planned	3	32768	0	0	32768	None
PEC inverse	planned	3	32768	0	0	32768	512
CDR (OLS)	planned	17	16384	16384	0	32768	None
TREX	planned	2	19715	0	13053	32768	None

Planned from the device's calibrated error rates: ZNE splits its target shots N_j ∝ |η_j|σ_j, readout mitigation gives each calibration circuit s* shots, PEC runs K circuits with N_t/K shots each (identical sampled circuits are merged into one with their shots added, so *circuits* counts what is actually sent), CDR/NIL spend their training shots. Contenders never share counts.

Figure 46: The hardware plan: every contender spends the same 32,768 shots.

3. Scroll down. The results view is the Playground's: headline cards, the analysis panel (here the *measured* output distribution with the readout-corrected and ZNE-extrapolated ones), estimates, the spread of repeated experiments, the equal-budget cost table and every method's internals. *sd* is the standard error of the allocation that was actually run.

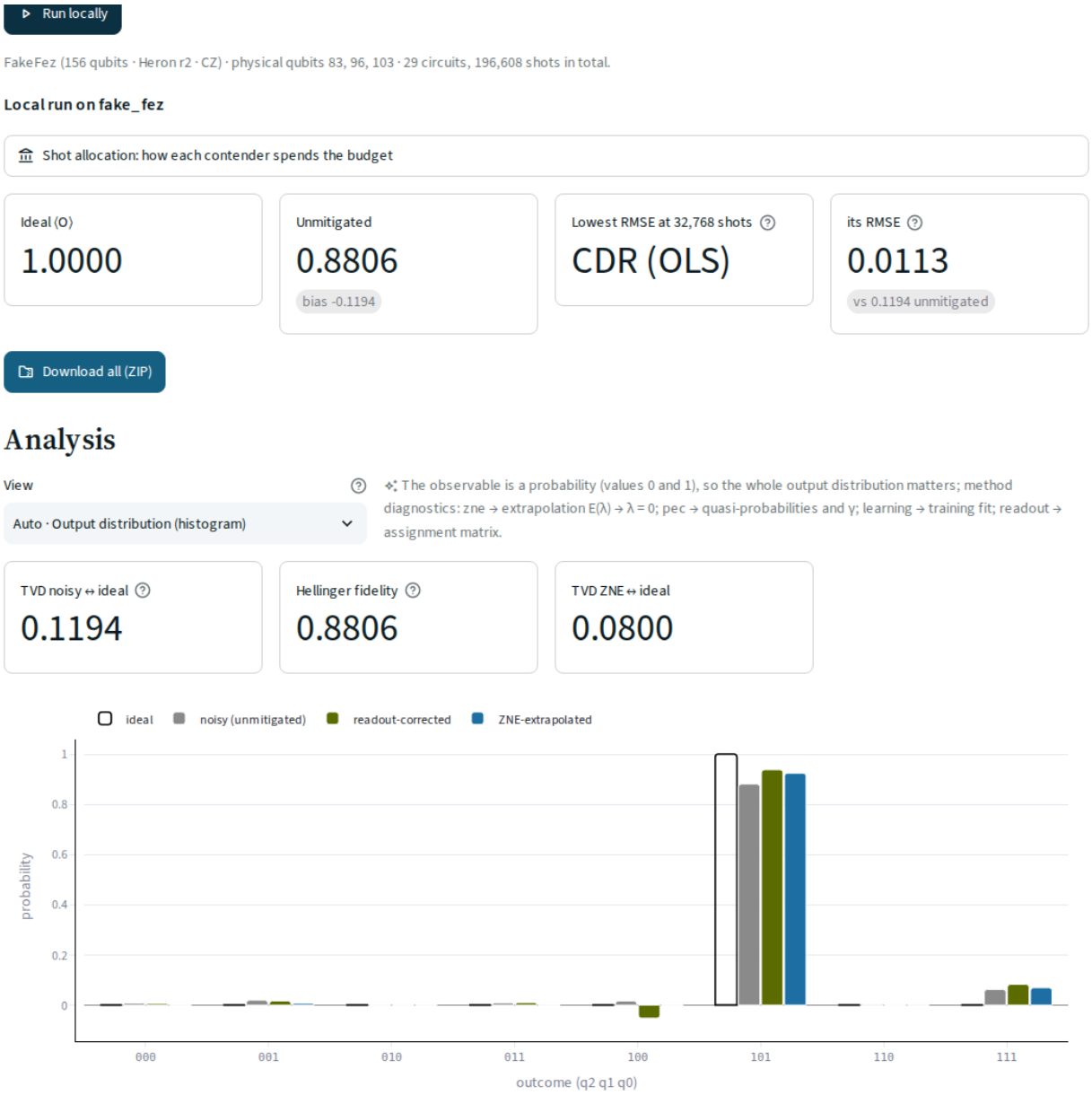


Figure 47: A local rehearsal on FakeFez: the Playground's results view on measured counts.

4. **Download all (ZIP)** holds the same files as the Playground's ZIP. It also includes the hardware plan, every circuit request with its role and shots, and, on real hardware, the job IDs. It never includes the token.

On real hardware the circuits are submitted as one job per option group (twirling, DD) and shot count, inside a Batch when the account allows it.

Tutorial 27 — One ZIP with everything

The Playground's **Download all (ZIP)** computes every cross-page result of the comparison when you press it, so one file documents the whole experiment:

```
README.txt                    what is inside, settings, date
tables/comparison/            cost table, estimates (CSV + LaTeX)
tables/cost_precision/        budget and QPU time for ε, crossovers, RMSE against budget
tables/pec_wall/              Γ, shots and seconds against depth for each p2
tables/bias_variance/         bias², variance and MSE per method
```

```
tables/batch, depth_sweep, …      other pages' stored results, when they exist
data/repeated_experiments.csv     the simulated repeats behind sd (repeats)
figures/comparison/…  figures/cost_precision/…  figures/pec_wall/…   PNG (3×) and SVG
state/qem_explorer_state.json     the app state (no credentials)
script/qem_explorer_export.py     the standalone Qiskit script
```

No HTML files are written. Without Chrome or Chromium each figure is saved as Plotly JSON, which `plotly.io.read_json(path).show()` reopens interactively. The README explains how to enable PNG/SVG.

Tutorial 28 — Inputs, bases and π in the composer

1. On **Build → Inputs and measurement**, type `ramp:1` into *All qubits: input* and `XZY` into *All qubits: basis*, then press **Apply to all qubits**.
2. The composer draws the state preparation as locked boxes on each wire, in front of the first column. For example H·P(7π/4) prepares $|\varphi.875\rangle$ on q0. The basis changes appear in front of the measurement: H for X, and S†·H for Y. The measurement box shows the basis (Mx, My).

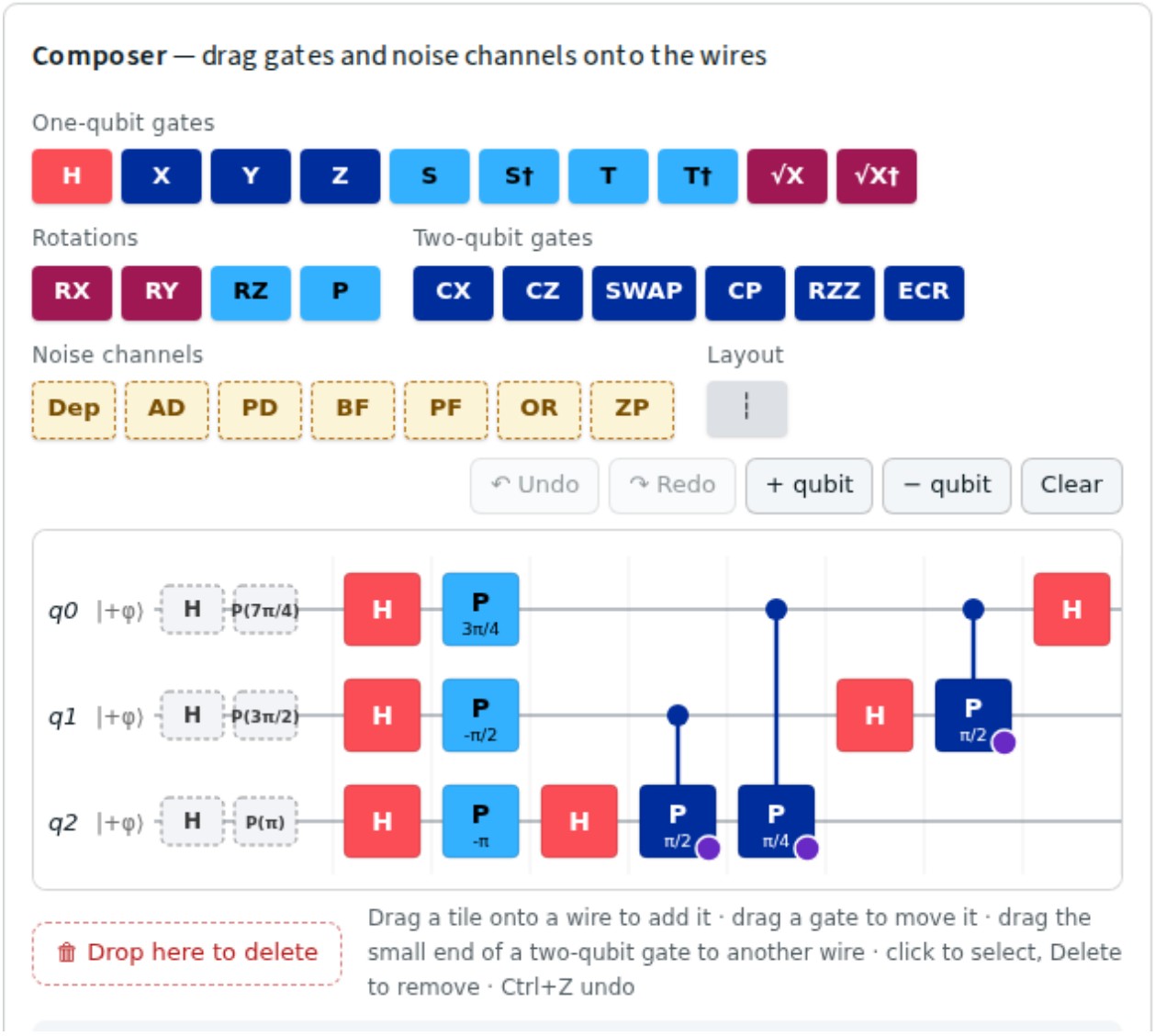


Figure 48: Register inputs and measurement bases drawn in the composer.

3. Angles can be typed the way you write them: `pi/4`, `π/4`, `\pi/128`, `Pi/3`, `2pi`, `3π/4`, `\frac{\pi}{8}`,

`2·π`, `−π/2`. Fields and labels show rational multiples of π as fractions up to $\pi/1024$. The expression reader is still a whitelist interpreter: `\frac` and implicit products are rewritten to arithmetic before parsing, never evaluated as code.

4. Hover a **?** icon. Formulas in tooltips are written in Unicode math, because the tooltip popup cannot typeset TeX reliably.

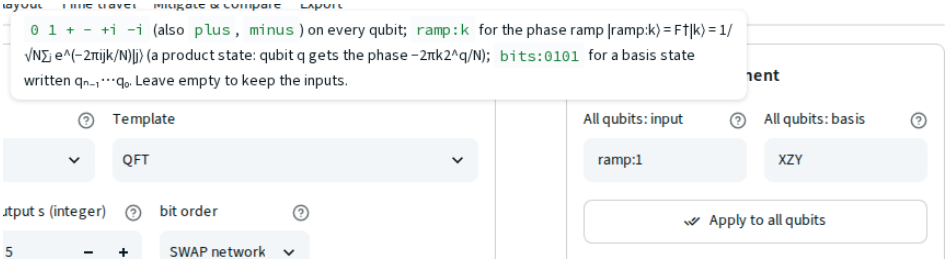


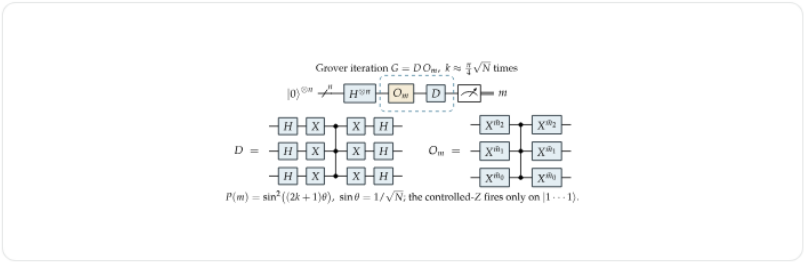
Figure 49: A tooltip with its formula in Unicode math.

Tutorial 29 — Grover's search and the hidden subgroup problem

1. Open **Learn → Algorithm walkthroughs → Grover's search** and press **Grover search, n = 3**. The template marks $m = 5$ among $N = 8$ items. After $k^* = 2$ iterations, $P(m) = \sin^2(5\theta) = 0.9453$ with $\sin\theta = 1/\sqrt{8}$.
2. Run the comparison. The output histogram shows the marked item standing out of a uniform floor. Noise lowers the peak and raises the floor. Readout mitigation and ZNE restore part of the peak.
3. Press **Grover, 3 iterations (overshoot)**. The rotation passes the target and $P(m)$ falls to 0.33: Grover's algorithm is periodic, not monotone.

Grover's search

The problem. Find the one marked item m among $N = 2^n$ items, given only an oracle that recognizes it. A classical search needs about $N/2$ queries on average; Grover's algorithm (1996) needs about $\frac{\pi}{4}\sqrt{N}$ — a quadratic speed-up that is provably optimal for unstructured search [Bennett et al. 1997].



The idea: a rotation in a two-dimensional plane

Start from the uniform superposition $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n}$ and write it in the plane spanned by the marked state $|m\rangle$ and the normalized rest $|m^\perp\rangle$:

$$|s\rangle = \sin\theta\,|m\rangle + \cos\theta\,|m^\perp\rangle, \qquad \sin\theta = \tfrac{1}{\sqrt{N}}.$$

One **Grover iteration** $G = D\,O_m$ is the product of two reflections:

- the **oracle** $O_m = I - 2|m\rangle\langle m|$ flips the sign of the marked amplitude (a reflection about $|m^\perp\rangle$);
- the **diffuser** $D = 2|s\rangle\langle s| - I$ reflects about the uniform state.

Two reflections make a rotation by 2θ, so after k iterations

$$P(m) = \sin^2\big((2k+1)\theta\big), \qquad k^* = \left\lfloor \tfrac{\pi}{4\theta} \right\rfloor \approx \tfrac{\pi}{4}\sqrt{N}.$$

Iterating further rotates *past* the target: Grover's algorithm is periodic, not monotone.

Try it in the Playground

Grover search, n = 3

Grover, 3 iterations (overshoot)

Loads the template with the noise model and contenders that suit it.

On this page

- The idea: a rotation in a two-dimensional plane
- In the app
- What noise does
- Which mitigation fits
- Try it

Figure 50: The Grover walkthrough with its circuit diagram.

4. Choose **Hidden subgroup problem**. For $G = \mathbb{Z}_2^n$ the oracle $f(x) = Ax$ is constant exactly on the cosets of $H = \mathrm{span}\{h_1, h_2\}$. The Hadamard transform then returns only strings y with $y \cdot h = 0$ for every $h \in H$, uniformly over $H^\perp$. The observable $P(y \in H^\perp)$ is 1 ideally, and noise leaks probability outside $H^\perp$. Symmetry verification on the parities $y \cdot h_i$ (proposed automatically) discards those shots.
5. For $G = \mathbb{Z}_N$, $N = 2^n$, the oracle is $f(x) = x \bmod 2^j$ and the QFT returns multiples of $N/2^j$: period finding, the core of Shor's algorithm.

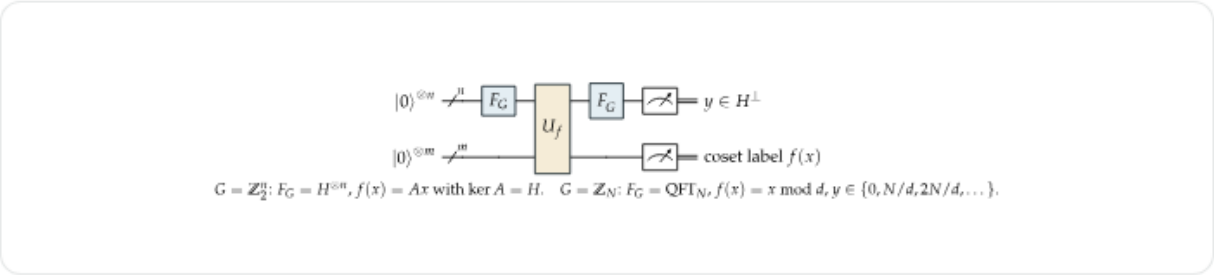


Figure 51: The hidden subgroup circuit for both groups.

Tutorial 30 — The redesigned handbook

Theory & sources now reads like a book:

- five parts with a contents column;
- a strict hierarchy of sections, subsections and sub-subsections;
- a narrower measure with wider line spacing;
- a typeset circuit diagram for every algorithm.

Search still covers the whole text. Part IV of this manual is the same handbook.

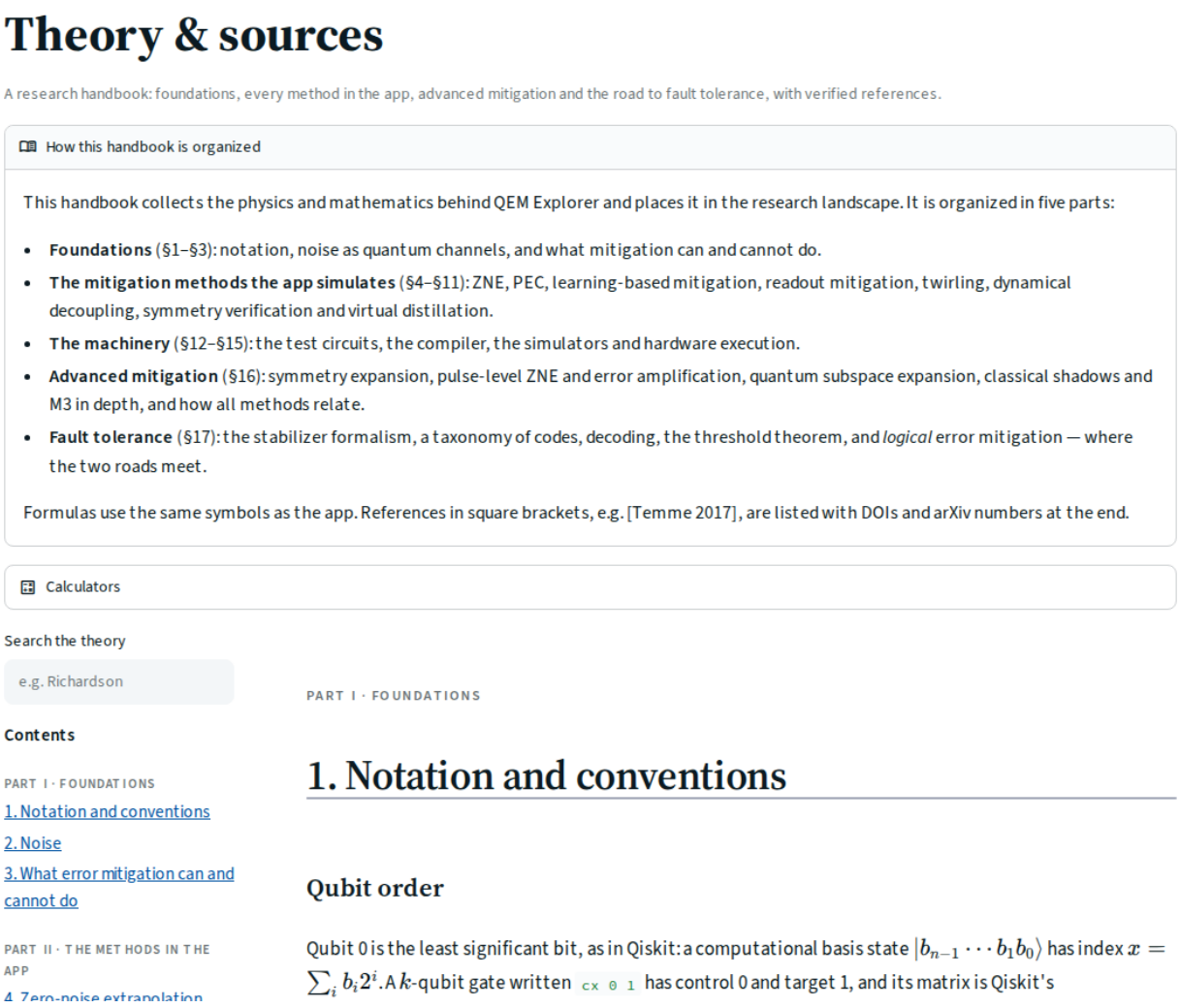


Figure 52: The handbook: parts, contents and a wider reading measure.

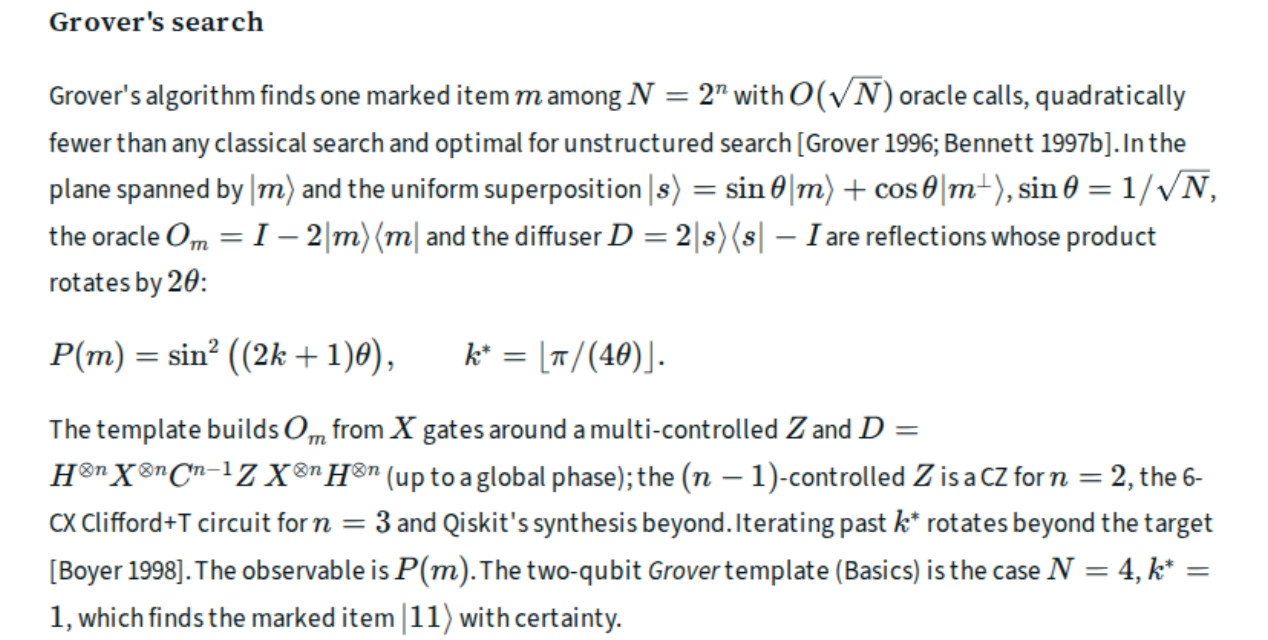


Figure 53: Grover's search in the handbook (§12.5).

What was verified. Two independent code-level audits compared every algorithm and estimator with its formal definition. Examples:

- The QFT equals the DFT matrix.
- Grover's success probability equals $\sin^2((2k+1)\theta)$.
- HSP outcomes are uniform on $H^\perp$.
- The ZNE weights satisfy $\sum_j \eta_j \lambda_j^k = \delta_{k0}$.
- PEC's quasi-probabilities invert the channel.
- M3 equals the tensored inverse.
- Symmetry verification's and virtual distillation's variances match brute-force sampling.

The release notes list every fix. `tests/test_v12.py` keeps these properties under test.

Part III · Reference

Noise injector parameters

All parameters are typed numbers next to a slider. Probabilities are limited to $[0, 1]$; angles and rates have no upper limit in the number field (the slider covers the usual range).

Parameter	Symbol	Meaning	Where it acts
`p1`	p_1	depolarizing probability, $\rho \mapsto (1-p)\rho + p\,I/2$	after every one-qubit gate
`p2`	p_2	two-qubit depolarizing probability, $\rho \mapsto (1-p)\rho + p\,I/4$	after every two-qubit gate
`amp_damp`	γ	amplitude damping (T_1-like)	after every gate, on each of its qubits
`phase_damp`	λ	phase damping (T_2-like)	after every gate
`kick`	κ	random-axis rotation by κ (rad), averaged	after every gate
`over_rot`	ε	coherent over-rotation, a fraction of each pulse	pulsed one-qubit gates
`zz_phase`	θ_{ZZ}	coherent ZZ phase (rad)	after two-qubit gates
`z_phase`	φ	coherent Z phase (rad)	after every gate
`field_sigma`	σ	quasi-static random detuning (rad per idle layer), averaged over stratified samples	idle qubits
`idle_dephase`	λ_{idle}	Markovian dephasing per idle layer	idle qubits
`idle_damp`	γ_{idle}	amplitude damping per idle layer	idle qubits
`ro01`, `ro10`	$\epsilon_{01}, \epsilon_{10}$	readout flips $P(1 \mid 0)$, $P(0 \mid 1)$	measurement
`xtalk_z`	δ_x	spectator $R_Z(\delta_x)$ on every idle neighbor of a driven qubit	each gate's idle neighbors on the coupling map
`xtalk_zz`	ζ	$R_{ZZ}(\zeta)$ between a driven qubit and each idle neighbor	each gate's idle neighbors

Noise **channels placed in the circuit** (palette row *Noise channels*, or text) act exactly where you put them: `depol(p)` on any number of qubits, `ampdamp(γ)`, `phasedamp(λ)`, `bitflip(p)`, `phaseflip(p)`, `overrot(ε)` (a coherent $R_X(\varepsilon)$) and `zphase(φ)`. They are excluded from the ideal reference.

Text syntax

```
# comments start with '#'
h 0
cx 0 1                # CNOT, control first
rz(pi/8) 1            # angles: numbers or expressions
```

```
cp(-pi/4) 0 2
depol(0.05) 0 1        # a noise channel on qubits 0 and 1
barrier                # all qubits (or: barrier 0 1)
```

Gates: `id x y z h s sdg t tdg sx sxdg rx ry rz p cx cz swap cp rzz ecr`.

Hamiltonian observables (*Observable → Hamiltonian: a sum of Pauli strings…*). A real linear combination of Pauli strings, one term after another; a new line also starts a new term:

```
0.5*Z0*Z1 + 0.2*X0 - 0.1*Y1
- Z0*Z1 - Z1*Z2 - X0 - X1 - X2
sqrt(2)/2 * X0 X1
```

The second line is the transverse-field Ising model with $J = h = 1$; the third shows that `*`, a space or nothing joins factors. Comments are not allowed in this field.

Factors are `X3`, `Y_3` or `Z3` (qubit index after the letter); coefficients use the same safe expression reader (at most 10^9 in size). A number directly after a Pauli factor, `**` next to a Pauli and division by a Pauli are refused with a message. Terms that agree qubit by qubit share one measurement setting; every method runs on each setting and the results are combined exactly. The menu offers ready-made examples for the current number of qubits (transverse-field Ising, Heisenberg XXX, MaxCut on a ring, and a mix to edit).

Initial layout (*Topology & layout*, or the Compilation card): physical qubits in logical order, e.g. `4 6 11` places q0 → p4, q1 → p6, q2 → p11. Only ASCII digits, spaces and commas are accepted; a layout that does not fit the coupling map (a qubit off the lattice, a repeated qubit) is ignored with a warning and compilation falls back to SABRE.

Sweeps (*Batch run → Sweep inputs, bases and parameters*) and **register inputs** (*All qubits: input / basis*):

```
ramp: 0, 1, 3              # phase-ramp inputs ramp:k = F†|k>
input: plus, 0, bits:0101  # 0 1 + - +i -i plus minus, ramp:k, bits:<q_(n-1)...q_0>
basis: Z, X, ZZXY          # one letter for all qubits, or one per qubit (q0 first)
m: 1, 2, 3                 # a parameter of the ticked templates
p2: 0.005, 0.01            # a noise parameter (replaces that rate in every noise model)
```

Lines may also be separated by `;`, and `#` starts a comment. Numbers go through the safe expression reader (`2*0.01`, `pi/4`), choices must match a template's options, probabilities must lie in $[0, 1]$. At most 2000 characters, 6 swept quantities and 64 values each; a sweep with more than 5000 combinations is refused as a probable typo. A phase input is stored per qubit as `ph:<turns>`, e.g. `ph:0.875` for $(|0\rangle + e^{2\pi i \cdot 0.875}|1\rangle)/\sqrt{2}$, and is saved, loaded and exported like the other inputs.

Further symmetry checks (*Symmetry verification → further parities*): groups of qubits separated by `;`, e.g. `2 3; 5 6` — each group's Z-parity must be +1 for a shot to be kept, in addition to the main check. Expressions: numbers, `+ - * / **`, parentheses, `pi` (or π), `e`, `tau`, and `sqrt sin cos asin acos atan`. At most 200 characters per parameter; large exponents are refused; nothing else is accepted, so a parameter can never run code.

Composer interactions

Action	How
Add	drag a palette tile onto a wire (and column)
Move	drag a gate
Retarget a two-qubit gate	drag the small violet handle at its second end
Delete	drop on *Drop here to delete*, or select and press Delete

Action	How
Edit an angle	double-click the gate, or select it and use the parameter field
Flip / duplicate	select the gate, then the buttons under the circuit
Undo / redo	buttons, or Ctrl+Z / Ctrl+Y
Add / remove qubits	*+ qubit* / *− qubit*

The comparison table

Column	Meaning
estimate	the method's exact mean
bias	estimate − ideal
sd	standard error of one experiment at the budget B, $\sqrt{v/N_t + w/s + u}$; *sd from*: analytic or from repeats
sd (repeats)	spread of 120 simulated experiments
RMSE	$\sqrt{\text{bias}^2 + \text{sd}^2}$
γ	sampling overhead (standard-deviation amplification)
circuits	circuits sent to the backend: one per ZNE scale, one per sampled PEC circuit (N_t/s), one per shot with twirling, ×8 measurement masks with TREX, plus training and calibration circuits
extra qubits	e.g. the copies of virtual distillation
accepted	post-selection acceptance (symmetry verification)
target shots	$N_t = B - F - cs$: shots on the circuits that carry the estimate
training shots	F: CDR/NIL training circuits × shots each
calibration shots	cs: readout-calibration circuits × the optimal shots each, s^*
QPU s	B× time per shot — the same for every method
status	*not applied …* or *weak training signal* when a result should not be read at face value

Templates and their parameters

Basics

Template	Parameters (default; range or options)	Default observable
Bell pair (2 qubits by default)	–	⟨Z0 Z1⟩
GHZ state (3 qubits by default)	qubits = 3 (≥ 2)	⟨Z0 Z2⟩
Grover, 2 qubits (2 qubits by default)	–	P(11)

Algorithms

Template	Parameters (default; range or options)	Default observable
QFT (3 qubits by default)	qubits = 3 (≥ 2); output s (integer) = 5 (0 – max (set by the other parameters)); bit order = swap (swap / bitreverse / relabel)	P(101)
Approximate QFT (4 qubits by default)	qubits = 4 (≥ 3); output s (integer) = 1 (0 – max (set by the other parameters)); bit order = swap (swap / bitreverse / relabel); AQFT degree m = 1 (1 – max (set by the other parameters))	P(0001)
Phase estimation (4 qubits by default)	counting qubits = 3 (≥ 1); phase φ = 0.25 (0.0 – 1.0)	P(j=2)
QPE of an Ising Hamiltonian (5 qubits by default)	counting qubits = 3 (≥ 1); Jτ/2π = 0.375 (0.0 – 1.0); eigenstate = 01 (00 / 01 / 10 / 11)	P(j=3)
Ising (Trotter) (3 qubits by default)	spins = 3 (≥ 2); Trotter steps = 2 (≥ 1); field h = 1.0 (0.0 – 5.0); time step = 0.25 (0.01 – 2.0)	⟨M⟩ = ⟨Σ Zi⟩/n
Bernstein–Vazirani (5 qubits by default)	input qubits = 4 (≥ 1); secret s (integer) = 11 (0 – max (set by the other parameters))	P(s=1011)
Simon (hidden subgroup) (4 qubits by default)	input qubits = 2 (≥ 1); secret s = 3 (1 – max (set by the other parameters))	⟨Z0 Z1⟩

NISQ · variational

Template	Parameters (default; range or options)	Default observable
VQE · hardware-efficient ansatz (4 qubits by default)	qubits = 4 (≥ 2); entangling layers = 2 (≥ 0); entanglement = linear (linear / circular / full); angles = VQE-optimized (VQE-optimized / random); seed = 7 (≥ 0)	⟨H⟩/(n-1), H = Σ Z(i)Z(i+1)
QAOA · MaxCut (4 qubits by default)	nodes (qubits) = 4 (≥ 2); graph = ring (ring / path / 3-regular / complete / random); QAOA depth p = 1 (≥ 1); angles = optimized (optimized / fixed); γ (fixed) = 0.6 (-3.141592653589793 – π); β (fixed) = 0.4 (-3.141592653589793 – π); seed = 1 (≥ 0)	⟨C⟩/C_max (MaxCut)
QML · ZZ feature map classifier (3 qubits by default)	qubits = features = 3 (≥ 2); feature-map repetitions = 2 (≥ 1); ansatz repetitions = 2 (≥ 0); data seed = 0 (≥ 0); weight seed = 1 (≥ 0)	⟨Z0 Z1 Z2⟩
QML · data re-uploading (2 qubits by default)	qubits = features = 2 (≥ 1); layers = 3 (≥ 1); data seed = 0 (≥ 0); weight seed = 1 (≥ 0)	⟨Z0⟩

FTQC · error correction

Template	Parameters (default; range or options)	Default observable
3-qubit bit-/phase-flip code (5 qubits by default)	code = bit-flip (bit-flip / phase-flip); decoder = syndrome + correction (syndrome + correction / syndrome only (no correction) / majority vote readout); logical θ (RY) = 1.0472 (0.0 – 2π); error = none (none / one error on qubit k / channel on every data qubit); qubit k = 0 (0 – 2); channel p = 0.1 (0.0 – 1.0)	⟨Z0⟩
Repetition code memory (distance d) (5 qubits by default)	distance d (odd) = 3 (≥ 3); syndrome rounds = 1 (≥ 0); logical θ (RY) = 0.0 (0.0 – 2π); bit-flip p per exposure (r + 1 exposures) = 0.05 (0.0 – 1.0)	⟨Z_L⟩ (majority of 3)
Surface code d = 2 (one plaquette) (4 qubits by default)	logical θ (RY) = 1.0472 (0.0 – 2π); error = none (none / X on qubit k / Y on qubit k / Z on qubit k / bit-flip channel on every data qubit); data qubit k = 0 (0 – 3); channel p = 0.05 (0.0 – 1.0); syndrome = transversal readout (transversal readout / ancilla extraction (3 ancillas))	⟨Z0 Z2⟩
Five-qubit code [[5,1,3]] (Laflamme) (5 qubits by default)	logical θ (RY) = 1.0472 (0.0 – 2π); error = none (none / X on qubit k / Y on qubit k / Z on qubit k / depolarizing channel on every qubit); qubit k = 0 (0 – 4); channel p = 0.05 (0.0 – 1.0); logical readout = decoded (lookup table) (decoded (lookup table) / raw (no correction))	⟨Z_L⟩ decoded (5-qubit code)
Steane [[7,1,3]] code (7 qubits by default)	logical θ (RY) = 1.0472 (0.0 – 2π); error = none (none / X on qubit k / Z on qubit k / bit-flip channel on every qubit); qubit k = 0 (0 – 6); channel p = 0.05 (0.0 – 1.0); logical readout = decoded (decoded / raw parity)	⟨Z_L⟩ decoded

FTQC · deep algorithms

Template	Parameters (default; range or options)	Default observable
Shor order finding, N = 15 (7 qubits by default)	base a = 7 (2 / 4 / 7 / 8 / 11 / 13 / 14); counting qubits = 3 (≥ 2); powers $U^{(2^k)}$ = precomputed (precomputed / repeated); multiplier = permutation (permutation / synthesized)	P(y gives r=4)

Template	Parameters (default; range or options)	Default observable
Grover search (3 qubits by default)	qubits (N = 2^n items) = 3 (≥ 2); marked item m (integer) = 5 (0 – max (set by the other parameters)); iterations = optimal (optimal / custom); k (custom) = 2 (≥ 0)	P(101)
Hidden subgroup problem (abelian) (4 qubits by default)	group G = Z2^n (Simon-type) (Z2^n (Simon-type) / Z_N cyclic, N = 2^n (QFT)); input qubits n = 3 (≥ 2); generator h1 (Z2^n) = 5 (1 – max (set by the other parameters)); generator h2 (Z2^n; 0 = none) = 3 (0 – max (set by the other parameters)); H = ⟨2^j⟩ (Z_N) = 2 (1 – max (set by the other parameters))	P(y ∈ H⊥)
HHL, 2×2 system (4 qubits by default)	–	$\langle P_1$(anc) Z_b$\rangle$

Methods and their parameters

Unmitigated (`raw`, family: raw)

No parameters.

Zero-noise extrapolation (`zne`, family: zne)

Parameter	Kind	Default	Range / options	Meaning
noise scales λ_j	floats	(1.0, 3.0, 5.0)	–	Odd integers fold whole circuits; other values fold part of the circuit.
folding	choice	global	global / local	global: U(U†U)^k; local: every two-qubit gate G → G(G†G)^k.
extrapolation	choice	richardson	richardson / linear / exponential	

Probabilistic error cancellation (`pec`, family: pec)

Parameter	Kind	Default	Range / options	Meaning
construction	choice	inverse	inverse / compensation	

Parameter	Kind	Default	Range / options	Meaning
noise model	choice	depolarizing	depolarizing / learned	depolarizing: told p_1, p_2 below; learned: the exact Pauli-twirled channel of every gate.
assumed p_1 (one-qubit gates)	float	(blank)	0.0 – 0.2	Blank = the injector's true value.
assumed p_2 (two-qubit gates)	float	(blank)	0.0 – 0.3	Blank = the injector's true value.
gates corrected	choice	all	all / 2q	
shots per sampled circuit	int	1	1 – 100000	The protocol samples one circuit (one Pauli correction per gate) for every this many shots. 1 is the textbook estimator (Temme et al.): every shot is an independent sample, Var = $(\gamma^2\langle o^2\rangle - \langle O\rangle^2)/N$. Larger values send fewer circuits and add the instance-to-instance spread.
instances simulated (Aer)	int	64	8 – 1024	Aer cannot apply negative weights, so its mean is estimated from this many sampled instances (the NumPy simulator computes the exact quasi-probability mean).

Learning-based (CDR / NIL) (`learning`, family: learning)

Parameter	Kind	Default	Range / options	Meaning
features	choice	cdr	cdr / nil	cdr: the noisy $\langle O\rangle$ only; nil: plus $\langle Z_i\rangle$, $\langle Z_iZ_j\rangle$.
regression	choice	ols	ols / ridge / lasso	
penalty α	float	0.001	0.0 – 1.0	
training circuits	int	16	4 – 64	

Parameter	Kind	Default	Range / options	Meaning
non-Clifford gates kept	int	0	0 – 64	Rotations left at their true angle in every training circuit (more = training circuits closer to the target). With the adaptive strategy this is the starting value.
training-set strategy	choice	adaptive	adaptive / fixed	adaptive: diversify the Clifford sampling and keep more non-Clifford gates until the training values vary; fixed: exactly the settings above.
min. spread of training values	float	0.05	0.0 – 1.0	Required standard deviation of the ideal training values, as a fraction of the observable's range max(o) − min(o).
shots per training circuit	int	1024	64 – 65536	Training shots are part of the method's total budget (training circuits × this).

Symmetry verification (`symmetry`, family: symmetry)

Parameter	Kind	Default	Range / options	Meaning
parity of qubits	ints	(0, 1)	–	
ideal eigenvalue	choice	1	1 / -1	
further parities, each +1 (e.g. 2 3; 5 6)	text	(blank)	digits, spaces, commas, semicolons	More Z-parity checks that must all hold, separated by ';' — e.g. every Z check of a code. Leave empty for a single symmetry.

Virtual distillation (`vd`, family: vd)

Parameter	Kind	Default	Range / options	Meaning
copies M	choice	2	2 / 3 / 4	

Options every recipe can add

Option	Choices
Twirling	`none` no twirling; `2q` Pauli twirling (2q gates); `all` randomized compiling (all gates)
Dynamical decoupling	`none` no DD; `X2` DD: X2 (CPMG); `XY4` DD: XY4
Readout mitigation	`none` no readout mitigation; `tensored` tensored inverse; `m3` M3 (subspace); `trex` TREX (twirled readout)

Preset contenders

Selected by default: Unmitigated, ZNE · Richardson, ZNE · linear, PEC · inverse, CDR · OLS, Readout · TREX.

Preset	Recipe
Unmitigated	Unmitigated
ZNE · Richardson	ZNE Richardson (global 1·3·5)
ZNE · linear	ZNE linear (global 1·3·5)
ZNE · exponential	ZNE exponential (global 1·3·5)
ZNE · local folding	ZNE Richardson (local 1·3·5)
PEC · inverse	PEC inverse
PEC · compensation	PEC compensation
PEC · learned model + RC	PEC inverse (learned) + RC
CDR · OLS	CDR (OLS)
NIL · Ridge	NIL (Ridge)
NIL · Lasso	NIL (Lasso)
Readout · tensored	RO-inv
Readout · M3	M3
Readout · TREX	TREX
Symmetry verification	Symmetry Z0Z1=+1
Virtual distillation (M=2)	VD M=2
Pauli twirling (RC) only	RC
Dynamical decoupling (XY4)	DD-XY4
ZNE + RC + TREX	ZNE Richardson (global 1·3·5) + twirl + TREX

Noise presets

Preset	Non-zero parameters
Ideal (no noise)	none
Depolarizing	p1 = 0.002, p2 = 0.02
Amplitude damping	amp_damp = 0.02, idle_damp = 0.01
Coherent over-rotation	over_rot = 0.06
Fixed phase (ZZ)	zz_phase = 0.12
Random field (quasi-static)	field_sigma = 0.15
Markovian dephasing	phase_damp = 0.01, idle_dephase = 0.03
Readout only	ro01 = 0.02, ro10 = 0.05

Preset	Non-zero parameters
Crosstalk (spectator)	p1 = 0.0005, p2 = 0.005, xtalk_z = 0.06, xtalk_zz = 0.04
Hardware-realistic (illustrative)	p1 = 0.0005, p2 = 0.01, amp_damp = 0.002, phase_damp = 0.002, over_rot = 0.01, zz_phase = 0.02, field_sigma = 0.03, idle_dephase = 0.004, idle_damp = 0.002, ro01 = 0.015, ro10 = 0.03

Circuits you can inspect

Circuits tab (‘Implemented circuit of’)	Time travel (‘Inspect’)
Compiled only	Implemented circuit
ZNE (folding)	ZNE: folded circuit
PEC (sampled instance)	Dynamical decoupling
Pauli twirling	Pauli-twirled noise
Dynamical decoupling	
CDR / NIL training circuit	
Virtual distillation	

Part IV · Theory and sources — a research handbook

This handbook collects the physics and mathematics behind QEM Explorer and places it in the research landscape. It is organized in five parts:

- **Foundations** (§1–§3): notation, noise as quantum channels, and what mitigation can and cannot do.
- **The mitigation methods the app simulates** (§4–§11): ZNE, PEC, learning-based mitigation, readout mitigation, twirling, dynamical decoupling, symmetry verification and virtual distillation.
- **The machinery** (§12–§15): the test circuits, the compiler, the simulators and hardware execution.
- **Advanced mitigation** (§16): symmetry expansion, pulse-level ZNE and error amplification, quantum subspace expansion, classical shadows and M3 in depth, and how all methods relate.
- **Fault tolerance** (§17): the stabilizer formalism, a taxonomy of codes, decoding, the threshold theorem, and *logical* error mitigation — where the two roads meet.

Formulas use the same symbols as the app. References in square brackets, e.g. [Temme 2017], are listed with DOIs and arXiv numbers at the end.

1. Notation and conventions

Qubit order

Qubit 0 is the least significant bit, as in Qiskit: a computational basis state $|b_{n-1}\cdots b_1 b_0\rangle$ has index $x = \sum_i b_i 2^i$. A k-qubit gate written `cx 0 1` has control 0 and target 1, and its matrix is Qiskit's `Gate.to_matrix()` for that qubit list.

States

An n-qubit state is a density matrix $\rho \geq 0$, $\mathrm{Tr}\rho = 1$, of size $2^n \times 2^n$. A single qubit is fully described by its **Bloch vector** $\vec{r} = (x, y, z)$,

$$\rho = \tfrac{1}{2}\left(I + xX + yY + zZ\right), \qquad x = \mathrm{Tr}(\rho X),\ y = \mathrm{Tr}(\rho Y),\ z = \mathrm{Tr}(\rho Z).$$

Pure states have $|\vec{r}| = 1$; mixed states lie inside the ball. For one qubit of a larger register, $\vec{r}$ is computed from the reduced density matrix $\rho_q = \mathrm{Tr}_{\bar{q}}\,\rho$, so a qubit can have $|\vec{r}| < 1$ because it is **entangled** with the others even if the global state is pure. The time-travel tab therefore always shows the ideal vector next to the noisy one: if the ideal vector is short too, the qubit is entangled; if only the noisy one is short, noise mixed it.

Angles on the sphere

$\theta = \arccos(z/|\vec{r}|)$ is the polar angle from $|0\rangle$ and $\varphi = \mathrm{atan2}(y, x)$ the azimuth from $|+\rangle$ toward $|+i\rangle$. The **tilt** between the noisy and ideal vectors is $\arccos(\hat{r}_{\text{noisy}} \cdot \hat{r}_{\text{ideal}})$: a coherent error rotates the vector (tilt grows) without shortening it; an incoherent error shortens it.

Figures of merit

Purity $\mathrm{Tr}\rho^2 = \sum_{ij} |\rho_{ij}|^2$ (1 for pure states, 2^{-n} for the maximally mixed state). Fidelity with a pure ideal state $|\psi\rangle$: $F = \langle\psi|\rho|\psi\rangle$; in general the Uhlmann fidelity $F(\rho,\sigma) = \left(\mathrm{Tr}\sqrt{\sqrt{\sigma}\rho\sqrt{\sigma}}\right)^2$.

Observables

Estimators measure diagonal observables $O = \sum_x o(x)|x\rangle\langle x|$ in the measured basis; X or Y measurements are made by adding basis-change gates before measurement (H for X, $S^\dagger H$ for Y). The expectation value is $\langle O\rangle = \sum_x o(x)\, p(x)$ and one shot has variance $\sigma^2 = \sum_x o(x)^2 p(x) - \langle O\rangle^2$.

2. Noise

2.1 Quantum channels

A noise process is a completely positive, trace-preserving (CPTP) map written with Kraus operators, $\mathcal{N}(\rho) = \sum_k K_k \rho K_k^\dagger$ with $\sum_k K_k^\dagger K_k = I$. In the **Pauli transfer matrix** (PTM) representation, $R_{PQ} = \frac{1}{d}\mathrm{Tr}(P\,\mathcal{N}(Q))$ for Paulis P, Q and $d = 2^k$. A #### Pauli channel
$\mathcal{N}(\rho) = \sum_Q q_Q\, Q\rho Q$ has a diagonal PTM whose entries, the **Pauli fidelities**, are

$$f_P = \sum_Q q_Q\, \omega(P,Q), \qquad \omega(P,Q) = \begin{cases} +1 & PQ = QP \\ -1 & PQ = -QP \end{cases}, \qquad q_Q = \frac{1}{d^2}\sum_P f_P\, \omega(P,Q).$$

This pair of formulas (a Walsh–Hadamard transform) is used everywhere in the app: for PEC's inverses, for Pauli twirling and for the device model's twirled errors.

2.2 The injector's channels

Symbol	Channel	Definition
p_1, p_2	depolarizing after one-/two-qubit gates	$\rho \mapsto (1-p)\rho + p\, I/d$
γ	amplitude damping (T_1-like)	$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}$, $K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}$
λ	phase damping (T_ϕ-like)	coherences multiplied by $\sqrt{1-\lambda}$
κ	random-axis kick	a rotation by κ about a uniformly random axis; averaged, a depolarizing channel with $p = \frac{2}{3}(1-\cos\kappa)$
ε	over-rotation	every physical pulse (X, Y, √X, H, RX, RY) rotates by $(1+\varepsilon)\theta$ about its own axis; virtual Z gates are exact
θ_{ZZ}	ZZ phase	$e^{-i\theta_{ZZ} ZZ/2}$ after each two-qubit gate (residual coupling)
φ	Z phase	$R_Z(\varphi)$ on every qubit a gate touches
σ	quasi-static field	a random detuning $\delta_q \sim \mathcal{N}(0,\sigma^2)$, fixed during one shot, rotating an idle qubit by $R_Z(\delta_q)$ per idle layer

Symbol	Channel	Definition
λ_{idle}, γ_{idle}	Markovian idle noise	dephasing and damping per idle layer; a fraction t of a layer gets $1-(1-\lambda)^t$
ϵ_{01}, ϵ_{10}	readout	$P(1 \mid 0)$ and $P(0 \mid 1)$ for every bit

From datasheets

Device datasheets quote the average gate infidelity $r = 1 - \bar{F}$. For a depolarizing channel $\bar{F} = 1 - p\,(d-1)/d$ [Nielsen 2002], so

$$r = \frac{d-1}{d}\,p: \qquad r = \tfrac{p}{2}\ \text{(one qubit)}, \qquad r = \tfrac{3p}{4}\ \text{(two qubits)}.$$

The sidebar shows r next to the injector's p values; the device model converts back with $p = r\,d/(d-1)$.

T_1 and T_2

Over a time t amplitude damping has $\gamma = 1 - e^{-t/T_1}$ and coherences decay as e^{-t/T_2}, with $1/T_2 = 1/(2T_1) + 1/T_\phi$ and $T_2 \le 2T_1$.

2.3 Coherent versus incoherent errors

A stochastic (Pauli) error with probability p per gate lowers the fidelity by about p per gate: errors add **in probability**, $1 - F \approx Gp$ after G gates. A coherent rotation error ε adds **in amplitude**: G aligned errors rotate by $G\varepsilon$, so $1-F \approx (G\varepsilon)^2/4$ can grow quadratically, and it can also cancel (folding $UU^\dagger U$ undoes an over-rotation half of the time, which is why ZNE can misbehave under coherent noise). Pauli twirling (§8) converts coherent errors into stochastic ones.

The **quasi-static field** is simulated by Monte Carlo over the detuning: m samples of δ_q taken at the Latin-hypercube midpoints $\Phi^{-1}((k+\frac{1}{2})/m)\sigma$, independently permuted per qubit, and the resulting density matrices averaged. Low-frequency noise of this kind is exactly what dynamical decoupling refocuses.

2.4 Idle noise and scheduling

Circuits are laid out in **as-soon-as-possible layers**. A qubit that waits while others are busy is *idle* for that layer and receives the idle channels (dephasing, damping, the quasi-static rotation). Noise operations you place yourself take no time: they act where you put them. Barriers align their qubits and therefore shape the idle windows.

2.5 Readout errors

Readout of bit i is described by its **assignment matrix**

$$A_i = \begin{pmatrix} 1-\epsilon_{01} & \epsilon_{10} \\ \epsilon_{01} & 1-\epsilon_{10} \end{pmatrix}, \qquad \vec{p}_{\text{meas}} = \Big(\bigotimes_i A_i\Big)\vec{p}_{\text{true}},$$

with $\epsilon_{01} = P(\text{read } 1 \mid 0)$. Relaxation during readout makes $\epsilon_{10} > \epsilon_{01}$ on most devices.

2.6 Fake IBM devices

Choosing a fake backend (FakeFez, FakeTorino, FakeBrisbane, ...) replaces the injector with a calibration snapshot of a real processor [Javadi-Abhari 2024]:

- the circuit is **compiled for the device**: its coupling map, its native gates (CZ on Heron, ECR on Eagle) and its gate durations; couplers or qubits the calibration marks as broken (error ≥ 0.5) are excluded;
- every native gate is followed by the error Qiskit Aer derives from the calibration: a depolarizing error followed by thermal relaxation over the gate's duration, with the depolarizing strength chosen so that together they reproduce the reported gate error;
- the circuit is **scheduled as late as possible** with the real durations, and every idle window becomes thermal relaxation with that qubit's T_1 and T_2;
- readout uses each qubit's own $P(1|0)$ and $P(0|1)$;
- the density matrix is kept only for the physical qubits the circuit uses, so a 156-qubit device costs no more than an n-qubit simulation.

This model is **Markovian**: idle dephasing is exponential in time. Dynamical decoupling cannot refocus Markovian noise, so on a fake device DD shows only its cost (extra pulses). On real hardware DD helps because real qubits also suffer low-frequency noise and ZZ crosstalk; the injector's quasi-static field and ZZ phase let you study that case.

3. What error mitigation can and cannot do

3.1 Estimators, bias and variance

Error mitigation runs noisy circuits (often several) and combines their measured statistics into an **estimator** $\hat{O}$ of the ideal $\langle O \rangle$. Every estimator in the app reports

- an exact mean $\mathbb{E}[\hat{O}]$, hence the **bias** $b = \mathbb{E}[\hat{O}] - \langle O \rangle_{\text{ideal}}$, and
- a per-shot variance v, so that with N_t target shots the target circuits contribute v/N_t to the variance.

Equal budget. Every method is compared at the same **total** number of shots B per estimate (the sidebar's shot budget), split between three kinds of circuits:

$$B = N_t + F + c\,s,$$

with N_t shots on the *target* circuits (the folded copies of ZNE, the sampled circuits of PEC, the post-selected circuit of symmetry verification, ...), F *training* shots (CDR/NIL: training circuits × shots each) and c readout *calibration* circuits with s shots each. A finitely sampled calibration estimates flip probabilities $ heta_k$ with binomial error; to first order (delta method) it adds w/s with $w = \sum_k (\partial\hat{O}/\partial\theta_k)^2 \theta_k(1-\theta_k)$. The training circuits' own shot noise propagates through the fit to a constant u that more target shots cannot remove. The variance and the mean squared error at budget B are

$$\mathrm{Var}(B) = \frac{v}{N_t} + \frac{w}{s} + u, \qquad \mathrm{MSE}(B) = b^2 + \mathrm{Var}(B), \qquad \mathrm{RMSE}(B) = \sqrt{\mathrm{MSE}(B)}.$$

With $R = B - F$ the calibration shots that minimize the variance are $s^* = R\kappa/(1+c\kappa)$, $\kappa = \sqrt{w/(cv)}$ (at least one shot per calibration circuit); a method whose training cost exceeds B gives no estimate at that budget. Unmitigated estimation has a large bias and a small variance ($v = \sigma^2$). Mitigation reduces b and increases v: the **sampling overhead** γ is the factor by which the standard deviation grows, $v \approx \gamma^2\sigma^2$. Without training or calibration a mitigated method beats the unmitigated one once

$$N > N^* = \frac{v_m - v_r}{b_r^2 - b_m^2},$$

and reaching a target RMSE ε needs $N_\varepsilon = v/(\varepsilon^2 - b^2)$ shots, infinite when $\varepsilon \le |b|$. With training and calibration the app solves the same two questions numerically on RMSE(B): the crossover budget B^* and the budget for ε, which is infinite when $\varepsilon^2 \le b^2 + u$. These are the contents of the **Cost vs precision** page (implementation: `qemx/budget.py`).

3.2 The exponential cost is fundamental

For a circuit of G gates each with error rate p, the unmitigated signal decays roughly like $(1-p)^G$. Any method that removes the bias must amplify the remaining signal by the inverse factor, so its variance grows like e^{2cpG} for some constant c. This is not a limitation of one method: lower bounds show that, for broad classes of noise, every unbiased mitigation protocol needs a number of samples exponential in circuit depth or size [Takagi 2022; Quek 2024]. What differs between methods is the constant c, the assumptions they need, and how their bias behaves when the assumptions fail [Cai 2023].

3.3 QPU time

The app converts shots into QPU time with the sidebar's time per shot t_{shot} (repetition period including reset): $t = B\,t_{\text{shot}}$ — the same for every method, because target, training and calibration shots all come out of the same budget B.

Circuits. The number of distinct circuits a method sends is part of its cost too (compilation, job size): ZNE sends one circuit per noise scale, PEC one randomly sampled circuit per s_{PEC} shots (N_t/s_{PEC} circuits; the textbook protocol has $s_{\text{PEC}} = 1$, one shot per sampled circuit), twirling and randomized compiling one random circuit per shot, CDR/NIL one per training circuit, and readout mitigation two calibration circuits ($|0\cdots0\rangle$, $|1\cdots1\rangle$; one twirled $|0\cdots0\rangle$ for TREX). TREX sends every fixed circuit with 8 random measurement masks (X gates before measurement, undone classically), as on hardware.

4. Zero-noise extrapolation (ZNE)

4.1 Idea

If the noise can be **amplified** in a controlled way by a factor $\lambda \ge 1$, one can measure $E(\lambda) = \langle O\rangle_\lambda$ at several λ_j and extrapolate to the unreachable $\lambda = 0$ [Temme 2017; Li 2017]. ZNE needs no noise model — only the assumption that the measured points follow a known curve.

4.2 Noise amplification by folding

On gate-based hardware the noise is amplified digitally by **unitary folding** [Giurgica-Tiron 2020]:

- **global folding** $U \to U\,(U^\dagger U)^k$ runs every gate $\lambda = 2k+1$ times while the ideal circuit is unchanged. For a scale between odd integers the app folds a tail of the circuit (the number of gates closest to the request), so λ is realized in steps of $2/G$;
- **local folding** replaces each chosen gate $G \to G(G^\dagger G)^k$; the app folds two-qubit gates (all gates when there are none) and, for a fractional λ, a random subset once more.

The achieved λ is counted in two-qubit gates (all gates when there are none) and shown next to the requested one. Folding amplifies gate errors in proportion; it does **not** amplify errors that do not scale with gate count (state preparation, readout, idle noise is amplified only through the longer schedule), and coherent errors can partly cancel in $U^\dagger U$.

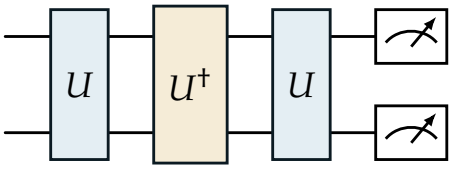


Figure 54: ZNE by global folding. global folding $U(U^\dagger U)^k$ multiplies the noise by $\lambda = 1 + 2k$ and leaves the ideal result unchanged.

4.3 Extrapolation rules

With measured points (λ_j, E_j), $j = 1..m$:

- **Richardson** — the degree-$(m-1)$ polynomial through all points evaluated at 0, i.e. Lagrange interpolation:

$$\hat{E}(0) = \sum_j \eta_j E_j, \qquad \eta_j = \prod_{l \neq j} \frac{\lambda_l}{\lambda_l - \lambda_j}, \qquad \sum_j \eta_j = 1.$$

- **linear** — the least-squares line; η is the first row of the pseudo-inverse of the Vandermonde matrix $[1, \lambda_j]$.
- **exponential** — $E(\lambda) = Ae^{-b\lambda}$, fitted as a line to $\log|E_j|$ (all E_j of one sign); natural for depolarizing-like decay $E(\lambda) \approx (1-p)^{\lambda G} E_{\text{ideal}}$. It fails (no finite value) when the E_j change sign; the app counts such failures in repeated experiments instead of averaging them.

4.4 Variance and optimal shot allocation

A linear estimator $\hat{E}(0) = \sum_j \eta_j \hat{E}_j$ with N_j shots on circuit j has $\text{Var} = \sum_j \eta_j^2 \sigma_j^2 / N_j$. Minimizing under $\sum_j N_j = N$ (Lagrange multiplier) gives

$$N_j \propto |\eta_j| \sigma_j, \qquad \text{Var}[\hat{E}(0)] = \frac{\left(\sum_j |\eta_j| \sigma_j\right)^2}{N},$$

so ZNE's overhead is $\gamma = \sum_j |\eta_j|$ (with all σ_j equal). Richardson with scales 1, 3, 5 has $\eta = (15/8, -10/8, 3/8)$ and $\gamma = 3.5$: small bias, large variance. The exponential rule is linearized around the measured values to obtain its η.

4.5 When ZNE fails

The bias is the error of the extrapolation model: a curved $E(\lambda)$ fitted linearly under-corrects, a high-order polynomial through noisy points over-shoots, and coherent errors make $E(\lambda)$ oscillate. IBM's large-scale experiments combine ZNE with learned noise models to amplify noise more faithfully (probabilistic error amplification) [Kim 2023a; Kim 2023b].

5. Probabilistic error cancellation (PEC)

5.1 Quasi-probability decomposition

Write each noisy gate as $\mathcal{O} = \mathcal{N} \circ \mathcal{U}$. If the noise $\mathcal{N}$ is known, its inverse can be written as a linear combination of implementable operations $\mathcal{B}_i$,

$$\mathcal{N}^{-1} = \sum_i q_i \mathcal{B}_i, \qquad \sum_i q_i = 1, \qquad \gamma = \sum_i |q_i| \geq 1,$$

where some $q_i < 0$ because $\mathcal{N}^{-1}$ is not a physical map [Temme 2017; Endo 2018]. The ideal expectation is $\langle O \rangle = \sum_i q_i \langle O \rangle_{\mathcal{B}_i \circ O}$. **Sampling**: pick $\mathcal{B}_i$ with probability $|q_i|/\gamma$, run it, and weight the outcome by $\gamma \operatorname{sgn}(q_i)$. The estimator is unbiased for the assumed model; per gate the standard deviation grows by γ_g, and for the circuit by

$$\Gamma = \prod_g \gamma_g, \qquad N_\varepsilon \approx \frac{\Gamma^2 \sigma^2}{\varepsilon^2}.$$

5.2 Depolarizing inverse

For $\mathcal{D}(\rho) = (1-p)\rho + p\, I/d$ every non-identity Pauli fidelity is $f = 1 - p$. The inverse has Pauli fidelities $1/f$; by the transform of §2.1 it is the Pauli quasi-mixture

$$q_I = \frac{1 + (d^2-1)/f}{d^2}, \qquad q_{P \neq I} = \frac{1 - 1/f}{d^2} < 0, \qquad \gamma = \frac{2(d^2-1)/f - (d^2-2)}{d^2},$$

so $\gamma = (3/f - 1)/2$ for one qubit and $\gamma = (15/f - 7)/8$ for two qubits, $\gamma \approx 1 + 2\frac{d^2-1}{d^2} p$ for small p. The **compensation** construction of [Endo 2018] instead writes $\mathcal{U} = \frac{1}{1-p} O - \frac{p}{1-p} \mathcal{T} \circ O$ with $\mathcal{T}$ the completely depolarizing map (a uniformly random Pauli); it is simpler to sample but costs $\gamma = (1+p)/(1-p)$, larger than the inverse's.

5.3 Learned models

Real noise is not depolarizing. Pauli twirling (§8) makes each gate's noise a Pauli channel, whose fidelities can be learned; IBM learns a **sparse Pauli–Lindblad** model per layer and inverts it [van den Berg 2023]. The app's *learned* PEC is the idealized version: in simulation it inverts the exact Pauli twirl of each gate's noise; on a fake device it uses a per-gate depolarizing model with that gate's calibrated error. Without twirling, coherent parts of the noise are not Pauli and remain uncorrected.

5.4 Model mismatch

If PEC is told p_{assumed} while the device has p_{true}, each corrected gate leaves a residual Pauli fidelity $f_{\text{true}}/f_{\text{assumed}}$ on every Pauli it propagates. Under-estimating the noise under-corrects, over-estimating it over-shoots the ideal value and costs more (γ grows with the assumed rate). Any noise absent from the model — damping, coherent rotations, readout — is left in place. The **Model mismatch** page maps these regimes.

5.5 How the app computes PEC

On the NumPy executor the correction maps are applied exactly as signed mixtures, giving the exact PEC mean; the variance is computed from the absolute-value mixture ($\mathbb{E}[X^2] = \Gamma \sum_x p_{\text{abs}}(x)\, o_{\text{eff}}(x)^2$, o_{eff} the observable with the readout mitigation folded in). Aer cannot apply negative weights, so it samples instances (random Pauli insertions with signs), reports their Monte Carlo mean and estimates $\mathbb{E}[X^2]$ as Γ^2 times the instances' average of $\sum_x p_i(x) o_{\text{eff}}(x)^2$. On hardware the app does the same with explicit Pauli gates.

These variances assume one shot per sampled circuit. With s shots per circuit (N_t/s circuits) the shots of one circuit share its sign and Pauli insertions, and the per-shot variance becomes $v_s = v_1 + (s-1)\,\text{Var}_{\text{inst}}$, where Var_{inst} is the variance over sampled circuits of $\Gamma \operatorname{sign}_i \langle o \rangle_i$ (a Monte Carlo estimate from the sampled instances). The circuit count in the cost table is $\lceil N_t/s \rceil$.

6. Learning-based mitigation: CDR and NIL

6.1 Clifford data regression

Circuits made only of Clifford gates can be simulated classically. **Clifford data regression** [Czarnik 2021] builds training circuits by replacing the target's non-Clifford rotations with nearby Clifford angles (the app picks $\theta' \in \{0, \frac{\pi}{2}, \pi, \frac{3\pi}{2}\}$ with probability $\propto e^{-(\Delta\theta/\sigma)^2}$, $\sigma = 1$, and optionally keeps a few rotations non-Clifford), runs them on the noisy device, computes their ideal values classically and fits

$$y_{\text{ideal}} \approx b + c\, x_{\text{noisy}}.$$

The fit is then applied to the target's noisy value. Training circuits share the target's structure, so they feel the same noise; the method fails when the noise depends on the angles that the training circuits changed, and when the training set does not span the target (the app reports the rank of the training features and warns when all training circuits have the same ideal value; §6.3 explains how the training set is chosen so that this does not happen).

6.2 Noisy-feature regression (NIL)

Learning-based mitigation can use more features than the noisy $\langle O\rangle$ [Strikis 2021; Lowe 2021]. The app's NIL uses $\mathbf{x} = (\langle O\rangle, \langle Z_i\rangle, \langle Z_iZ_j\rangle)$ from the same shots and fits $\hat{O} = b + \mathbf{c}\cdot\mathbf{x}$ by

- ordinary least squares $\min \sum_k (y_k - b - \mathbf{c}\cdot\mathbf{x}_k)^2$,
- Ridge, adding $\alpha\|\mathbf{c}\|_2^2$, or
- Lasso, adding $\alpha\|\mathbf{c}\|_1$, which keeps few features.

Because $\hat{O}$ is linear in the measured frequencies, its per-shot variance is that of $\sum_x g(x)$ with $g = F\mathbf{c}$; the overhead is roughly $\gamma = \|\mathbf{c}\|_1$. Training shots are a fixed part of the budget, and their shot noise makes the fit itself noisy: to first order its contribution is $u = \sum_i \mathbf{g}_i^\top \Sigma_i \mathbf{g}_i / N_{\text{train}}$, with $\mathbf{g}_i = \partial\hat{O}/\partial\mathbf{x}_i$ through the fit and Σ_i the per-shot covariance of training circuit i's features — a floor that more target shots cannot lower. For CDR this first-order value agrees with simulated repeats (within their ± 7 %); for NIL, whose many near-collinear features make the refit strongly non-linear, the repeats show a larger spread, and the app then uses the spread of the repeats and books the excess as u. The training set counts only feature directions whose spread exceeds the shot noise of the training circuits (singular values of the centered features above $\sigma(\sqrt{n_{\text{train}}} + \sqrt{n_{\text{features}}})$); with none, learning is not applied.

6.3 Designing the training set: why CDR can collapse, and the adaptive strategy

For training pairs (x_k, y_k) (x noisy, y ideal) the least-squares slope and intercept are

$$\hat{c} = \frac{\sum_k (x_k - \bar{x})(y_k - \bar{y})}{\sum_k (x_k - \bar{x})^2}, \qquad \hat{b} = \bar{y} - \hat{c}\,\bar{x}.$$

If every training circuit has the **same ideal value**, $y_k \equiv y_0$, the covariance vanishes, $\hat{c} = 0$ and $\hat{O} = \hat{b} = y_0$ for *every* target: the "mitigated" result is a constant that carries no information about the target (often exactly 0.0000 or 1.0000), and a shot-noise simulation of it has zero variance. This is not overfitting in the usual sense but an **unidentifiable** regression. It happens when the observable is blind to the Clifford replacements — e.g. a single $\langle Z_q\rangle$ at the end of phase estimation, where every near-Clifford training circuit returns the same basis state. With several features (NIL) the analogous failure is a rank-deficient design matrix: directions the training set never explores are extrapolated blindly.

The app therefore chooses the training set **before** any noisy circuit is run, using only classical (ideal) simulations, and escalates until the spread of the training values reaches a threshold, $\mathrm{sd}(y) \geq s_{\min}\,(\max_x o(x) - \min_x o(x))$ (default $s_{\min} = 0.05$):

1. k non-Clifford gates kept (the user's value, default 0), near-Clifford angles ($\sigma = 1$);
2. the same k with Clifford angles drawn uniformly (more diverse circuits);
3. $k+1$, $k+2$, ... kept non-Clifford gates, each with both samplings (training circuits closer to the target, at a classical cost that grows exponentially with k for stabilizer simulators [Czarnik 2021]);
4. every rotation redrawn as a random Clifford angle;
5. the same with a random computational basis state as input (X gates on a random subset of qubits).

The first stage that passes is used and the stages tried are reported. Because the choice depends only on the circuit, the observable and the seed, a hardware run records and replays exactly the same training circuits. If no stage gives varying training values — for instance $\langle Z_i \rangle = 0$ for every QAOA circuit, by the $X^{\otimes n}$ symmetry of $|+\rangle^{\otimes n}$ and of both QAOA unitaries — the regression is **not applied**: the unmitigated value is returned, the table's *status* column says so, and a warning replaces the regression plot. The *fixed* strategy reproduces plain CDR with exactly the chosen k and $\sigma = 1$, for comparison.

7. Readout mitigation

7.1 Tensored inverse

Calibrate each bit with two circuits (all qubits in $|0\rangle$; all in $|1\rangle$) to obtain A_i (§2.5), then apply $\bigotimes_i A_i^{-1}$ to the measured distribution [Bravyi 2021]. The result is unbiased for uncorrelated readout errors but is a quasi-distribution (it can have negative entries) and amplifies the variance by $\prod_i \frac{1+|\epsilon_{01}-\epsilon_{10}|}{1-\epsilon_{01}-\epsilon_{10}}$.

7.2 M3

The full inverse is exponentially large. **Matrix-free measurement mitigation** (M3) solves the assignment equations only on the bitstrings actually observed (optionally within a Hamming distance), with the reduced matrix's columns renormalized, so the cost scales with the number of distinct outcomes rather than 2^n [Nation 2021].

7.3 TREX

Twirled readout error extinction randomly flips each qubit with an X gate before measurement and flips the recorded bit back [van den Berg 2022]. Averaged over the flips, the assignment matrix becomes symmetric with flip probability $\bar{p}_i = \frac{1}{2}(\epsilon_{01} + \epsilon_{10})$, so every Z-string expectation is scaled by a known factor:

$$\langle Z_S \rangle_{\text{meas}} = \prod_{i \in S} (1 - 2\bar{p}_i)\, \langle Z_S \rangle,$$

which a single calibration circuit ($|0 \cdots 0\rangle$ with the same twirl) measures. Dividing removes readout bias without modeling asymmetric or correlated structure.

8. Pauli twirling and randomized compiling

Surrounding a Clifford gate G with random Paulis P before and $P' = GPG^\dagger$ after leaves the ideal gate unchanged. Averaged over P, the gate's noise channel becomes its **Pauli twirl**: only the diagonal

of its PTM survives [Knill 2005; Wallman 2016]. Coherent errors become stochastic Pauli errors, which accumulate in probability rather than amplitude and are exactly what Pauli-model PEC and learned models assume. **Randomized compiling** twirls all gate layers and merges the Paulis into neighboring one-qubit gates [Hashim 2021]. The app applies the twirl exactly (the average over all twirl instances) by replacing each Clifford gate's noise with its PTM diagonal; the Circuits tab shows one random instance.

9. Dynamical decoupling

An idle qubit accumulates phase from slow frequency fluctuations. Inserting π pulses reverses the sign of that phase halfway, so a static detuning cancels [Viola 1998]. The app places

- **X2** (CPMG-like): X at $T/4$ and $3T/4$ of an idle window of length T;
- **XY4**: X, Y, X, Y at $T/8, 3T/8, 5T/8, 7T/8$, which also compensates pulse errors to first order.

In the toggling frame the pulses multiply the noise by a switching function $y(t) = \pm 1$; the accumulated phase $\int \delta(t) y(t)\, dt$ vanishes for constant δ. DD filters low-frequency noise; it cannot remove Markovian (white) noise and adds the pulses' own errors [Ezzell 2023]. The Time-travel tab shows a static field being refocused.

10. Symmetry verification

If the ideal state is an eigenstate of a symmetry — here a Z-parity Z_S with eigenvalue $s = \pm 1$ — shots violating it must be errors and are discarded [Bonet-Monroig 2018; McArdle 2019]:

$$\hat{O} = \frac{\sum_{x:\,\mathrm{par}_S(x)=s} o(x)\,\hat{p}(x)}{\sum_{x:\,\mathrm{par}_S(x)=s} \hat{p}(x)}.$$

The cost is the discarded fraction: the effective shots are NP_{accept}, so $\gamma = P_{\mathrm{accept}}^{-1/2}$. Errors that preserve the symmetry are not detected. The app warns when the ideal state is not an eigenstate of the chosen parity (post-selection then biases the answer). In HHL the same machinery post-selects the ancilla.

11. Virtual distillation

Given M copies of $\rho = \sum_k \lambda_k |\psi_k\rangle\langle\psi_k|$, an entangling **derangement** (cyclic shift) circuit measures $\mathrm{Tr}(\rho^M O)$ and $\mathrm{Tr}(\rho^M)$ [Huggins 2021; Koczor 2021]:

$$\langle O\rangle_M = \frac{\mathrm{Tr}(\rho^M O)}{\mathrm{Tr}(\rho^M)} \xrightarrow[M\to\infty]{} \langle\psi_1|O|\psi_1\rangle.$$

Incoherent errors are suppressed exponentially in M; a coherent error that rotates the dominant eigenvector $|\psi_1\rangle$ is not (the "coherent mismatch" floor). The cost is M times the qubits and a variance amplified by about $\mathrm{Tr}(\rho^M)^{-2}$. The app evaluates the formula from the exact density matrix with ideal copies and derangement and shows the spectrum of ρ.

12. Test circuits

The templates are the benchmarks every method is tested on: they range from two-qubit states to circuits with hundreds of two-qubit gates, and every one has a known ideal answer.

12.1 Entanglement and phase kickback

Bell and GHZ states

$|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ and its n-qubit version; the parity $Z_0 Z_{n-1} = +1$ is a symmetry.

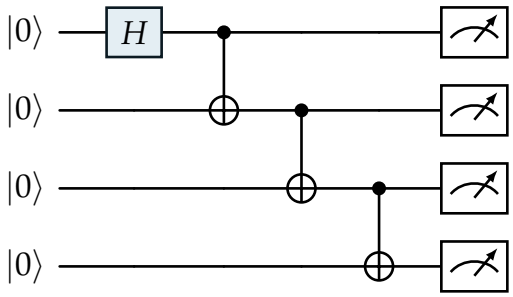


Figure 55: GHZ state.

Bernstein–Vazirani

One query of $f(x) = s \cdot x \bmod 2$, with the ancilla in $|-\rangle$ (phase kickback), gives $H^{\otimes n} U_f H^{\otimes n} |0\rangle^{\otimes n} = |s\rangle$ [Bernstein 1997]; the observable is $P(s)$.

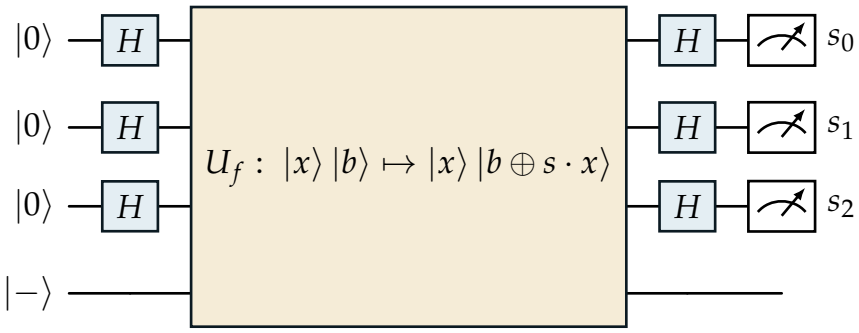


Figure 56: Bernstein–Vazirani.

12.2 Fourier-based algorithms

Quantum Fourier transform

$F|j\rangle = 2^{-n/2} \sum_k e^{2\pi i jk/2^n} |k\rangle$. The textbook circuit is Hadamards and controlled phases $\mathrm{CP}(\pi/2^d)$ followed by a SWAP network that reverses the bit order [Nielsen 2010]. The template runs $F\,F^\dagger|s\rangle = |s\rangle$ (the input $F^\dagger|s\rangle$ is a product state) in three ways: with the SWAP network; without SWAPs, loading the Fourier state of the bit-reversed s; or without SWAPs, relabeling the output bits classically. The last two save $3\lfloor n/2 \rfloor$ CX gates.

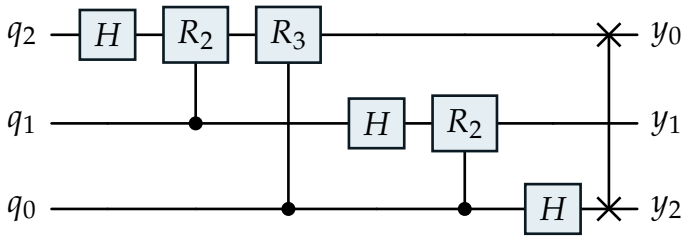


Figure 57: Quantum Fourier transform. $R_k = \mathrm{diag}(1, e^{2\pi i/2^k})$. The approximate QFT of degree m keeps only the R_k with $k \le m+1$.

Approximate QFT

Controlled phases $\pi/2^d$ with $d > m$ are dropped [Coppersmith 1994]; the error is $O(n2^{-m})$ in operator norm while the gate count falls from $O(n^2)$ to $O(nm)$, and under noise the shorter circuit can be more accurate than the exact one [Barenco 1996]. Here m is the largest qubit distance d that is kept; in terms of $R_k = \mathrm{CP}(2\pi/2^k)$ the circuit keeps $k \le m+1$ (conventions for the "degree" of the AQFT differ by one between papers). The approximation error depends on the input state, so the bit-reversed-input variant (which feeds the truncated circuit the Fourier state of $\mathrm{rev}(s)$) has a different success probability from the SWAP and relabel variants.

Phase estimation

t counting qubits, controlled U^{2^k}, inverse QFT [Kitaev 1995]; with $U = P(2\pi\varphi)$ and $\varphi = j/2^t$ the output is exactly j. **QPE of an Ising Hamiltonian** uses $U = e^{-iJZ_aZ_b\tau}$ on the eigenstate $|ab\rangle$; controlled-U^{2^k} is $\mathrm{CX} \cdot \mathrm{CRZ}(2J\tau 2^k) \cdot \mathrm{CX}$.

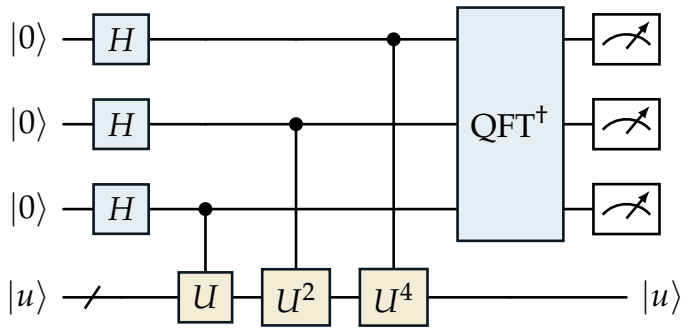


Figure 58: Phase estimation.

Trotterized Ising dynamics

$H = -J\sum_i Z_iZ_{i+1} - h\sum_i X_i$, first-order Trotter steps $\prod_i R_{ZZ}(-2J\Delta t)\prod_i R_X(-2h\Delta t)$; the observable is the magnetization $\langle\sum_i Z_i\rangle/n$.

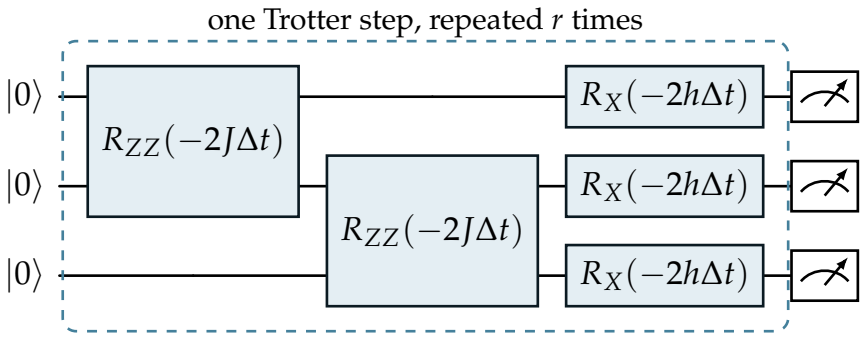


Figure 59: Trotterized Ising dynamics.

12.3 NISQ: variational circuits

VQE with a hardware-efficient ansatz

The variational quantum eigensolver minimizes $E(\theta) = \langle\psi(\theta)|H|\psi(\theta)\rangle$ over a parametrized state [Peruzzo 2014]. The hardware-efficient ansatz uses only native operations [Kandala 2017]: $|\psi\rangle = \prod_{\ell=1}^{L}[\bigotimes_q R_Z(\phi_{\ell q})R_Y(\theta_{\ell q})\,U_{\text{ent}}]\bigotimes_q R_Z(\phi_{0q})R_Y(\theta_{0q})|0\rangle^{\otimes n}$ with a linear, circular or all-to-all CX entangler U_{ent} ($2n(L+1)$ angles). The template's Hamiltonian is the antiferromagnetic chain $H = \sum_i Z_i Z_{i+1}$, whose ground energy is $-(n-1)$ (Néel states); the observable is $\langle H\rangle/(n-1) \in [-1,1]$. With *VQE-optimized* angles the app minimizes E on a noiseless statevector with L-BFGS-B when the template loads, so the ideal value is the optimum (-1) and every deviation in the comparison is noise — the setting in which Kandala et al. applied ZNE to VQE [Kandala 2019]. *Random* angles give a generic HEA state instead.

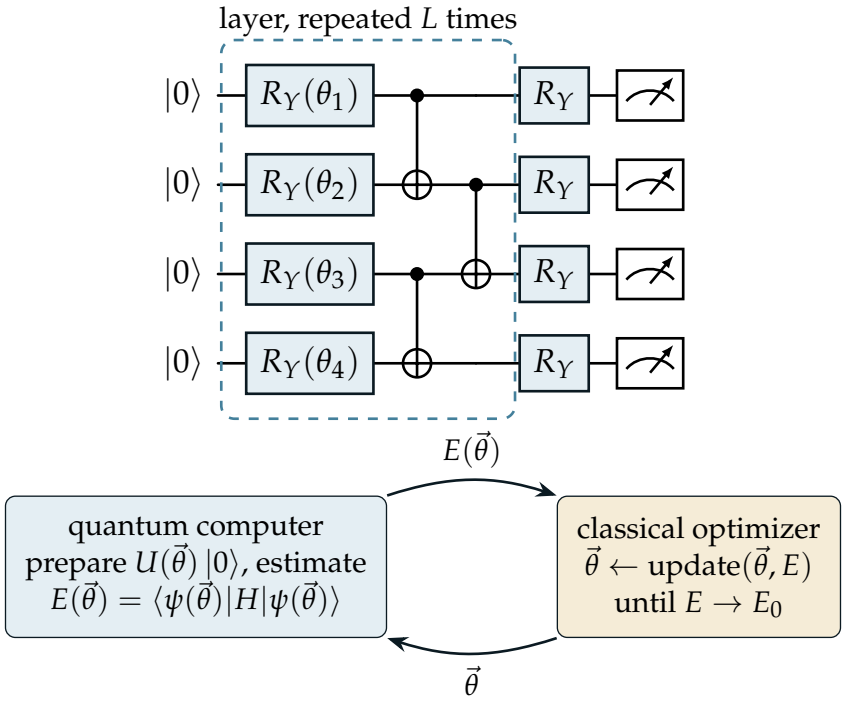


Figure 60: VQE with a hardware-efficient ansatz.

QAOA for MaxCut

For a graph $G = (V, E)$ the cut size is $C(x) = \sum_{(i,j)\in E}[x_i \neq x_j] = \sum_{(i,j)} \frac{1}{2}(1 - Z_iZ_j)$. QAOA prepares $|\gamma, \beta\rangle = \prod_{\ell=1}^{p} e^{-i\beta_\ell B} e^{-i\gamma_\ell C}|+\rangle^{\otimes n}$ with the mixer $B = \sum_i X_i$ [Farhi 2014]. Up to a global phase $e^{-i\gamma C} = \prod_{(i,j)} R_{ZZ}(-\gamma)$ and $e^{-i\beta B} = \prod_i R_X(2\beta)$, which is how the template is built. The observable is the approximation ratio $\langle C\rangle / C_{\max}$. Graphs: ring, path, complete, the 3-regular Möbius ladder (ring plus diameters, n even) and a seeded random $G(n, \frac{1}{2})$. For a ring with $n \geq 4$ at $p = 1$ the optimum is $\langle C\rangle/|E| = 3/4$ [Farhi 2014], which equals the ratio $\langle C\rangle / C_{\max}$ for even n (an odd ring has $C_{\max} = |E| - 1$: 0.9375 for $n = 5$); the optimizer reproduces these values.

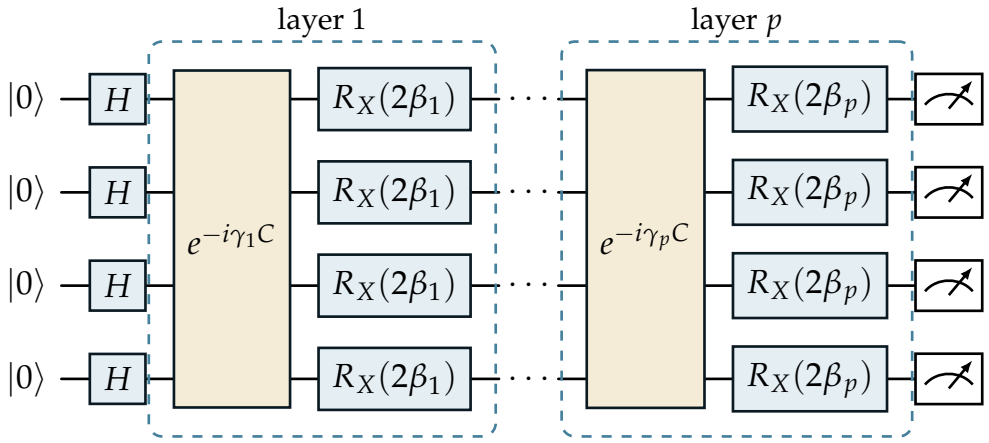


Figure 61: QAOA for MaxCut. $C = \sum_{(i,j)\in E} \frac{1}{2}(1 - Z_iZ_j)$: $e^{-i\gamma C}$ is one $R_{ZZ}(-\gamma)$ per edge (up to a global phase).

Quantum machine learning

The ZZ *feature map* encodes $x \in \mathbb{R}^n$ as $U_\Phi(x) = [e^{i\sum_i \phi_{i,i+1}(x) Z_iZ_{i+1}} e^{i\sum_i \phi_i(x) Z_i} H^{\otimes n}]^r$ with $\phi_i = x_i$, $\phi_{ij} = (\pi - x_i)(\pi - x_j)$ [Havlíček 2019]. The template follows Qiskit's `ZZFeatureMap` with `entanglement="linear"` gate by gate — $P(2\phi_i)$ on each qubit and $\mathrm{CX}\,P(2\phi_{ij})\,\mathrm{CX}$ for neighbors, which implements the exponentials above with the opposite sign, $e^{-i\phi Z}$ and $e^{-i\phi ZZ}$, up to a global phase (the template's formula shows that sign); a RealAmplitudes-type layer $W(\theta)$ follows and the classifier output is $\langle Z^{\otimes n}\rangle$. The *data re-uploading* classifier interleaves data and weights in every layer, $R_Z(w'_{\ell q}x_q + b'_{\ell q})R_Y(w_{\ell q}x_q + b_{\ell q})$ followed by a CZ chain, and reads $\langle Z_0\rangle$ [Pérez-Salinas 2020]. Data and weights come from seeds: for error mitigation the circuit structure — depth, entangling pattern, non-Clifford content — is what matters, not whether the model is trained.

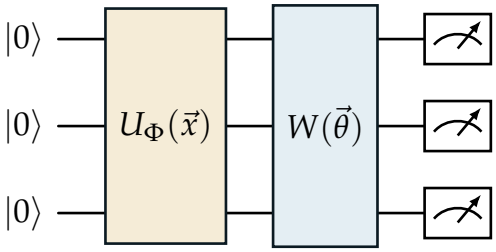


Figure 62: Quantum machine learning. feature map $U_\Phi(\vec{x})$ ($\times r$), trainable ansatz $W(\vec{\theta})$; the model is $f(\vec{x}) = \langle Z \cdots Z\rangle$.

12.4 Fault tolerance: error-correction primitives

Three-qubit bit- and phase-flip codes

[Shor 1995; Nielsen 2010, §10.1]. $|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle$ is encoded as $\cos\frac{\theta}{2}|000\rangle + \sin\frac{\theta}{2}|111\rangle$ by two CX gates (the phase-flip code adds $H^{\otimes 3}$, so Z errors act as X errors in that basis). Two ancillas measure the stabilizers Z_0Z_1 and Z_1Z_2; the syndrome $(s_1, s_2) = (1,0), (1,1), (0,1)$ points to a flip of q_0, q_1, q_2. In *syndrome + correction* mode three Toffoli gates apply that correction coherently (no mid-circuit measurement), the code is decoded and $\langle Z_0\rangle = \cos\theta$ is restored after any single error. *Syndrome only* extracts but ignores the syndrome; *majority vote readout* measures the three data qubits and decodes classically. An error can be placed on one qubit or as a bit-/phase-flip channel of probability p on every data qubit; the logical error rate is then $p_L = 3p^2 - 2p^3$ instead of p.

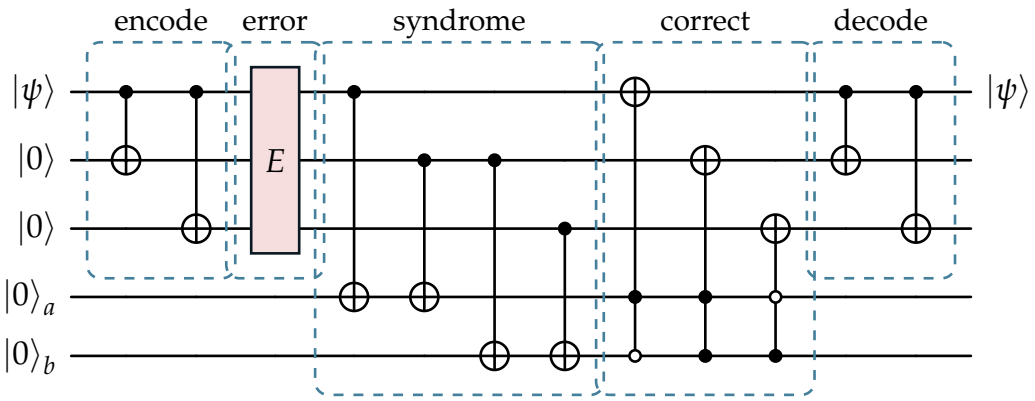


Figure 63: Three-qubit bit-flip code.

Repetition-code memory

A distance-d bit-flip code with r rounds of Z_iZ_{i+1} checks into fresh ancillas and a bit-flip channel with probability p before every round and once more before readout ($r+1$ exposures, so each data qubit ends flipped with $p_{\text{eff}} = \frac{1}{2}[1 - (1-2p)^{r+1}]$); the final data bits are decoded by majority vote, which fails with probability $\sum_{j>d/2}\binom{d}{j}p_{\text{eff}}^j(1-p_{\text{eff}})^{d-j}$ — exponentially small in d below the threshold $p_{\text{eff}} < 1/2$. The circuit noise of the chosen model (the CX gates of the checks) competes with this suppression, which is the essence of the fault-tolerance threshold.

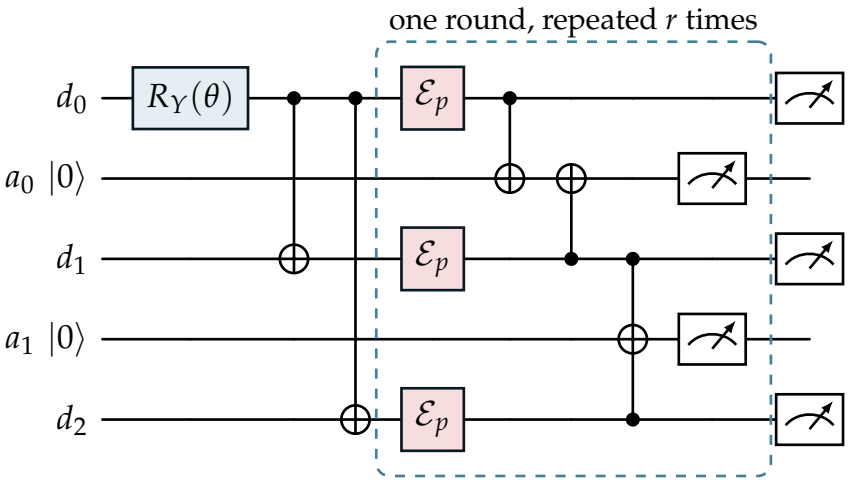


Figure 64: Repetition-code memory. ancillas measure $Z_{d_0}Z_{d_1}$ and $Z_{d_1}Z_{d_2}$ each round; the logical bit is the majority of $d_0 d_1 d_2$.

Steane [[7,1,3]] code

[Steane 1996]. The stabilizers are X and Z on the supports of the rows of the Hamming check matrix, $\{3,4,5,6\}, \{1,2,5,6\}, \{0,2,4,6\}$ (0-indexed qubits; column q is the binary number $q+1$). $|0_L\rangle = 2^{-3/2} \sum_{c \in C^\perp} |c\rangle$ is the uniform superposition of the 8 codewords of the dual (simplex) code. The encoder places $|\psi\rangle$ on q_2, applies the logical-X representative $X_2X_4X_5$ controlled by it, then Hadamards on the pivots q_0, q_1, q_3 and CX gates from each pivot to the rest of its row. The logical Z is read transversally as the parity of all 7 bits; the *decoded* readout first computes the Hamming syndrome of the measured bits, flips the indicated bit, and then takes the parity — so any single X error leaves $\langle Z_L \rangle = \cos\theta$ unchanged, whereas the raw parity flips sign. The stabilizer $Z_3Z_4Z_5Z_6 = +1$ is offered as the symmetry for verification (post-selection on a stabilizer is error *detection*).

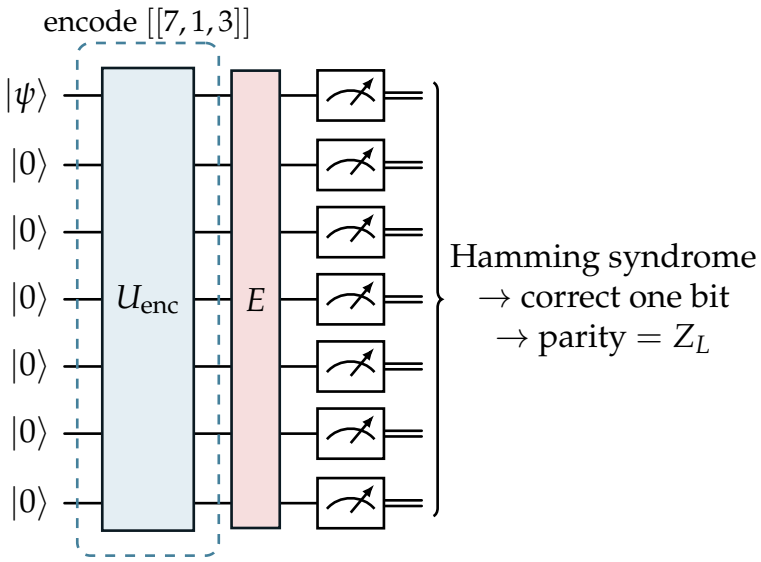


Figure 65: Steane [[7,1,3]] code.

Five-qubit code [[5,1,3]]

[Laflamme 1996; Bennett 1996]. Stabilizers $XZZXI$ and its cyclic shifts; $Z_L = Z^{\otimes 5}$, $X_L = X^{\otimes 5}$. The encoder is a Clifford circuit U (11 CX) synthesized from the tableau $UZ_0U^\dagger = Z_L$, $UX_0U^\dagger = X_L$, $UZ_jU^\dagger = S_j$. After an error E, the inverse encoder gives $U^\dagger EU = P_0 \otimes Q$: a logical Pauli P_0 on q_0 and, on $q_1 \dots q_4$ (prepared in $|0\rangle$), bit flips exactly where E anticommutes with S_j — the syndrome. Measuring all five qubits therefore yields the syndrome and the logical bit; the *decoded* observable flips the logical bit when the syndrome's error acts as X or Y on it (a 16-entry lookup table: the code is perfect, $2^4 = 1 + 3 \cdot 5$). Every single-qubit X, Y or Z error leaves $\langle Z_L \rangle = \cos\theta$.

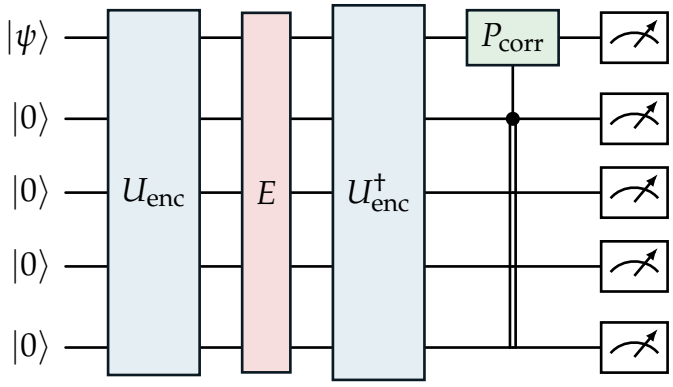


Figure 66: Five-qubit [[5,1,3]] code. $U_{\text{enc}}^\dagger E U_{\text{enc}}$ = (logical Pauli on q_0) ⊗ (syndrome s on q_1–q_4); a 16-entry lookup gives P_{corr}; q_0 reads Z_L.

Surface code, one plaquette ([[4,1,2]])

Data q_0q_1/q_2q_3 on a 2×2 patch; stabilizers $X_0X_1X_2X_3$ (the plaquette) and the boundary checks Z_0Z_1, Z_2Z_3; $Z_L = Z_0Z_2$, $X_L = X_0X_1$; $|0_L\rangle = (|0000\rangle + |1111\rangle)/\sqrt{2}$, prepared by CX, H and CX gates from $RY(\theta)|0\rangle$ on q_0. With distance 2 any single-qubit error is detected but none is correctable. *Transversal* readout computes the Z checks from the data bits; *ancilla extraction* measures the plaquette on q_4 (it flags Z errors) and the Z checks on q_5, q_6. Symmetry verification post-selects on both Z checks, which removes every single X or Y error — the errors that can flip Z_L — while weight-2 errors such as $X_0X_1 = X_L$ remain undetectable, as distance 2 implies.

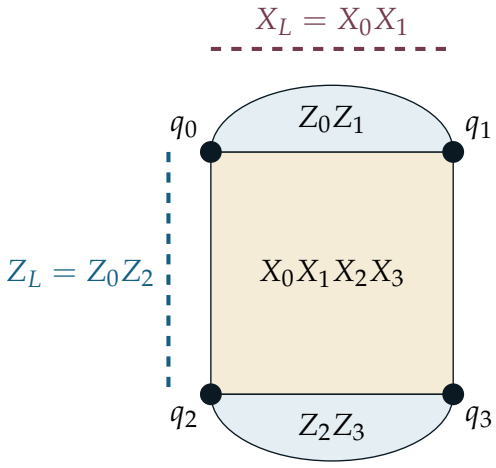


Figure 67: Surface code d = 2. $[[4,1,2]]$: detects any single error, corrects none; ancillas q_4, q_5, q_6 measure the three checks.

12.5 Fault tolerance: deep algorithms

Shor's order finding for N = 15

With t counting qubits and a 4-qubit work register in $|1\rangle$, controlled multiplications by $a^{2^k} \bmod 15$ and an inverse QFT give outcomes $y \approx s2^t/r$ from which continued fractions recover the order r of a [Shor 1997]. The multiplications are "compiled" (the powers are computed classically), as in the first experimental demonstrations [Vandersypen 2001]. The observable is the probability of an outcome that yields r.

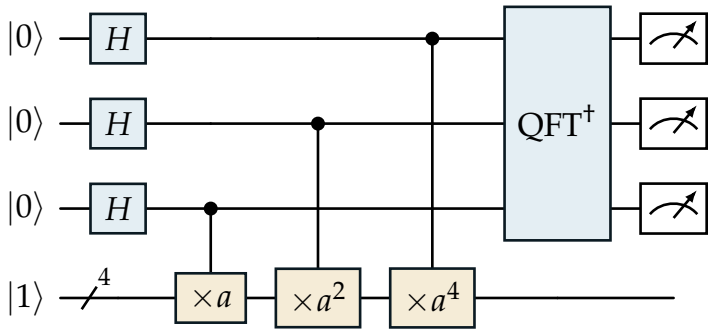


Figure 68: Shor's order finding. multiplications $\bmod 15$; the counting register reads $y \approx 2^t s/r$, and continued fractions give the order r.

Deep variants. Besides the compiled circuit, controlled-U^{2^k} can be implemented as 2^k repeated controlled multiplications by a ($2^t - 1$ in total, as a circuit that does not know the order would do), and each controlled multiplication can be the textbook SWAP/NOT network or a generic unitary synthesis of the controlled 16×16 permutation $|y\rangle \mapsto |ay \bmod 15\rangle$ (hundreds of CX each). The ideal output distribution is the same; the depth grows by one to two orders of magnitude, which is the regime where error *mitigation* stops working and error *correction* becomes necessary [Takagi 2022; Quek 2024].

Grover's search

Grover's algorithm finds one marked item m among $N = 2^n$ with $O(\sqrt{N})$ oracle calls, quadratically fewer than any classical search and optimal for unstructured search [Grover 1996; Bennett 1997b]. In the plane spanned by $|m\rangle$ and the uniform superposition $|s\rangle = \sin\theta|m\rangle + \cos\theta|m^\perp\rangle$, $\sin\theta = 1/\sqrt{N}$, the oracle $O_m = I - 2|m\rangle\langle m|$ and the diffuser $D = 2|s\rangle\langle s| - I$ are reflections whose product rotates by 2θ:

$$P(m) = \sin^2\left((2k+1)\theta\right), \qquad k^* = \lfloor \pi/(4\theta) \rfloor.$$

The template builds O_m from X gates around a multi-controlled Z and $D = H^{\otimes n} X^{\otimes n} C^{n-1}Z\, X^{\otimes n} H^{\otimes n}$ (up to a global phase); the $(n-1)$-controlled Z is a CZ for $n = 2$, the 6-CX Clifford+T circuit for $n = 3$ and Qiskit's synthesis beyond. Iterating past k^* rotates beyond the target [Boyer 1998]. The observable is $P(m)$. The two-qubit *Grover* template (Basics) is the case $N = 4$, $k^* = 1$, which finds the marked item $|11\rangle$ with certainty.

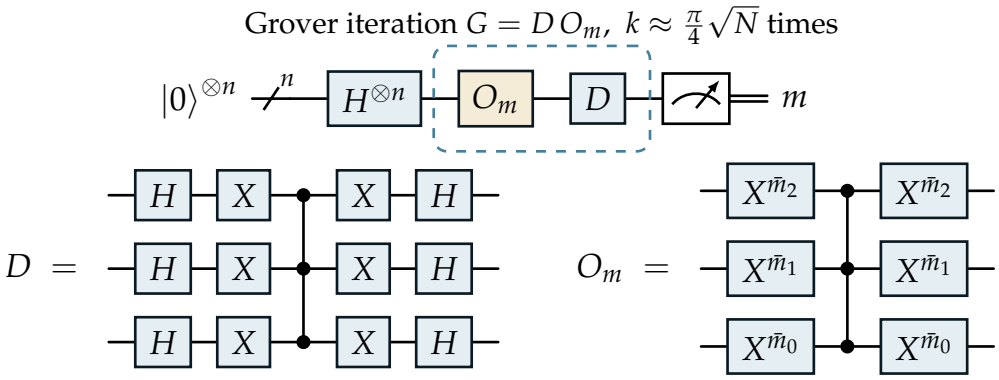


Figure 69: Grover's search. $P(m) = \sin^2((2k+1)\theta)$, $\sin\theta = 1/\sqrt{N}$; the controlled-Z fires only on $|1\cdots1\rangle$.

The hidden subgroup problem

A function f on a group G hides a subgroup H if it is constant on the cosets of H and distinct on different cosets. For abelian G the same Fourier-sampling circuit solves every instance: prepare $\sum_x |x\rangle|f(x)\rangle$, apply the quantum Fourier transform of G to the input register, and measure; the outcome is a uniformly random character trivial on H, i.e. an element of the annihilator $H^\perp$, and $O(\log|G|)$ samples determine H [Jozsa 2001; Childs 2010]. *Simon's algorithm* (the instance $G = \mathbb{Z}_2^n$, $H = \{0, s\}$; its own template under Algorithms) finds s with $f(x) = f(x \oplus s)$ [Simon 1997]. Every measured y satisfies $y \cdot s = 0$, i.e. the parity of the input bits selected by s is $+1$ — an exact symmetry. The template *Hidden subgroup problem* generalizes it in two directions: $G = \mathbb{Z}_2^n$ with a hidden subgroup of any dimension, $H = \mathrm{span}\{h_1, h_2\}$, whose oracle $f(x) = Ax$ uses a basis of $H^\perp$ as the rows of A (so $\ker f = H$), and the cyclic group $G = \mathbb{Z}_N$, $N = 2^n$, with $H = \langle 2^j\rangle$, $f(x) = x \bmod 2^j$ and the QFT over $\mathbb{Z}_N$ — period finding, the subroutine behind Shor's algorithm. The observables are $P(y \in H^\perp)$ and $P(y \equiv 0 \bmod N/2^j)$, both ideally 1; for $\mathbb{Z}_2^n$ every generator of H is a Z-parity that symmetry verification can check.

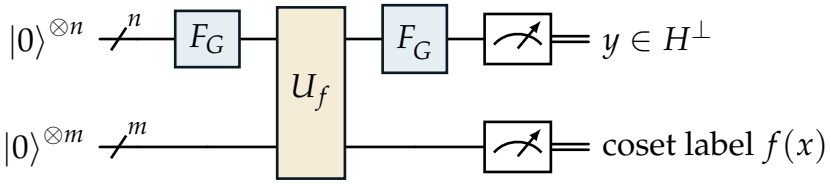


Figure 70: Hidden subgroup problem. $G = \mathbb{Z}_2^n$: $F_G = H^{\otimes n}$, $f(x) = Ax$ with $\ker A = H$. · $G = \mathbb{Z}_N$: $F_G = \mathrm{QFT}_N$, $f(x) = x \bmod d$, $y \in \{0, N/d, 2N/d, \dots\}$.

HHL

For $A = \begin{pmatrix} 1 & -1/3 \\ -1/3 & 1 \end{pmatrix}$ and $|b\rangle = |0\rangle$, phase estimation with e^{iAt_0}, $t_0 = 3\pi/4$, writes the eigenvalues $2/3, 4/3$ as 1 and 2 on two clock qubits; controlled R_Y rotations put C/λ into an ancilla amplitude; uncomputation returns the clock [Harrow 2009; Cao 2012]. Given ancilla $= 1$ the system qubit holds $|x\rangle \propto A^{-1}|b\rangle = (3|0\rangle + |1\rangle)/\sqrt{10}$, so $\langle Z\rangle = 0.8$.

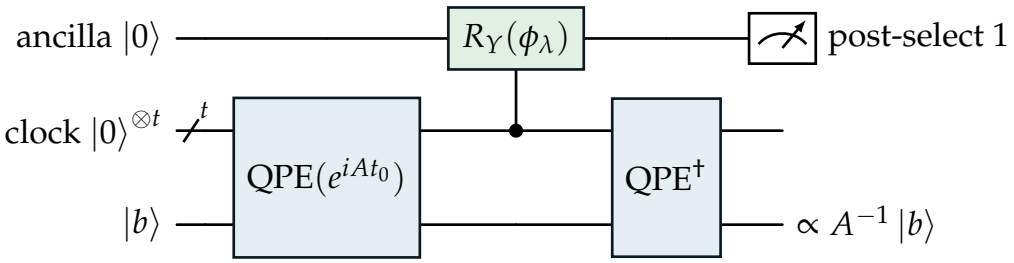


Figure 71: HHL.

13. Compilation

The **logical** circuit is what you draw plus input-state preparation and measurement-basis changes. The **implemented** circuit is what a device runs:

- **abstract** mode keeps the circuit as drawn;
- **ISA** mode transpiles with Qiskit to $\{R_Z, \sqrt{X}, X\}$ plus CX or CZ on a chosen coupling map (all-to-all, line, ring), with SABRE layout and routing [Li 2019] or a trivial layout, and optimization levels 0–3;
- **device** mode (a fake backend) transpiles for that processor's target (§2.6).

Routing inserts SWAPs (3 two-qubit gates each) and permutes which qubit ends up measured into which classical bit; the app tracks this **measurement map** so that observables refer to the right bits. R_Z is virtual (a frame change) and error-free; $\sqrt{X}$ and X are physical pulses.

14. The simulators

One model, two executors

`resolve()` turns a circuit and a noise specification into one instruction list (unitaries, Kraus maps, and mixtures that may carry negative weights). The NumPy executor applies it to a density matrix held as a tensor with $2n$ axes; Qiskit Aer runs the same list in `density_matrix` mode. They agree to machine precision. Both return exact outcome distributions, from which the app computes exact means and variances; repeated experiments are simulated by multinomial sampling.

Time travel

stores the state after every step. When $2 \times (\text{steps} + 1)$ density matrices would take a large share of memory, it switches to a compact mode that keeps per-step summaries (Bloch vectors, fidelity with the ideal state vector, purity).

Resources

A density matrix of n qubits takes $16 \cdot 4^n$ bytes (16 MB at $n = 10$, 4.3 GB at $n = 14$). The app has no limit on qubits or depth; it estimates memory before heavy runs, catches out-of-memory errors, and stops cleanly at a time limit you set (0 means none).

15. Running on hardware

The IBM Quantum page runs the same estimators on a real device — or rehearses locally with the Sampler on a fake backend — by *record* → *allocate* → *execute* → *replay*:

1. **Record.** Each estimator runs once against a recorder that notes every circuit it asks for, with its role (target, training or calibration), and answers with the circuit simulated under the device's calibrated median error rates.
2. **Allocate.** The recorded result gives each contender's cost model, and its budget B is split exactly as in simulation: s^* shots per calibration circuit, the method's training shots, and the target shots as the estimator's variance assumes — ZNE's $N_j \propto |\eta_j|\sigma_j$, a Pauli sum's $N_g \propto \sqrt{v_g}$, PEC's K sampled circuits with N_t/K shots each. Contenders never share counts: each is measured on its own budget.
3. **Execute.** The circuits are compiled for the backend's physical qubits and sent with the Sampler primitive, one job per option group and shot count (gate twirling and dynamical decoupling as primitive options; TREX with explicit random X gates before measurement). The local rehearsal runs the same Sampler calls on Qiskit Aer with the device's gate and readout errors on the qubits the circuits use.
4. **Replay.** The estimators run again on the measured frequencies.

The result has the simulator's structure, so the page shows the Playground's figures, analysis views and cost table, and its ZIP holds the same files plus the plan and the job IDs. The *sd* column is the standard error of the allocation actually run, $\sqrt{v/N_t + w/s + u}$ with v, w, u from the measured data; *sd* (*repeats*) resamples the measured counts. The API token is used once to connect and is never stored or exported.

16. Advanced mitigation methods

The methods of §4–§11 are the ones the app simulates end to end. This section surveys the wider toolbox a thesis on mitigation has to position itself in: how each method works, what it assumes, what it costs, and how it relates to the others. All of them are estimators of $\langle O\rangle = \mathrm{Tr}(\rho O)$ built from a noisy device; they differ in which *prior knowledge* they spend (a symmetry, a noise model, a scalable noise knob, extra copies, a subspace) and in how the sampling overhead γ grows.

16.1 Symmetry verification and symmetry expansion

Setting

Let $\{S\}$ be a group of commuting symmetries of the ideal state, $S|\psi\rangle = s|\psi\rangle$ — a parity $Z^{\otimes n}$, a particle number $N = \sum_j (1 - Z_j)/2$, a total spin, or the stabilizers of a code. The projector onto the correct symmetry sector is $\Pi_s = \frac{1}{|G|}\sum_{g\in G} \chi_s(g)\, g$.

Post-selection

(§10) prepares $\rho_{\mathrm{SV}} = \Pi_s\rho\Pi_s/\mathrm{Tr}(\Pi_s\rho)$: every error that leaves the sector is removed. When the symmetry is diagonal in the measured basis it costs nothing but discarded shots, $\gamma^2 = 1/P_{\mathrm{acc}}$ with $P_{\mathrm{acc}} = \mathrm{Tr}(\Pi_s\rho)$. If the symmetry is not diagonal (e.g. particle number in a Jordan–Wigner encoding measured in a rotated basis), it can be verified *in post-processing* [Bonet-Monroig 2018; McArdle 2019]: because Π_s is a sum of Pauli operators,

$$\langle O\rangle_{\mathrm{SV}} = \frac{\mathrm{Tr}(\Pi_s\rho\,\Pi_s O)}{\mathrm{Tr}(\Pi_s\rho)} = \frac{\sum_{g,g'} \chi_s(g)\chi_s(g')\,\mathrm{Tr}(g\rho g' O)}{|G|\sum_g \chi_s(g)\mathrm{Tr}(g\rho)},$$

and for commuting O the numerator reduces to $\langle \Pi_s O\rangle$, a sum of Pauli expectation values.

Symmetry expansion

[Cai 2021] generalizes the construction: any weighted combination $\langle O\rangle_{\mathrm{SE}} = \sum_g w_g\langle gO\rangle / \sum_g w_g\langle g\rangle$

is an estimator, and choosing the weights trades bias against variance (post-selection is one choice; others use only a subset of group elements to cut the sampling cost).

What it cannot do

Errors that commute with every symmetry (a Z error for a Z-parity, a logical error for a code) are invisible; the residual bias is the probability of an undetectable error. Symmetry verification is *error detection*: with code stabilizers as the symmetries it is exactly post-selected QEC (§17.5).

In the app: *Symmetry verification* post-selects on a Z-parity; the QEC templates set their stabilizer (Steane $Z_3Z_4Z_5Z_6$; for the $d = 2$ surface code both Z checks together, via the method's *further parities*) so detection can be compared with correction. Several parities can be verified at once; the accepted shots are those in which every check reads +1.

16.2 Zero-noise extrapolation at the pulse level

ZNE needs a knob λ that scales the noise while leaving the ideal operation unchanged. Two families exist:

- **Pulse stretching** (the original proposal [Temme 2017] and the first large demonstration [Kandala 2019]). Every gate is re-calibrated with its drive stretched in time by c_j and its amplitude reduced by $1/c_j$, so the unitary is unchanged. If the noise is time-invariant and weak — Lindbladian $\mathcal{L}$ acting during the pulse — the expectation value is an analytic function of the noise strength, $E(\lambda) = E_0 + \sum_{k\geq 1} a_k \lambda^k$, and Richardson extrapolation from $\lambda \in \{c_1, \dots, c_m\}$ cancels the first $m-1$ orders: the residual bias is $O(\lambda^m)$ (for the exact conditions see [Temme 2017]). Stretching scales the noise that accrues *in time* during the gates — relaxation, dephasing, always-on ZZ coupling, detunings — but not a fractional amplitude miscalibration (it keeps the same relative error at any stretch), nor readout or state-preparation errors. The price is calibration: every stretch factor needs its own pulse calibration.
- **Digital folding** (§4.2) [Giurgica-Tiron 2020]: $U \to U(U^\dagger U)^k$ inserts gates. No recalibration is needed and it runs on any gate-based device, but it scales noise only in the Markovian, gate-local picture; coherent errors can partly cancel in $U^\dagger U$ and non-Markovian noise does not scale linearly.
- **Probabilistic error amplification** (PEA) [Kim 2023b]: learn a sparse Pauli–Lindblad model of each layer [van den Berg 2023] and *add* sampled Pauli errors to reach $\lambda > 1$ exactly in that model. It combines the model accuracy of PEC with the modest overhead of ZNE and was used for the 127-qubit utility experiment.

The variance factor is $(\sum_j |\eta_j|)^2$ for linear extrapolation weights η_j (§4.4); Richardson weights grow quickly with the number of points and the spread of λ, which is why two or three scale factors are common.

16.3 Quantum subspace expansion (QSE)

QSE [McClean 2017] turns the noisy state into a small *variational space*: choose expansion operators $\{\sigma_i\}_{i=1}^m$ (e.g. identity, single-qubit Paulis, stabilizers) and look for the best state of the form $|\tilde{\psi}\rangle = \sum_i c_i \sigma_i |\psi\rangle$ (or, for mixed states, $\tilde{\rho} \propto C\rho C^\dagger$ with $C = \sum_i c_i \sigma_i$). Measuring the matrices

$$\tilde{H}_{ij} = \mathrm{Tr}(\rho\, \sigma_i^\dagger H \sigma_j), \qquad \tilde{S}_{ij} = \mathrm{Tr}(\rho\, \sigma_i^\dagger \sigma_j)$$

and solving the generalized eigenvalue problem $\tilde{H}\vec{c} = E\,\tilde{S}\vec{c}$ gives an energy that is variationally lower and, when the σ_i contain the inverse of the likely errors, corrects them. With the stabilizers of a code as expansion operators QSE acts as a *decoder* without mid-circuit measurements [McClean 2020]. The **generalized subspace expansion** [Yoshioka 2022] also allows powers of ρ (as in virtual

distillation) and the Hamiltonian itself, unifying QSE, symmetry verification and virtual distillation as special choices of subspace.

Costs and pitfalls: $O(m^2)$ matrix elements, each a measured expectation value; the generalized eigenproblem is ill-conditioned when $\tilde{S}$ is nearly singular, so shot noise must be regularized (thresholding small eigenvalues of $\tilde{S}$). The estimator is non-linear in the data and slightly biased at finite shots.

16.4 Classical shadows

A **classical shadow** [Huang 2020] is a compact classical description of ρ from randomized single-shot measurements. For random single-qubit Pauli bases, each shot gives a unitary $U = \bigotimes_j U_j$ and outcome $|b\rangle$; the unbiased snapshot is

$$\hat{\rho} = \bigotimes_{j=1}^{n} (3\, U_j^\dagger |b_j\rangle\langle b_j| U_j - I), \qquad \mathbb{E}[\hat{\rho}] = \rho.$$

Any k-local observable is estimated from the same data; M observables of locality k need $N = O\big(\log(M)\, 4^k \max_i \|O_i\|^2/\varepsilon^2\big)$ snapshots with median-of-means estimation (3^k for Pauli strings), independent of n. For mitigation the relevant facts are:

- **Noise-robust shadows.** Measurement noise changes the inverse of the measurement channel. Calibrating that channel on a known state ($|0\rangle^{\otimes n}$) and inverting the *noisy* channel removes readout and gate-dependent measurement errors that are Pauli-twirled [Chen 2021; Koh 2022] — the shadow analog of readout mitigation.
- **Shadow distillation.** Estimating $\mathrm{Tr}(\rho^2 O)/\mathrm{Tr}(\rho^2)$ from shadows gives virtual distillation without two copies or entangling derangement circuits, at the price of more samples [Seif 2023].
- **Shadow-based subspace expansion.** The $\tilde{H}, \tilde{S}$ matrices of §16.3 can all be read off one shadow data set.

Shadows do not reduce the bias of gate noise by themselves; they are an efficient *measurement* layer on which mitigation estimators are built.

16.5 M3 in depth

For correlated readout errors the full assignment matrix $A \in \mathbb{R}^{2^n \times 2^n}$ is intractable, and even the tensored inverse (§7.1) produces a dense quasi-distribution. **Matrix-free measurement mitigation** [Nation 2021] restricts the problem to the set $\mathcal{B}$ of bit strings actually observed:

1. Build $\tilde{A}$, the sub-matrix of A on $\mathcal{B}$ (optionally only entries within Hamming distance D), from per-qubit (or small-cluster) calibrations; renormalize columns so probability leaking out of $\mathcal{B}$ is accounted for.
2. Solve $\tilde{A}\,\vec{p} = \vec{f}$ for the measured frequencies $\vec{f}$ — directly for small $|\mathcal{B}|$, otherwise with a preconditioned iterative solver (GMRES/BiCGSTAB with the diagonal as preconditioner). The matrix is never formed in the 2^n space.
3. Return the quasi-probabilities $\vec{p}$ and, optionally, the nearest true probability distribution.

Cost scales with $|\mathcal{B}| \le N_{\text{shots}}$ instead of 2^n. With exact (noise-free) data every bit string is observed and M3 equals the tensored inverse, which is what the app uses; the difference appears with finite shots on many qubits. Variance amplification is the same as for the tensored inverse, $\prod_i (1 - \epsilon_{01}^{(i)} - \epsilon_{10}^{(i)})^{-2}$ to leading order, so readout mitigation is cheap only while readout errors are small.

16.6 How the methods relate, and their limits

method	knowledge spent	bias removed	overhead	scaling with circuit size
readout (tensored, M3, TREX)	calibration of the measurement	readout only	$\prod_i (1-2\bar{\epsilon}_i)^{-2}$	per measured qubit
symmetry verification / expansion	a symmetry of the ideal state	errors that break it	$1/P_{\text{acc}}$	P_{acc} decays exponentially
ZNE (folding, pulse, PEA)	a noise knob λ	up to order λ^{m-1}	$(\sum_j \lvert\eta_j\rvert)^2$	modest; bias grows with depth
PEC	a full noise model	all modeled noise	$\gamma^2 = \prod_g \gamma_g^2$	exponential in total error
learning (CDR, NIL)	near-Clifford training data	the learned map	training circuits, $\lVert c\rVert_1^2$	depends on training fidelity
virtual distillation / QSE	copies / a subspace	incoherent errors	$\mathrm{Tr}(\rho^M)^{-2}$ / $O(m^2)$ terms	exponential in total error

Two general results frame all of them. First, lower bounds show that for broad classes of noise every unbiased mitigation protocol needs a number of samples exponential in circuit depth or size (§3.2) [Takagi 2022; Quek 2024]. Second, mitigation and correction compose: logical qubits have much smaller error rates, so the same estimators at the logical level cost exponentially less (§17.5) [Suzuki 2022]. A thesis that proposes a method should therefore report bias *and* sampling overhead as functions of the circuit's total error $\sum_g \epsilon_g$, as the Cost vs precision and Head-to-head pages do.

17. Toward fault tolerance: a QEC roadmap

Error mitigation (§3–§16) corrects *averages* after the fact, with a sampling cost that grows exponentially with the total error of the circuit. Quantum error correction (QEC) removes errors *during* the computation, with a qubit cost that grows only polylogarithmically with the circuit size once the physical error rate is below a threshold. This section is a compact map of that second road — the vocabulary a thesis on mitigation needs to place its results — and of the regime where the two roads meet (§17.5).

17.1 The stabilizer formalism

Pauli group and stabilizers

The n-qubit Pauli group $\mathcal{P}_n$ consists of tensor products of I, X, Y, Z with phases $\pm 1, \pm i$. A **stabilizer code** is defined by an abelian subgroup $\mathcal{S} = \langle g_1, \dots, g_{n-k} \rangle \subset \mathcal{P}_n$ with $-I \notin \mathcal{S}$; its code space

$$\mathcal{C}(\mathcal{S}) = \{|\psi\rangle : \; g|\psi\rangle = |\psi\rangle \;\; \forall g \in \mathcal{S}\}$$

has dimension 2^k: k logical qubits in n physical ones [Gottesman 1997].

Logical operators and distance

Operators that commute with every stabilizer but are not stabilizers themselves, $\mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$, act on the code space as logical Paulis $\bar{X}_j, \bar{Z}_j$. The **distance** d is the minimum weight of such an operator; the code is written $[[n, k, d]]$ and corrects any error on up to $t = \lfloor (d-1)/2 \rfloor$ qubits, or detects up to $d-1$.

Syndromes

A Pauli error E either commutes or anticommutes with each generator; the **syndrome** is the bit vector $s_j = 1$ iff $Eg_j = -g_jE$, measured with ancilla qubits without disturbing the encoded information. Errors that differ by a stabilizer have the same effect (**degeneracy**); errors that differ by a logical operator have the same syndrome but act differently on the encoded information (they differ by a logical operation) — the decoder's fundamental ambiguity.

Knill–Laflamme conditions

A set of errors $\{E_a\}$ is correctable iff $PE_a^\dagger E_b P = c_{ab}P$ for the code projector P [Knill 1997]; for stabilizer codes this means $E_a^\dagger E_b \notin \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$.

Symplectic picture and CSS codes

Writing a Pauli as a binary vector $(\mathbf{x}|\mathbf{z}) \in \mathbb{F}_2^{2n}$ turns commutation into a symplectic inner product and a code into a check matrix $H = (H_X|H_Z)$. **CSS codes** [Calderbank 1996; Steane 1996] use separate X- and Z-type checks from two classical codes with $H_X H_Z^\mathsf{T} = 0$; their X and Z errors are decoded independently (Steane, surface, color and most qLDPC codes are CSS).

Clifford group and Gottesman–Knill

Unitaries that map Paulis to Paulis form the Clifford group (generated by H, S, CX). Clifford circuits with Pauli measurements are simulable in polynomial time with a stabilizer tableau [Aaronson 2004] — which is why QEC circuits can be simulated at scale, and why universality needs a non-Clifford gate such as T.

The codes in this app.

template	parameters	stabilizers	corrects / detects
3-qubit bit-flip	$[[3,1,1]]$ (distance 3 against X)	$Z_0Z_1,\ Z_1Z_2$	one X
repetition memory	d bits, distance d against X	Z_iZ_{i+1}	$\lfloor (d-1)/2 \rfloor$ flips
surface code, one plaquette	$[[4,1,2]]$	$X_0X_1X_2X_3,\ Z_0Z_1,\ Z_2Z_3$	detects any one error (the app post-selects on both Z checks: the X/Y errors that can flip Z_L)
five-qubit code	$[[5,1,3]]$	$XZZXI$ and cyclic shifts	any one error (perfect code)
Steane code	$[[7,1,3]]$	X and Z on the Hamming checks	any one error (CSS)

17.2 A taxonomy of codes

family	examples	parameters and geometry	why it matters
repetition	bit-flip, phase-flip	$[[d,1]]$ against one error type	the classical intuition; biased-noise building block

family	examples	parameters and geometry	why it matters
small perfect / CSS codes	Shor $[[9,1,3]]$ [Shor 1995], five-qubit [Laflamme 1996; Bennett 1996], Steane $[[7,1,3]]$	few qubits, non-local checks	first demonstrations; transversal Cliffords (Steane)
surface / toric codes	Kitaev's toric code [Kitaev 2003], rotated surface code [Fowler 2012]	toric $[[2d^2,2,d]]$; rotated patch $[[d^2,1,d]]$; 2D grid, weight-4 checks	circuit-level threshold ≈ 1 % with local checks; the leading architecture
heavy-hex and subsystem codes	Bacon–Shor [Bacon 2006], heavy-hex codes [Chamberland 2020]	degree-3 connectivity, gauge operators	matched to IBM's lattice; flags catch hook errors
color codes	[Bombin 2006]	2D/3D, weight-6 checks	transversal Clifford group in 2D
quantum LDPC codes	hypergraph product [Tillich 2014], bivariate bicycle "gross" code $[[144,12,12]]$ [Bravyi 2024]	constant check weight, long-range links	many logical qubits per physical qubit
bosonic codes	cat codes [Mirrahimi 2014], GKP [Gottesman 2001]	one oscillator per logical qubit	hardware-efficient; biased noise
concatenated codes	a code inside a code	$[[n^L,1,d^L]]$	the original route to the threshold theorem

17.3 Decoding

A **decoder** maps a syndrome history to a correction. For a stabilizer code with independent errors of probability p, the optimal (maximum-likelihood) decoder picks the most likely *logical class* of errors consistent with the syndrome; practical decoders approximate it:

- **Lookup tables** for small codes — the app decodes the five-qubit and Steane codes this way (16 and 8 syndromes).
- **Minimum-weight perfect matching** (MWPM) [Edmonds 1965; Dennis 2002]: for codes where each error flips at most two checks (surface and repetition codes), syndromes are vertices of a graph weighted by $-\log p$ and the most likely error is a minimum-weight matching. PyMatching [Higgott 2022] and Sparse Blossom [Higgott 2025] run at the speed hardware needs.
- **Union–find** [Delfosse 2021]: almost-linear time with a threshold close to matching.
- **Belief propagation with ordered-statistics decoding** (BP+OSD) [Panteleev 2021; Roffe 2020] for qLDPC codes.
- **Tensor-network / maximum-likelihood** decoders [Bravyi 2014]: near-optimal, used as a benchmark.
- **Neural decoders** trained on device data, e.g. AlphaQubit [Bausch 2024], learn correlated and leakage errors.

Circuit-level decoding works on **detectors** (parities of consecutive syndrome measurements) and a detector error model generated from the noisy syndrome-extraction circuit, e.g. with Stim [Gidney 2021]. Below threshold the logical error per cycle falls by a factor Λ each time d increases by 2,

$$\varepsilon_L(d) \approx A\,\Lambda^{-(d+1)/2}, \qquad \Lambda \approx p_{\mathrm{th}}/p,$$

and $\Lambda > 1$ is the experimental signature of operating below threshold. Surface-code memories have

shown $\Lambda > 1$ [Google 2023] and $\Lambda \approx 2$ up to distance 7, with real-time decoding demonstrated in the same experiment [Google 2025].

17.4 Fault tolerance

Threshold theorem

If every component fails independently with probability below a constant p_{th}, an ideal circuit of size G can be simulated with error ε using $O(G \operatorname{polylog}(G/\varepsilon))$ components [Aharonov 2008; Knill 1998; Kitaev 2003]. It requires **fault-tolerant gadgets**: syndrome extraction that does not spread one fault into an uncorrectable error (repeated measurement, flag qubits [Chao 2018], careful CX ordering to avoid hook errors).

Logical gates

Transversal gates apply the same gate to every physical qubit and cannot spread errors, but no code has a universal transversal gate set (Eastin–Knill [Eastin 2009]). The standard solution: Clifford gates by transversal operations or **lattice surgery** [Horsman 2012], and the T gate by **magic-state distillation** [Bravyi 2005] — a protocol that turns many noisy $|T\rangle$ states into fewer, cleaner ones and dominates the space-time cost of fault-tolerant algorithms.

Resource estimates

Factoring RSA-2048 with surface codes was estimated at about 20 million noisy qubits [Gidney 2021b], later reduced to under one million with better arithmetic and magic-state cultivation [Gidney 2025]. Such numbers — and the depth of the "repeated powers" Shor template in this app — are why error correction, not mitigation, is the route to large algorithms.

17.5 Logical error mitigation

Early fault-tolerant machines will have a modest code distance: logical error rates ε_L of perhaps 10^{-3}–10^{-6} per logical operation, noisy magic states, and circuits with more logical operations than $1/\varepsilon_L$. Mitigation applied **to logical qubits** fills that gap, and it is exponentially cheaper there because the overhead depends on the *logical* error budget: with G logical gates, $\gamma^2 \approx e^{4G\varepsilon_L}$ instead of e^{4Gp} [Suzuki 2022].

- **Logical PEC.** The noise of logical operations after decoding is close to Pauli and can be characterized from syndrome statistics; PEC with that logical noise model cancels residual logical errors, including those of non-Clifford gates implemented with imperfect magic states [Piveteau 2021; Lostaglio 2021].
- **Logical ZNE.** The code distance is a natural noise knob: $\varepsilon_L(d)$ from runs at $d = 3, 5, 7$ extrapolated to zero logical error [Wahl 2023].
- **Detection and post-selection.** Small codes used as *detection* codes (distance 2, like the surface-code plaquette here, post-selected on both of its Z checks) discard shots with a non-trivial syndrome — symmetry verification with stabilizers as the symmetry (§16.1). The acceptance rate is the price, exactly as in §10.
- **Hybrid error budgets.** For a fixed number of physical qubits one can trade code distance (qubits) against mitigation overhead (shots); lower distance plus mitigation can beat higher distance alone for short circuits [Suzuki 2022]. First experiments demonstrate mitigation on encoded qubits [Logical QEM 2025].

In the app: the *surface code d = 2* template with *Symmetry verification* is detection + post-selection; the *five-qubit code* with *ZNE* or *PEC* is correction + mitigation. The *Logical error & syndromes* view reports the effective logical error $p_L = \frac{1}{2}(1 - \langle Z_L\rangle / \langle Z_L\rangle_{\text{ideal}})$ of each.

17.6 The roadmap at a glance

regime	what protects the computation	cost grows with	the app's examples
NISQ	error mitigation (ZNE, PEC, learning, readout, twirling, DD)	shots: $\sim e^{c\,Gp}$	every template, *Mitigate & compare*
early fault tolerance	small codes for detection + mitigation of residual logical errors	shots $\sim e^{c\,G\varepsilon_L}$ and qubits $\sim d^2$	surface plaquette + post-selection, five-qubit code + ZNE/PEC
fault tolerance	error correction below threshold, lattice surgery, magic states	qubits $\sim d^2$ polylog, time $\sim d$ per step	repetition memory (d), Steane and five-qubit decoders

Reading the table from top to bottom is reading the next decade of the field: the estimators this app compares stay useful at every stage — first on physical qubits, then on logical ones.

References

- [Aaronson 2004] S. Aaronson, D. Gottesman, *Improved simulation of stabilizer circuits*, Phys. Rev. A **70**, 052328 (2004). doi:10.1103/PhysRevA.70.052328 · arXiv:quant-ph/0406196
- [Aharonov 2008] D. Aharonov, M. Ben-Or, *Fault-tolerant quantum computation with constant error rate*, SIAM J. Comput. **38**, 1207 (2008). doi:10.1137/S0097539799359385 · arXiv:quant-ph/9906129
- [Bacon 2006] D. Bacon, *Operator quantum error-correcting subsystems for self-correcting quantum memories*, Phys. Rev. A **73**, 012340 (2006). doi:10.1103/PhysRevA.73.012340 · arXiv:quant-ph/0506023
- [Barenco 1996] A. Barenco, A. Ekert, K.-A. Suominen, P. Törmä, *Approximate quantum Fourier transform and decoherence*, Phys. Rev. A **54**, 139 (1996). doi:10.1103/PhysRevA.54.139 · arXiv:quant-ph/9601018
- [Bausch 2024] J. Bausch et al., *Learning high-accuracy error decoding for quantum processors*, Nature **635**, 834 (2024). doi:10.1038/s41586-024-08148-8
- [Bennett 1996] C. H. Bennett, D. P. DiVincenzo, J. A. Smolin, W. K. Wootters, *Mixed-state entanglement and quantum error correction*, Phys. Rev. A **54**, 3824 (1996). doi:10.1103/PhysRevA.54.3824 · arXiv:quant-ph/9604024
- [Bennett 1997b] C. H. Bennett, E. Bernstein, G. Brassard, U. Vazirani, *Strengths and weaknesses of quantum computing*, SIAM J. Comput. **26**, 1510 (1997). doi:10.1137/S0097539796300933 · arXiv:quant-ph/9701001
- [Bernstein 1997] E. Bernstein, U. Vazirani, *Quantum complexity theory*, SIAM J. Comput. **26**, 1411 (1997). doi:10.1137/S0097539796300921
- [Bombin 2006] H. Bombin, M. A. Martin-Delgado, *Topological quantum distillation*, Phys. Rev. Lett. **97**, 180501 (2006). doi:10.1103/PhysRevLett.97.180501 · arXiv:quant-ph/0605138
- [Bonet-Monroig 2018] X. Bonet-Monroig, R. Sagastizabal, M. Singh, T. E. O'Brien, *Low-cost error mitigation by symmetry verification*, Phys. Rev. A **98**, 062339 (2018). doi:10.1103/PhysRevA.98.062339 · arXiv:1807.10050
- [Boyer 1998] M. Boyer, G. Brassard, P. Høyer, A. Tapp, *Tight bounds on quantum searching*, Fortschr. Phys. **46**, 493 (1998). doi:10.1002/(SICI)1521-3978(199806)46:4/5%3C493::AID-PROP493%3E3.0.CO;2-P · arXiv:quant-ph/9605034
- [Bravyi 2005] S. Bravyi, A. Kitaev, *Universal quantum computation with ideal Clifford gates and noisy ancillas*, Phys. Rev. A **71**, 022316 (2005). doi:10.1103/PhysRevA.71.022316 · arXiv:quant-ph/0403025

- [Bravyi 2014] S. Bravyi, M. Suchara, A. Vargo, *Efficient algorithms for maximum likelihood decoding in the surface code*, Phys. Rev. A **90**, 032326 (2014). doi:10.1103/PhysRevA.90.032326 · arXiv:1405.4883
- [Bravyi 2021] S. Bravyi, S. Sheldon, A. Kandala, D. C. Mckay, J. M. Gambetta, *Mitigating measurement errors in multiqubit experiments*, Phys. Rev. A **103**, 042605 (2021). doi:10.1103/PhysRevA.103.042605 · arXiv:2006.14044
- [Bravyi 2024] S. Bravyi et al., *High-threshold and low-overhead fault-tolerant quantum memory*, Nature **627**, 778 (2024). doi:10.1038/s41586-024-07107-7 · arXiv:2308.07915
- [Cai 2021] Z. Cai, *Quantum error mitigation using symmetry expansion*, Quantum **5**, 548 (2021). doi:10.22331/q-2021-09-21-548 · arXiv:2101.03151
- [Cai 2023] Z. Cai et al., *Quantum error mitigation*, Rev. Mod. Phys. **95**, 045005 (2023). doi:10.1103/RevModPhys.95.045005 · arXiv:2210.00921
- [Calderbank 1996] A. R. Calderbank, P. W. Shor, *Good quantum error-correcting codes exist*, Phys. Rev. A **54**, 1098 (1996). doi:10.1103/PhysRevA.54.1098 · arXiv:quant-ph/9512032
- [Cao 2012] Y. Cao, A. Daskin, S. Frankel, S. Kais, *Quantum circuit design for solving linear systems of equations*, Mol. Phys. **110**, 1675 (2012). doi:10.1080/00268976.2012.668289 · arXiv:1110.2232
- [Chamberland 2020] C. Chamberland, G. Zhu, T. J. Yoder, J. B. Hertzberg, A. W. Cross, *Topological and subsystem codes on low-degree graphs with flag qubits*, Phys. Rev. X **10**, 011022 (2020). doi:10.1103/PhysRevX.10.011022 · arXiv:1907.09528
- [Chao 2018] R. Chao, B. W. Reichardt, *Quantum error correction with only two extra qubits*, Phys. Rev. Lett. **121**, 050502 (2018). doi:10.1103/PhysRevLett.121.050502 · arXiv:1705.02329
- [Chen 2021] S. Chen, W. Yu, P. Zeng, S. T. Flammia, *Robust shadow estimation*, PRX Quantum **2**, 030348 (2021). doi:10.1103/PRXQuantum.2.030348 · arXiv:2011.09636
- [Childs 2010] A. M. Childs, W. van Dam, *Quantum algorithms for algebraic problems*, Rev. Mod. Phys. **82**, 1 (2010). doi:10.1103/RevModPhys.82.1 · arXiv:0812.0380
- [Coppersmith 1994] D. Coppersmith, *An approximate Fourier transform useful in quantum factoring*, IBM Research Report RC19642 (1994). arXiv:quant-ph/0201067
- [Czarnik 2021] P. Czarnik, A. Arrasmith, P. J. Coles, L. Cincio, *Error mitigation with Clifford quantum-circuit data*, Quantum **5**, 592 (2021). doi:10.22331/q-2021-11-26-592 · arXiv:2005.10189
- [Delfosse 2021] N. Delfosse, N. H. Nickerson, *Almost-linear time decoding algorithm for topological codes*, Quantum **5**, 595 (2021). doi:10.22331/q-2021-12-02-595 · arXiv:1709.06218
- [Dennis 2002] E. Dennis, A. Kitaev, A. Landahl, J. Preskill, *Topological quantum memory*, J. Math. Phys. **43**, 4452 (2002). doi:10.1063/1.1499754 · arXiv:quant-ph/0110143
- [Eastin 2009] B. Eastin, E. Knill, *Restrictions on transversal encoded quantum gate sets*, Phys. Rev. Lett. **102**, 110502 (2009). doi:10.1103/PhysRevLett.102.110502 · arXiv:0811.4262
- [Edmonds 1965] J. Edmonds, *Paths, trees, and flowers*, Can. J. Math. **17**, 449 (1965). doi:10.4153/CJM-1965-045-4
- [Endo 2018] S. Endo, S. C. Benjamin, Y. Li, *Practical quantum error mitigation for near-future applications*, Phys. Rev. X **8**, 031027 (2018). doi:10.1103/PhysRevX.8.031027 · arXiv:1712.09271
- [Ezzell 2023] N. Ezzell, B. Pokharel, L. Tewala, G. Quiroz, D. A. Lidar, *Dynamical decoupling for superconducting qubits: a performance survey*, Phys. Rev. Applied **20**, 064027 (2023). doi:10.1103/PhysRevApplied.20.064027 · arXiv:2207.03670
- [Farhi 2014] E. Farhi, J. Goldstone, S. Gutmann, *A quantum approximate optimization algorithm*, arXiv:1411.4028 (2014)
- [Fowler 2012] A. G. Fowler, M. Mariantoni, J. M. Martinis, A. N. Cleland, *Surface codes: towards practical large-scale quantum computation*, Phys. Rev. A **86**, 032324 (2012). doi:10.1103/PhysRevA.86.032324 · arXiv:1208.0928
- [Gidney 2021] C. Gidney, *Stim: a fast stabilizer circuit simulator*, Quantum **5**, 497 (2021). doi:10.22331/q-2021-07-06-497 · arXiv:2103.02202
- [Gidney 2021b] C. Gidney, M. Ekerå, *How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits*, Quantum **5**, 433 (2021). doi:10.22331/q-2021-04-15-433 · arXiv:1905.09749
- [Gidney 2025] C. Gidney, *How to factor 2048 bit RSA integers with less than a million noisy qubits*,

arXiv:2505.15917 (2025)
- [Giurgica-Tiron 2020] T. Giurgica-Tiron et al., *Digital zero noise extrapolation for quantum error mitigation*, IEEE QCE 2020, p. 306. doi:10.1109/QCE49297.2020.00045 · arXiv:2005.10921
- [Google 2023] Google Quantum AI, *Suppressing quantum errors by scaling a surface code logical qubit*, Nature **614**, 676 (2023). doi:10.1038/s41586-022-05434-1
- [Google 2025] Google Quantum AI and collaborators, *Quantum error correction below the surface code threshold*, Nature **638**, 920 (2025). doi:10.1038/s41586-024-08449-y · arXiv:2408.13687
- [Gottesman 1997] D. Gottesman, *Stabilizer codes and quantum error correction*, PhD thesis, California Institute of Technology (1997). arXiv:quant-ph/9705052
- [Gottesman 2001] D. Gottesman, A. Kitaev, J. Preskill, *Encoding a qubit in an oscillator*, Phys. Rev. A **64**, 012310 (2001). doi:10.1103/PhysRevA.64.012310 · arXiv:quant-ph/0008040
- [Grover 1996] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proc. 28th ACM STOC, 212 (1996). doi:10.1145/237814.237866 · arXiv:quant-ph/9605043
- [Harrow 2009] A. W. Harrow, A. Hassidim, S. Lloyd, *Quantum algorithm for linear systems of equations*, Phys. Rev. Lett. **103**, 150502 (2009). doi:10.1103/PhysRevLett.103.150502 · arXiv:0811.3171
- [Hashim 2021] A. Hashim et al., *Randomized compiling for scalable quantum computing on a noisy superconducting quantum processor*, Phys. Rev. X **11**, 041039 (2021). doi:10.1103/PhysRevX.11.041039 · arXiv:2010.00215
- [Havlíček 2019] V. Havlíček et al., *Supervised learning with quantum-enhanced feature spaces*, Nature **567**, 209 (2019). doi:10.1038/s41586-019-0980-2 · arXiv:1804.11326
- [Higgott 2022] O. Higgott, *PyMatching: a Python package for decoding quantum codes with minimum-weight perfect matching*, ACM Trans. Quantum Comput. **3**, 16 (2022). doi:10.1145/3505637 · arXiv:2105.13082
- [Higgott 2025] O. Higgott, C. Gidney, *Sparse Blossom: correcting a million errors per core second with minimum-weight matching*, Quantum **9**, 1600 (2025). doi:10.22331/q-2025-01-20-1600 · arXiv:2303.15933
- [Horsman 2012] C. Horsman, A. G. Fowler, S. Devitt, R. Van Meter, *Surface code quantum computing by lattice surgery*, New J. Phys. **14**, 123011 (2012). doi:10.1088/1367-2630/14/12/123011 · arXiv:1111.4022
- [Huang 2020] H.-Y. Huang, R. Kueng, J. Preskill, *Predicting many properties of a quantum system from very few measurements*, Nat. Phys. **16**, 1050 (2020). doi:10.1038/s41567-020-0932-7 · arXiv:2002.08953
- [Huggins 2021] W. J. Huggins et al., *Virtual distillation for quantum error mitigation*, Phys. Rev. X **11**, 041036 (2021). doi:10.1103/PhysRevX.11.041036 · arXiv:2011.07064
- [Javadi-Abhari 2024] A. Javadi-Abhari et al., *Quantum computing with Qiskit*, arXiv:2405.08810 (2024)
- [Jozsa 2001] R. Jozsa, *Quantum factoring, discrete logarithms, and the hidden subgroup problem*, Comput. Sci. Eng. **3**(2), 34 (2001). doi:10.1109/5992.909000 · arXiv:quant-ph/0012084
- [Kandala 2017] A. Kandala et al., *Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets*, Nature **549**, 242 (2017). doi:10.1038/nature23879 · arXiv:1704.05018
- [Kandala 2019] A. Kandala et al., *Error mitigation extends the computational reach of a noisy quantum processor*, Nature **567**, 491 (2019). doi:10.1038/s41586-019-1040-7 · arXiv:1805.04492
- [Kim 2023a] Y. Kim et al., *Scalable error mitigation for noisy quantum circuits produces competitive expectation values*, Nat. Phys. **19**, 752 (2023). doi:10.1038/s41567-022-01914-3 · arXiv:2108.09197
- [Kim 2023b] Y. Kim et al., *Evidence for the utility of quantum computing before fault tolerance*, Nature **618**, 500 (2023). doi:10.1038/s41586-023-06096-3
- [Kitaev 1995] A. Yu. Kitaev, *Quantum measurements and the Abelian stabilizer problem*, arXiv:quant-ph/9511026 (1995)
- [Kitaev 2003] A. Yu. Kitaev, *Fault-tolerant quantum computation by anyons*, Ann. Phys. **303**, 2 (2003). doi:10.1016/S0003-4916(02)00018-0 · arXiv:quant-ph/9707021

- [Knill 1997] E. Knill, R. Laflamme, *Theory of quantum error-correcting codes*, Phys. Rev. A **55**, 900 (1997). doi:10.1103/PhysRevA.55.900 · arXiv:quant-ph/9604034
- [Knill 1998] E. Knill, R. Laflamme, W. H. Zurek, *Resilient quantum computation*, Science **279**, 342 (1998). doi:10.1126/science.279.5349.342 · arXiv:quant-ph/9702058
- [Knill 2005] E. Knill, *Quantum computing with realistically noisy devices*, Nature **434**, 39 (2005). doi:10.1038/nature03350 · arXiv:quant-ph/0410199
- [Koczor 2021] B. Koczor, *Exponential error suppression for near-term quantum devices*, Phys. Rev. X **11**, 031057 (2021). doi:10.1103/PhysRevX.11.031057 · arXiv:2011.05942
- [Koh 2022] D. E. Koh, S. Grewal, *Classical shadows with noise*, Quantum **6**, 776 (2022). doi:10.22331/q-2022-08-16-776 · arXiv:2011.11580
- [Laflamme 1996] R. Laflamme, C. Miquel, J. P. Paz, W. H. Zurek, *Perfect quantum error correcting code*, Phys. Rev. Lett. **77**, 198 (1996). doi:10.1103/PhysRevLett.77.198 · arXiv:quant-ph/9602019
- [Li 2017] Y. Li, S. C. Benjamin, *Efficient variational quantum simulator incorporating active error minimization*, Phys. Rev. X **7**, 021050 (2017). doi:10.1103/PhysRevX.7.021050 · arXiv:1611.09301
- [Li 2019] G. Li, Y. Ding, Y. Xie, *Tackling the qubit mapping problem for NISQ-era quantum devices*, ASPLOS 2019. doi:10.1145/3297858.3304023 · arXiv:1809.02573
- [Logical QEM 2025] *Demonstrating quantum error mitigation on logical qubits*, Nat. Commun. (2025). doi:10.1038/s41467-025-67768-4 · arXiv:2501.09079
- [Lostaglio 2021] M. Lostaglio, A. Ciani, *Error mitigation and quantum-assisted simulation in the error corrected regime*, Phys. Rev. Lett. **127**, 200506 (2021). doi:10.1103/PhysRevLett.127.200506 · arXiv:2103.07526
- [Lowe 2021] A. Lowe et al., *Unified approach to data-driven quantum error mitigation*, Phys. Rev. Research **3**, 033098 (2021). doi:10.1103/PhysRevResearch.3.033098 · arXiv:2011.01157
- [Mari 2021] A. Mari, N. Shammah, W. J. Zeng, *Extending quantum probabilistic error cancellation by noise scaling*, Phys. Rev. A **104**, 052607 (2021). doi:10.1103/PhysRevA.104.052607 · arXiv:2108.02237
- [McArdle 2019] S. McArdle, X. Yuan, S. Benjamin, *Error-mitigated digital quantum simulation*, Phys. Rev. Lett. **122**, 180501 (2019). doi:10.1103/PhysRevLett.122.180501 · arXiv:1807.02467
- [McClean 2017] J. R. McClean, M. E. Kimchi-Schwartz, J. Carter, W. A. de Jong, *Hybrid quantum-classical hierarchy for mitigation of decoherence and determination of excited states*, Phys. Rev. A **95**, 042308 (2017). doi:10.1103/PhysRevA.95.042308 · arXiv:1603.05681
- [McClean 2020] J. R. McClean, Z. Jiang, N. C. Rubin, R. Babbush, H. Neven, *Decoding quantum errors with subspace expansions*, Nat. Commun. **11**, 636 (2020). doi:10.1038/s41467-020-14341-w · arXiv:1903.05786
- [Mirrahimi 2014] M. Mirrahimi et al., *Dynamically protected cat-qubits: a new paradigm for universal quantum computation*, New J. Phys. **16**, 045014 (2014). doi:10.1088/1367-2630/16/4/045014 · arXiv:1312.2017
- [Nation 2021] P. D. Nation, H. Kang, N. Sundaresan, J. M. Gambetta, *Scalable mitigation of measurement errors on quantum computers*, PRX Quantum **2**, 040326 (2021). doi:10.1103/PRXQuantum.2.040326 · arXiv:2108.12518
- [Nielsen 2002] M. A. Nielsen, *A simple formula for the average gate fidelity of a quantum dynamical operation*, Phys. Lett. A **303**, 249 (2002). doi:10.1016/S0375-9601(02)01272-0 · arXiv:quant-ph/0205035
- [Nielsen 2010] M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary edition, Cambridge University Press (2010)
- [Panteleev 2021] P. Panteleev, G. Kalachev, *Degenerate quantum LDPC codes with good finite length performance*, Quantum **5**, 585 (2021). doi:10.22331/q-2021-11-22-585 · arXiv:1904.02703
- [Peruzzo 2014] A. Peruzzo et al., *A variational eigenvalue solver on a photonic quantum processor*, Nat. Commun. **5**, 4213 (2014). doi:10.1038/ncomms5213 · arXiv:1304.3061
- [Piveteau 2021] C. Piveteau, D. Sutter, S. Bravyi, J. M. Gambetta, K. Temme, *Error mitigation for universal gates on encoded qubits*, Phys. Rev. Lett. **127**, 200505 (2021).

doi:10.1103/PhysRevLett.127.200505 · arXiv:2103.04915

- [Pérez-Salinas 2020] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, J. I. Latorre, *Data re-uploading for a universal quantum classifier*, Quantum **4**, 226 (2020). doi:10.22331/q-2020-02-06-226 · arXiv:1907.02085
- [Quek 2024] Y. Quek, D. Stilck França, S. Khatri, J. J. Meyer, J. Eisert, *Exponentially tighter bounds on limitations of quantum error mitigation*, Nat. Phys. **20**, 1648 (2024). doi:10.1038/s41567-024-02536-7 · arXiv:2210.11505
- [Roffe 2020] J. Roffe, D. R. White, S. Burton, E. Campbell, *Decoding across the quantum low-density parity-check code landscape*, Phys. Rev. Research **2**, 043423 (2020). doi:10.1103/PhysRevResearch.2.043 · arXiv:2005.07016
- [Seif 2023] A. Seif, Z.-P. Cian, S. Zhou, S. Chen, L. Jiang, *Shadow distillation: quantum error mitigation with classical shadows for near-term quantum processors*, PRX Quantum **4**, 010303 (2023). doi:10.1103/PRXQuantum.4.010303 · arXiv:2203.07309
- [Shor 1995] P. W. Shor, *Scheme for reducing decoherence in quantum computer memory*, Phys. Rev. A **52**, R2493 (1995). doi:10.1103/PhysRevA.52.R2493
- [Shor 1997] P. W. Shor, *Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer*, SIAM J. Comput. **26**, 1484 (1997). doi:10.1137/S0097539795293172 · arXiv:quant-ph/9508027
- [Simon 1997] D. R. Simon, *On the power of quantum computation*, SIAM J. Comput. **26**, 1474 (1997). doi:10.1137/S0097539796298637
- [Steane 1996] A. M. Steane, *Error correcting codes in quantum theory*, Phys. Rev. Lett. **77**, 793 (1996). doi:10.1103/PhysRevLett.77.793
- [Strikis 2021] A. Strikis, D. Qin, Y. Chen, S. C. Benjamin, Y. Li, *Learning-based quantum error mitigation*, PRX Quantum **2**, 040330 (2021). doi:10.1103/PRXQuantum.2.040330 · arXiv:2005.07601
- [Suzuki 2022] Y. Suzuki, S. Endo, K. Fujii, Y. Tokunaga, *Quantum error mitigation as a universal error reduction technique*, PRX Quantum **3**, 010345 (2022). doi:10.1103/PRXQuantum.3.010345 · arXiv:2010.03887
- [Takagi 2022] R. Takagi, S. Endo, S. Minagawa, M. Gu, *Fundamental limits of quantum error mitigation*, npj Quantum Inf. **8**, 114 (2022). doi:10.1038/s41534-022-00618-z · arXiv:2109.04457
- [Temme 2017] K. Temme, S. Bravyi, J. M. Gambetta, *Error mitigation for short-depth quantum circuits*, Phys. Rev. Lett. **119**, 180509 (2017). doi:10.1103/PhysRevLett.119.180509 · arXiv:1612.02058
- [Tillich 2014] J.-P. Tillich, G. Zémor, *Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength*, IEEE Trans. Inf. Theory **60**, 1193 (2014). doi:10.1109/TIT.2013.2292061 · arXiv:0903.0566
- [van den Berg 2022] E. van den Berg, Z. K. Minev, K. Temme, *Model-free readout-error mitigation for quantum expectation values*, Phys. Rev. A **105**, 032620 (2022). doi:10.1103/PhysRevA.105.032620 · arXiv:2012.09738
- [van den Berg 2023] E. van den Berg, Z. K. Minev, A. Kandala, K. Temme, *Probabilistic error cancellation with sparse Pauli–Lindblad models on noisy quantum processors*, Nat. Phys. **19**, 1116 (2023). doi:10.1038/s41567-023-02042-2 · arXiv:2201.09866
- [Vandersypen 2001] L. M. K. Vandersypen et al., *Experimental realization of Shor's quantum factoring algorithm using nuclear magnetic resonance*, Nature **414**, 883 (2001). doi:10.1038/414883a · arXiv:quant-ph/0112176
- [Viola 1998] L. Viola, S. Lloyd, *Dynamical suppression of decoherence in two-state quantum systems*, Phys. Rev. A **58**, 2733 (1998). doi:10.1103/PhysRevA.58.2733 · arXiv:quant-ph/9803057
- [Wahl 2023] M. Wahl et al., *Zero noise extrapolation on logical qubits by scaling the error correction code distance*, IEEE International Conference on Quantum Computing and Engineering (QCE 2023). arXiv:2304.14985
- [Wallman 2016] J. J. Wallman, J. Emerson, *Noise tailoring for scalable quantum computation via randomized compiling*, Phys. Rev. A **94**, 052325 (2016). doi:10.1103/PhysRevA.94.052325 · arXiv:1512.01098

- [Yoshioka 2022] N. Yoshioka, H. Hosaka, T. Kobayashi, S. Endo, Y. Suzuki, *Generalized quantum subspace expansion*, Phys. Rev. Lett. **129**, 020502 (2022). doi:10.1103/PhysRevLett.129.020502 · arXiv:2107.02611

Part V · Security, robustness and troubleshooting

Security measures

IBM Quantum token.

- The token is entered only in a password field (`st.text_input(..., type="password")`) inside a form.
- It is read in the submit callback, the field is cleared immediately, and the only reference is dropped when the callback returns. It is never kept in the session between steps: collecting results asks for it again.
- It is excluded from every download: exported `.py` scripts, `.json` state files and ZIP bundles.
- Error messages are passed through a scrubber that removes the token before anything is displayed or logged.
- Exported scripts read `IBM_QUANTUM_TOKEN` (or `QISKIT_IBM_TOKEN`) from the environment or use a saved account, and only report which of the two they used.

> A token is a password. Before this notebook goes into a repository or an appendix, clear it and use one of the other two.

Input sanitization.

- Gate parameters and every number typed as text are evaluated by a whitelist interpreter over Python's syntax tree — never by `eval` — with length and exponent limits.
- The text-to-gate parser accepts only the listed gate and noise names (comments after `#` are stripped per line before statements are split); the drag-and-drop composer's actions are validated in Python before they change the circuit.
- The Hamiltonian parser accepts only Pauli factors, numbers and the whitelisted expression grammar (at most 4000 characters, 256 terms, $|c| \le 10^9$) and refuses ambiguous input rather than guessing. Typed layouts accept ASCII digits only; extra symmetry checks accept digits, spaces, commas and semicolons (at most 200 characters).
- Sweeps and register inputs accept only known names (`input`, `ramp`, `basis`, template and noise parameters) and whitelisted value characters; numbers go through the same expression reader (at most 2000 characters, 6 swept quantities, 64 values each).
- Uploaded state files are limited in size and validated field by field; every error becomes a readable message. Validation covers types, bounds (probabilities in $[0, 1]$, shot counts, optimization levels, method parameters within their schema) and consistency (observable length $= 2^n$).
- Labels are reduced to a safe character set; text that comes from a user, a file or an exception is escaped before it is rendered, so it can never inject links, images, HTML or LaTeX.

Robust execution without hard limits. There is no hard limit on qubits or depth. Instead:

- every simulation runs inside `try/except`; out-of-memory errors, time-outs and any unexpected exception become an `st.error` with an explanation, never a crash (stack traces are hidden from visitors);
- before heavy runs the app estimates the memory of a density matrix ($16 \cdot 4^n$ bytes) and warns;
- the sidebar's **time limit** stops runs cooperatively between operations, including the optimization of variational template angles;
- very long circuits are drawn in a window of columns around the cursor so the browser stays responsive;
- batch runs isolate every scenario in its own `try/except` and apply the sidebar's time limit to

each scenario; parameter scans show their progress and the time left.
- a crash inside native code cannot be caught by `try/except`, so it is prevented instead: every entry into Qiskit (compilation, circuit construction, Aer, drawing) holds one process-wide lock, because Streamlit runs each browser session in its own thread; and one session's runs never overlap (`runner.fastReruns = false`), so a run interrupted by a widget change finishes before the next one starts.

Troubleshooting

Symptom	What to do
Results are out of date	Press **Run simulation** (or the page's Run button). Results are never recomputed silently.
A run takes very long	Set a **time limit** in the sidebar; reduce qubits, PEC instances or training circuits; use the NumPy simulator.
budget too small in the status column	The method's training shots (plus one shot per calibration circuit) exceed the shot budget; raise the budget or lower the training shots.
Heavy run warning	The density matrix needs a lot of memory; the app will catch an out-of-memory error, but the machine may slow down.
PNG/SVG downloads fail	Install Chrome or Chromium (Kaleido needs it); meanwhile the ZIP holds each figure's Plotly JSON.
Could not build this template	A parameter combination is invalid (e.g. a 3-regular graph with an odd number of nodes); the message says why and the previous circuit is kept.
CDR/NIL *not applied*	The observable does not vary over any training set (see Tutorial 7); choose another observable or method.
Hardware submission fails	Check the channel (`ibm_quantum_platform`), the instance and that the backend has enough qubits; the message is shown without the token.
The exported script stops a Jupyter cell	Use the script generated by version 1.0: it detects IPython and never calls `sys.exit` there.

Testing

The test suite (`pytest`, 293 tests in version 1.1) covers the physics identities, agreement between the NumPy and Aer executors, exactness of every method, the templates (including numerical checks of the codes and the variational optima), the security checks, every page through Streamlit's AppTest (state survives reruns and page switches, the token is never kept), and the exported script both in a terminal and in a Jupyter kernel. Version 1.1 adds tests for the compile diff and lattices, the layout search, Hamiltonian parsing and grouping, crosstalk with DD, live parameters and scans, batch runs, the Learn page, the new codes (every single-qubit error corrected by the five-qubit code; every single error detected by the surface plaquette), and the analysis engine — including the invariant that the observable summed over each mitigated distribution reproduces that method's estimate — plus a regression test for every finding of two independent QA reviews.